Bitwise-Reproducible Reduction Across Cluster Reshape: A Compiler/Fabric ABI

Siddharth Patel, Aman Awasthi, Akshansh Chaurasia, and Rohit Singh

***Abstract*—Floating-point reductions executed across multi-accelerator systems are not bitwise reproducible under *cluster reshape*: when the number of processing elements G or the physical topology changes, the underlying collective's reduction order changes, and so do the last bits of the result. We measure that premise rather than assume it: reducing identical input *bytes* at fourteen rank counts from 1 to 64 under PyTorch's collectives returns fourteen distinct 64-bit roots, spreads reaching 5,120 ulp, while repeats at a fixed shape are bitwise identical — the sweep running above $G{=}2$ on the CPU backend, because NCCL requires one rank per device and the node has two GPUs. Rerunning the same deterministic computation on a resized cluster therefore breaks bit-for-bit regression testing and audit-grade replay. We argue that reduction order is an *interface* concern rather than a library concern, and treat it as a compiler/fabric Application Binary Interface: a size-indexed contract that fixes the operand binding at every internal node of a canonical full binary tree $\mathrm{FBT}(N)$. Bitwise reproducibility under a change of resource count is not new, and no conjunct below is ours alone; what we claim is the conjunction. The order is fixed at the compiler/fabric interface and enforced in hardware by scoreboard-gated operand binding, so a violated assumption fail-stops rather than returning a quietly different number; the result is invariant across network *topologies* and not resource counts alone; the operator is an unmodified IEEE-754 FP64 addition with no wide accumulator; and N is arbitrary rather than a power of two. Given the ABI, invariance is close to definitional: if every `COMPUTE` always adds the same predetermined pair of addresses, neither timing nor placement can reach the result. What is not definitional, and what we test, is whether the discipline survives contact with a real compiler search space and real RTL. We instantiate it as SPRS: a topology-parametric SystemVerilog network-on-chip with a four-opcode reduction ISA and a combinational flush-to-zero/round-to-nearest-even FADD, plus a seven-stage compiler of which this campaign exercises five. We state two properties under six compiler obligations and nine fabric invariants — *cross-topology bitwise invariance* (Claim 1) and *presence-gated fail-stop* (Property 1) — connected by an explicit *mapping/topology orthogonality* theorem. The compiler's algorithmic configuration space spans $18 \times 10 \times 122 \approx 22{,}000$ (assignment, mapping, refinement) triples per instance; the campaign enumerates the refinement-free face of that space *exhaustively* — all 180 assignment/mapping pairs on each of 40 instances, 0.82% of the triples, with refinement fixed at `None` — and RTL-validates every distinct program image produced on the 39 hardware-validated instances. Of 2,844 images attempted on 39 hardware-validated instances spanning $N \in [8, 8192]$, $G \in [2, 2048]$ and six direct-network topologies, 2,822 settled and every one produced a root bit-identical to the canonical root; 22 did not settle, all but one a fabric watchdog fail-stop, and no run produced a finite root differing from the canonical one. An injected-fault campaign establishes oracle non-vacuity on 61–65 of 107 verdicts, the remainder being fabric fail-stops rather than value comparisons. Two defect classes in our own artifact were nonetheless invisible to that campaign *by construction* — the leaves are all positive, so no run asserts the adder's effective-subtraction path, and the oracle is a transcription of the RTL adder that carried the identical defect — and we report that as a finding about exhaustive evaluation rather than an erratum. Because the compiler's search space cannot reach the bit result, we report what it can reach: a cross-validated portfolio of 4 of the 180 configurations attains the hardware-measured optimum on 38 of 39 held-out instances and, refitted with an entire topology withheld, has mean out-of-sample regret 1.0391 on the topology it never saw (95% bootstrap interval $[1.0025, 1.0885]$) — better than a rule that conditions on topology and does see it in training (1.0488). Every regime map we could fit over $(N, G,$ topology$)$ is worse out of sample than not conditioning at all. We price the ABI structurally rather than against a non-deterministic baseline we did not build, and on this fabric the structural price is small: the ISA's `COMPUTE` is two-input, so no same-fabric reducer is shallower than $\mathrm{FBT}(N)$, and wherever N and G are both powers of two — 31 of the 39 hardware-validated instances — recursive doubling over contiguous leaf blocks is DAG-isomorphic to the ABI schedule, so $\eta = 1$ there is a theorem rather than a measurement we failed to take. What remains is leaf-generation serialisation and root-gather fan-in, which bind any reducer here. Finally we export the discipline to production collectives in software — *pinned-FBT* — and price it. The index-range construction we published is exact only at power-of-two N and G; the cut-aligned repair for arbitrary N given here is bitwise reshape-invariant on all 2,414 (case, world size, rank) cells swept, each returning the canonical root, with a pre-registered negative control that fires on the unrepaired form. It is not cheap: the pre-registered statistic against stock `ncclAllReduce` at $G{=}2$ in FP64 is a geometric mean of $2.57\times$ (best $1.49\times$, worst $3.91\times$; provisional, the sweep having run under GPU contention), a factor of about two of it the algorithmic price of the pinned tree at two ranks. The export is usable where reshape reproducibility is the requirement and the collective is not the bottleneck, and not inside a bandwidth-critical loop.**

***Index Terms*—Deterministic reduction, collective communication, compiler/fabric co-design, IEEE 754 floating point, network-on-chip, bitwise reproducibility, all-reduce.**

I. Introduction

DISTRIBUTED reductions are the connective tissue of modern parallel computation: gradient all-reduce in synchronous data-parallel training [1], [2], partial-sum aggregation in scientific codes [3], logits reduction in pipelined inference. In each case, a multiset of per-worker partial sums is collapsed into a single scalar or tensor through a collective whose performance has been the subject of more than a decade of aggressive engineering [4]–[7]. Performance, however, is not the same as determinism, and the two are in tension. Production implementations of all-reduce — ring, recursive

The authors are with the Department of Electrical Engineering, Shiv Nadar University, India. E-mail: {sp839, aa634, ac198, rohit.singh}@snu.edu.in

Manuscript received September 4, 2026; revised X, XXXX.

halving–doubling, double-binary tree, hierarchical hybrid — select a reduction order that maximises throughput for the observed network topology and worker count [5], [8]. When either changes, so does the order.

IEEE 754 double-precision addition is non-associative: $(a+b)+c \neq a+(b+c)$ in the low-order bits whenever alignment shifts a non-zero mantissa below the round bit. The classical bound [9] gives, for two reduction orders σ and σ' of the same multiset,

$$\left| \sum_{\sigma} x_i - \sum_{\sigma'} x_i \right| \;\leq\; (N-1)\, u \sum_i |x_i| + \mathcal{O}(u^2), \qquad (1)$$

with u the unit in the last place. The classical statement bounds one summation order against the exact sum; applying it to the difference of two orders costs a factor of two by the triangle inequality, which we absorb into the constant since only the scaling matters here. Under cancellation this bound is catastrophically loose and the actual discrepancy is a function of the summation tree. The consequence is concrete, and we measure it rather than assert it (§VII-B). Reducing identical input *bytes* at fourteen rank counts from 1 to 64 under PyTorch's distributed collectives [2] — same data, same seed, same code, only the shape of the job changing — returns fourteen distinct 64-bit results, with spreads reaching 5,120 ulp and at least 1,023 of the 1,024 elements of the result vector moving in every case, while repeated runs at any *fixed* shape are bitwise identical five times out of five. Reshape moves the answer; noise does not. The rank-count sweep above $G{=}2$ runs on the CPU backend, because NCCL requires one rank per device and this node carries two GPUs; §VII-B states that scope in full and draws no timing claim from the measurement. In a training loop the difference does not stay local: the reduced gradient becomes the next step's weights, so a bit-level divergence at the first step is an input difference at the second. Bit-for-bit regression testing of scientific codes fails under the same mechanism: a code that ran to acceptance on a development cluster produces different last bits on the production cluster, and the reviewer cannot distinguish a new bug from the reshape artefact. In regulated deployments — medical-device inference, aerospace controls, financial model replay — the inability to reproduce a computation bitwise across hardware generations is increasingly disqualifying.

A. Two Sources of Non-Determinism, One Bad Outcome

The cluster-reshape non-determinism we target has two distinct sources, which current software tools confuse.

Source 1 — order-dependence of the reducer. The collective chooses a reduction tree or ring shape as a function of the runtime environment. In NCCL, for instance, the (algorithm, protocol) pair is selected at communicator initialisation by an internal cost model parameterised by GPU and CPU architecture, node and rank counts and message size [5]; the data are never consulted. Even with all randomness suppressed, the algorithm is therefore a function of $(G, \text{topology})$, not of the input, and reshaping the deployment changes it deterministically.

Source 2 — reassociation inside the reducer. Within a chosen algorithm, performance-critical implementations reorder partial sums to hide latency — overlapping pipeline segments, exploiting SIMD lanes, merging small messages. Each micro-optimisation is an implicit reassociation, and each changes the last bit.

Both sources have been attacked before, and two of the existing answers reach further than is usually acknowledged. Order-independent accumulators — ReproBLAS [10], ExBLAS [11], and the bit-reproducible distributed reductions of Arteaga et al. [12] — eliminate both sources at cluster scale by making the operator effectively associative; their price is that the answer is no longer the IEEE-754 FP64 sum that any particular addition order would produce, and that a wide accumulator datatype is required. Fixed-order schemes keep the FP64 operator and pin the order instead, but each along one axis inside one implementation: Intel oneMKL's strict conditional-numerical-reproducibility mode is bitwise identical regardless of thread count and input partitioning [13], and TBIK [14] is bitwise identical across tensor-parallel size, while the "deterministic" modes of production collectives hold only within one $(G, \text{topology})$ [5], [15]. Compensated summation [16], [17] reduces the error of a reduction without fixing either source. Section IX states each of these relationships precisely.

What no line of work states is the invariant *at the interface*, enforced below the software that could violate it. What we set out to obtain is therefore a conjunction rather than a first: reduction order fixed as a *size-indexed contract* between the collective-scheduling compiler and the fabric, so that the 64-bit root is invariant not only under worker count but under network topology, task assignment, physical placement, routing and schedule; with the operator an unmodified IEEE-754 FP64 addition and no wide accumulator; enforced in hardware by scoreboard-gated operand binding, so that a violated assumption fail-stops instead of returning a quietly different number; and holding for arbitrary, not merely power-of-two, leaf counts. Every conjunct has a precedent; the conjunction, as far as we have been able to determine, does not.

B. Position: Reduction Order Is an Interface Concern

We take the position that bitwise reproducibility under cluster reshape cannot be achieved inside any single layer of the conventional software stack, and must instead be moved into the *interface* between the collective-scheduling compiler and the underlying fabric. A library-level flag cannot forbid Source 1, because the library has no authority over whether a different cluster size will be launched tomorrow. A fabric-level guarantee alone cannot forbid Source 2, because the fabric cannot see the reduction tree a scheduler intends to use. The discipline has to be a contract shared by both.

The principle is not new; its reach is. IEEE 754-2019 already makes reproducibility an interface obligation rather than an implementation property: reproducible results, it states, require cooperation from language standards, language processors and users, and the reproducible attribute it specifies forbids the processor from reordering or fusing operations [18]. That obligation binds language processors, within one program; it says nothing about how many workers run a collective

or how they are wired. We extend it to the interface the standard does not reach — between a collective-scheduling compiler and the fabric — and index it by problem size, so that it survives a change of deployment.

Concretely, let the operation be:

$$R = x_0 \oplus x_1 \oplus \cdots \oplus x_{N-1}, \qquad \oplus \triangleq \text{IEEE-754 FP64 ADD.} \tag{2}$$

We define a size-indexed contract: for every internal node v of a canonical full binary tree $\mathrm{FBT}(N)$, the compiler emits exactly one compute instruction whose operand pair is $(\mathrm{left}(v), \mathrm{right}(v))$, and the fabric's arithmetic unit reads those operands at a single clock edge after a scoreboard gate confirms both have been written. The contract is indexed only by N; it says nothing about which processing element hosts which node, how packets are routed, or when any instruction fires. These are left free precisely so that schedulers may continue to optimise performance without touching the contract.

Under this discipline, the observed 64-bit root result is a pure function of N and the leaf vector L, irrespective of $(G, \text{topology})$, assignment α, mapping μ, refinement r, or timing. We call this property *fabric self-consistency under cluster reshape*. To be clear, we do not claim that this result agrees with any external reducer's output — that would be a different (stronger) claim, requiring a shared canonical form. We claim that any two executions of our system on the same L agree bit-for-bit, under any reshape within a declared family $\mathcal{T}$ of flat direct-network topologies.

C. Contributions

This paper makes three contributions, supported throughout by a compile-time post-condition on the emitted image, root equality against a software oracle on every validated run, and a mutation campaign that shows the oracle is not vacuous (§X states what that oracle can and cannot witness). We use "mechanised" nowhere in its formal-methods sense: the invariance argument is by code inspection and dynamic checking, not machine-checked proof (§III-I).

C1 — Reduction order as a size-indexed compiler/fabric ABI, with a structural invariance argument. We claim the framing, not the difficulty: once operand binding is a function of tree-node identity alone, invariance follows by unwinding the definitions, and we say so rather than dressing it as a discovery. We define the ABI as post-order operand binding over $\mathrm{FBT}(N)$ together with a canonical leaf enumeration, and we state two properties that follow from it. *Claim 1 (cross-topology bitwise invariance)* establishes that, under six compiler obligations and nine fabric invariants, the 64-bit root result is a pure function of (N, L) — unchanged by worker count, network topology, assignment, placement, routing or schedule, with the operator an unmodified IEEE-754 FP64 addition and no wide accumulator, for arbitrary non-power-of-two N. *Property 1 (presence-gated fail-stop)* establishes that any assumption violation leaving an operand absent yields no root at all — a localised `P_ERROR` with diagnostic context, and no completion — and we state the boundary of that property rather than claiming the fabric catches everything. The two are connected by an explicit *mapping/topology orthogonality* theorem. Bitwise reproducibility under a change of resource count is not itself new (§IX); what C1 claims is the conjunction of that invariant with this enforcement locus, this failure mode, this range of N, and invariance across network topologies rather than resource counts alone.

C2 — A compiler and fabric that jointly realise the ABI. The SPRS compiler translates $(N, G, \text{topology})$ into a per-PE instruction image through the five stages this campaign exercises: a composite lower-bound Ω (S0); $\mathrm{FBT}(N)$ instantiation (S1); task-to-worker *assignment* selected from eighteen algorithms spanning list-scheduling, graph partitioning, tree dynamic programming, and critical-path variants (S2); worker-to-physical-PE *mapping* from ten algorithms (S3); and code emission with cycle-accurate NOP padding (S6). Three further passes exist in the compiler but produced no result reported here: connectivity repair (S2.5), memory enforcement against the 512-entry IMEM/DMEM budgets (S5), and refinement (S4, twelve base methods chainable to 122 configurations), fixed at the identity throughout. Operand binding is by node identity, so it is indifferent to whether an assignment leaves a worker's task set connected, and the invariance result therefore quantifies over the raw output of all eighteen assignment algorithms. The fabric is a topology-parametric SystemVerilog NoC with a four-opcode reduction ISA, a three-stage IF–DE–EX pipeline with two-level DMEM write-to-read forwarding, a scoreboard gate at DE, and a combinational IEEE-754 adder implementing flush-to-zero subnormals and round-to-nearest-even rounding. Topology is a runtime configuration of adjacency and routing tables, not an RTL artefact.

C3 — Hardware validation at scale, and a structural pricing of the ABI. We evaluate on 40 instances spanning sixteen leaf counts $N \in [8, 8192]$, requested worker counts $G \in [2, 4096]$, and six direct-network topologies (linear, ring, mesh, torus, fat-tree, hypercube). The compiler's algorithmic configuration space spans $\approx 22{,}000$ (assignment, mapping, refinement) triples per instance; the campaign enumerates the refinement-free face of that space *exhaustively* — all 180 assignment/mapping pairs on every instance, 7,200 compilations, 0.82% of the triples, with refinement fixed at `None` — and RTL-validates with cycle-accurate Vivado xsim every distinct program image produced on the 39 hardware-validated instances. Of 2,844 images attempted over 39 hardware-validated instances, 2,822 settled and in every one the root PE's final result is bit-identical to the canonical root $R_{\mathrm{canonical}}(N, L)$ the software oracle predicts for that N; 22 did not settle, all but one a fabric watchdog fail-stop, and no configuration produced a finite root differing from $R_{\mathrm{canonical}}(N, L)$. Mutation testing under injected address-substitution and leaf-corruption faults shows the oracle is not vacuous, and separates fail-stop from wrong-root outcomes exactly where Property 1 places the boundary. We price the ABI structurally rather than against a non-deterministic baseline: the fabric's ALU is two-input, so no same-fabric reducer is shallower than the canonical tree, and what the ABI can cost is leaf-generation serialisation and root-gather fan-in, which bind any reducer here. Finally, because the compiler's search space is provably incapable of changing the bit result,

we report what it *can* change: a cross-validated portfolio of 4 of the 180 configurations attains the hardware-measured optimum on 38 of 39 held-out instances, while every regime map we could fit over $(N, G, \text{topology})$ does worse out of sample than not conditioning at all. Separately, we export the discipline to production collectives in software — *pinned-FBT* (§VIII-D) — correcting the index-range construction we published, which is exact only at power-of-two N and G, and measuring the cut-aligned repair for arbitrary N on both counts that matter: bitwise reshape-invariance against a negative control that fires, and cost against stock `ncclAllReduce`.

D. Non-Goals

To forestall mis-scoping, we state explicitly what this paper does *not* claim.

Interoperability. SPRS results are self-consistent under reshape for a fixed SPRS/fabric pair. They do not match NCCL, RCCL, ReproBLAS, ExBLAS, or any external reducer whose canonical order differs. Interoperability requires either a thin input-permutation adapter (at zero runtime cost, trading the cross-reshape guarantee) or a shared canonical form — a direction we sketch in §VIII.

Strict IEEE 754. The FADD unit implements FTZ subnormals on input and output, RNE rounding only, a deterministic but non-canonical qNaN propagation rule, and a deterministic signed-zero rule that returns $+0$ on cancellation. These choices are documented (§III-G) and strictly narrower than IEEE 754-2019 [18]; a subnormal-aware alternative inherits Claim 1 by construction.

Silicon scalability. The fabric is a research prototype characterised by cycle-accurate RTL simulation. We make no claim of timing closure, area, power, or frequency at production-die scale, and we do not benchmark SPRS against production all-reduce libraries running on NVLink/NVSwitch silicon. The one head-to-head we do report — the software export of §VIII-D against stock `ncclAllReduce` on a PCIe pair — prices that export, and not the fabric.

Novel component algorithms. Our eighteen assignment algorithms, ten mappings, twelve refinements, minimal-distance routing, credit-based flow control, and scoreboarded pipeline are all individually due to prior work [19]–[26]. Our contribution is not a new component but the interface observation: that *if* compiler and fabric agree on a size-indexed reduction tree as a shared contract, *then* any choice among those prior-art components inherits cross-topology invariance by composition. The entire compiler variation space is structurally orthogonal to the ABI; the campaign searches a space incapable of changing the bit result.

E. Roadmap

Section II fixes notation for IEEE 754 non-associativity and the threat model, then tabulates prior reproducibility approaches. Section III is the paper's theoretical core: the canonical tree, the ABI definition, the compiler-side and fabric-side assumption lists, Claim 1, Property 1, and the orthogonality proposition. Section IV describes the fabric — ISA, pipeline, write-arbitration, fail-stop discipline, and resource envelope. Section V describes the compiler, with particular emphasis on the pass that constructs the compiler-side obligations rather than assuming them (SU-ordered DMEM allocation) and on why connectivity repair, which looks load-bearing, is not. Section VI describes the evaluation methodology (instance suite, campaign protocol, mutation testing, metric convention). Section VII measures the premise on a production collective, then presents the cross-topology bitwise invariance result, the mutation-detection campaign that shows the oracle is non-vacuous, a structural pricing of the ABI with the measured distance to Ω, and the selection rule the campaign licenses. Section VIII discusses scope, generalisation, and the software export of the discipline onto production collective libraries — *pinned-FBT* — whose published form we correct there, whose repair we measure for both correctness and cost, and whose relation to prior art we state. Section IX covers related work. Sections X and XI state limitations and conclude. Appendices A–G provide the ISA/packet specification, the full algorithm catalogue, a worked invariance example, per-instance coverage and results, a selection-bias ablation, and a reproducibility checklist.

II. Background and Threat Model

A. IEEE 754 Non-Associativity

Let $x_0, x_1, \ldots, x_{N-1}$ be N binary64 values and let σ be a summation order induced by a binary reduction tree τ. Denote by $\Sigma_\tau(x)$ the 64-bit result of evaluating τ with RNE rounding at every internal node. For two trees τ, τ', the worst-case componentwise bound (1) is loose in the generic case but tight in the presence of cancellation: when $\sum_i x_i = 0$ by exact real arithmetic but intermediate partial sums differ in magnitude, the last bits of Σ_τ and $\Sigma_{\tau'}$ can differ by an exponent-magnitude of the largest partial sum. The practical consequence for reductions of noisy tensors is that two trees executing on identical x will, with overwhelming probability, produce different last bits of Σ.

Production collectives choose τ as a function of their schedule. Ring all-reduce [27] orders along a physical worker ring; a recursive halving–doubling all-reduce [8] orders by a power-of-two recursive pattern; double-binary-tree all-reduce [28] uses two interleaved binary trees. Runtime-hybrid collectives [5], [15] select among these per message size, using topology discovered at communicator initialisation and an internal cost model parameterised by architecture and rank count. In each case τ is a function of $(G, \text{topology}, \text{message size})$ and never of the data. Reshaping the cluster reshapes τ and therefore Σ_τ.

B. Threat Model

We target a specific, precisely stated reproducibility property.

Definition 1 (Cross-topology bitwise reproducibility)**.** Let $L \in (\text{bits}_{64})^N$ be a fixed leaf-value vector. For every $(G, \text{topology}) \in \mathcal{T}$ (the declared family of supported direct networks), every valid SPRS compilation, and every valid runtime execution of the fabric, the observed 64-bit root result is the same sequence of bits.

The family $\mathcal{T}$ contains flat direct networks whose adjacency can be realised by a configuration of the per-router adjacency and routing tables: linear, ring, 2D mesh, 2D torus, hypercube, and fat-tree. The property is *structural* — it is a statement about what the fabric–compiler pair can be proven to guarantee, not an empirical average over executions.

Out of scope: (i) agreement between SPRS and external reducers at different canonical orders; (ii) tolerance to single-event upsets or transient hardware faults; (iii) cross-fabric portability of bit patterns between fabrics with different FADD microarchitectures; (iv) adversarial traffic patterns affecting cycle counts (these cannot affect the bit result; head-of-line behaviour is discussed in §IV-J).

C. Prior Reproducibility Approaches

Table I classifies prior approaches. Two groups already deliver invariance under a change of resource count, and we do not claim otherwise: order-independent accumulators do it by changing the operator, and fixed-order schemes do it with the operator intact but each within one library and along one axis. What no row other than ours combines is invariance across network *topologies* rather than resource counts alone, an unmodified FP64 operator, and enforcement below the software stack with a fail-stop failure mode. §IX treats each row in turn, including the collective-scheduling compilers [6], [7], [33] that optimise the very schedules the ABI leaves free.

III. The Reduction ABI and the Invariance Claim

This section is the paper's theoretical core. We define the canonical reduction tree (§III-A), state the ABI as a pair of binding disciplines (§III-B), and enumerate the fifteen assumptions the two properties rest on, partitioned into compiler obligations and fabric invariants (§III-C). Eleven of the fifteen are hypotheses of cross-topology bitwise invariance (Claim 1); the other four are liveness invariants used only by the presence-gated fail-stop property (Property 1). Which assumption belongs where is not a matter of taste — we state below, for each one, the failure it does and does not prevent, because that is what a reader reimplementing the ABI needs. The supporting arguments are given in §III-E. A key corollary — that the mapping and topology choices are *orthogonal* to the bit result — is stated and argued as an explicit proposition in §III-F. We close with the FADD policy (§III-G), edge cases (§III-H), and a discussion of runtime checks and a path to mechanisation (§III-I).

A. The Canonical Tree

Definition 2 (Canonical full binary tree)**.** For $N \geq 1$, the *canonical full binary tree* $\mathrm{FBT}(N)$ is the heap-indexed binary tree with $2N-1$ nodes: root at index 0, left child of i at $2i+1$, right child at $2i+2$. The set of internal nodes is $\{0, 1, \ldots, N-2\}$; the set of leaf nodes is $\{N-1, \ldots, 2N-2\}$. Since $2N-1$ is always odd, every internal node has exactly two children: the single-child edge case is eliminated at the tree level.

Definition 3 (Canonical leaf enumeration)**.** Let $\mathcal{L}(N) \subset \{0, \ldots, 2N-2\}$ be the leaf set of $\mathrm{FBT}(N)$. The canonical enumeration $\mathrm{idx} : \mathcal{L}(N) \to \{0, \ldots, N-1\}$ assigns index k to the k-th smallest *heap index* in $\mathcal{L}(N)$.

This is worth stating precisely, because the obvious informal paraphrase is wrong. Ascending heap index and left-to-right *visual* order coincide exactly when N is a power of two. When it is not, $\mathrm{FBT}(N)$ is depth-imbalanced: some leaves sit one level higher than the rest, and a higher leaf carries a *smaller* heap index than the deeper leaves to its left. For $N = 7$ the leaf set is $\{6, \ldots, 12\}$; ascending heap index gives $\langle 6, 7, 8, 9, 10, 11, 12\rangle$, whereas left-to-right traversal gives $\langle 7, 8, 9, 10, 11, 12, 6\rangle$ — node 6 is a leaf at depth two and appears *last* visually. Seven of the sixteen leaf counts we evaluate $(10, 50, 100, 200, 500, 1500, 3000)$ are of this shape, and at $N = 200$ the two enumerations produce root values differing by one ulp. The ABI is defined by heap index; an implementation using visual order is a *different* contract, not an equivalent restatement of this one.

The post-order traversal of $\mathrm{FBT}(N)$ — left, right, node — is a pure function of N. We write π_N for that traversal restricted to internal nodes: $\pi_4 = \langle 1, 2, 0\rangle$, i.e. internal node 1 (children $3, 4$) is evaluated first, then internal 2 (children $5, 6$), then the root 0 (children $1, 2$). The canonical evaluation is

$$R_{\mathrm{canonical}}(N, L) \;=\; \mathrm{evaluate}\big(\mathrm{FBT}(N), L, \mathrm{fadd}_{\mathrm{rtl}}\big), \quad (3)$$

where L is the leaf-value vector indexed by the canonical leaf enumeration of Definition 3 and $\mathrm{fadd}_{\mathrm{rtl}}$ denotes the combinational semantics of our FP64 adder (§III-G). Equation (3) is the *target* of every SPRS execution on input (N, L).

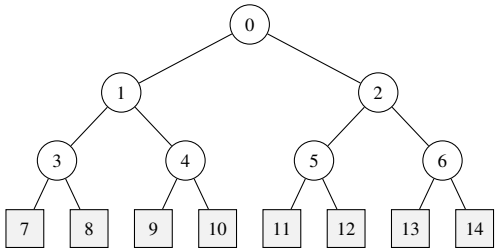
The canonical leaf enumeration idx of Definition 3 is likewise a pure function of N, and is the referent for every use of "leaf index" below.

The canonical order is a constraint, not a tautology. A reader may reasonably suspect that bitwise invariance is definitional — that any association order would have produced one root value per N, so agreement across topologies measures nothing. It is not, and we measured how much it is not. On the leaf vectors this paper evaluates (§VI-D), re-associating the same N addends moves the 64-bit root: right-to-left sequential summation differs from $R_{\mathrm{canonical}}$ by up to 55 ulp at $N = 8192$, and left-to-right sequential summation differs from $R_{\mathrm{canonical}}$ at twelve of the sixteen leaf counts in the suite. The discriminating power is strongly N-dependent and is not monotone in N. Exhaustively at $N = 8$, all 40,320 leaf permutations yield only three distinct roots, of which 88.6% equal the canonical one, so a uniformly random leaf-order violation is detected with probability 0.114; at $N = 16$, all 9,694,845 in-order tree shapes yield four distinct roots spanning two ulp. Sampling association orders uniformly, the probability that a reassociation moves the root ranges from 0.08 to 0.90 across the sixteen leaf counts. Two consequences follow, and we adopt both. The oracle is non-vacuous, so an agreeing run is evidence; and the rows of the invariance table (§VII-C) are not equally informative — at $N = 8$ and $N = 16$ agreement is close to free, and the weight of the evidence for Claim 1 sits at the large leaf counts.

TABLE I
REPRODUCIBILITY APPROACHES, BY WHAT FIXES THE ORDER, WHAT THE RESULT IS THEN INVARIANT UNDER, AND WHETHER THE FP64 OPERATOR SURVIVES. SOURCE 1 = ALGORITHM SELECTION AS A FUNCTION OF $(G, \text{TOPOLOGY})$; SOURCE 2 = REASSOCIATION INSIDE THE ALGORITHM. ✓ = THE SOURCE CANNOT CHANGE THE BITS WITHIN THE STATED SCOPE; *vac.* = MADE VACUOUS BY AN ORDER-INDEPENDENT OPERATOR RATHER THAN CONSTRAINED.

Approach	Order-fixing mechanism	Invariant across	Src 1	Src 2	FP64 op.
ReproBLAS [10]	Binned accumulator (order-independent)	Processor count, partition, MPI tree shape	✓ vac.	✓	no
ExBLAS [11], [29]	Long accumulator + FP expansions	Processor count, partition; two clusters	✓ vac.	✓	no
Kahan / pairwise [9], [16]	Compensated arithmetic	Nothing: error is reduced, bits are not	–	$\sim^{\dagger}$	yes
oneMKL strict CNR [13]	Fixed operation order in a shipping BLAS	Thread count, input partitioning; one node	✓	✓	yes
TBIK [14]	One fixed tree, intra- + inter-GPU	Tensor-parallel size; power-of-two, one node	✓	✓	yes
Pinned collectives [5], [15]	Fixed algorithm + single channel	One $(G, \text{topology})^{\star}$	✓*	✓	yes
SHARP [30], [31]	Operand order pinned in the switch	One application-to-system mapping*	✓*	✓	yes
Groq TSP [32]	Software-scheduled routing, no arbitration	One compiled system configuration‡	$\sim^{\ddagger}$	–	n/a
SPRS (this paper)	**Size-indexed $\mathrm{FBT}(N)$ ABI, scoreboard-gated in RTL**	**All $(G, \text{topology}) \in \mathcal{T}$; assignment, placement, routing, schedule**	✓	✓	**yes**

† Reduces the error bound, not the bit pattern: the same values in a different order, or with a different number of lane accumulators, still differ in the last bit. * Within one deployment only — SHARP is reproducible "for a given application-to-system mapping" [31], and NCCL selects its algorithm from rank and node counts. ‡ Schedule determinism only: no bitwise claim is made, and the all-reduce is keyed to the packaging hierarchy.



$\pi_8^{\text{internal}} = \langle 3, 4, 1, 5, 6, 2, 0 \rangle$

leaf enum: $7 \mapsto 0,\ 8 \mapsto 1, \ldots, 14 \mapsto 7$

Fig. 1. The canonical tree $\mathrm{FBT}(8)$. Circles are internal nodes (heap indices 0–6); rounded squares are leaves (heap indices 7–14). The internal-node post-order $\pi_8^{\text{internal}} = \langle 3, 4, 1, 5, 6, 2, 0 \rangle$ and the leaf enumeration idx are both pure functions of N and constitute the ABI's structural referents.

Figure 1 illustrates $\mathrm{FBT}(8)$ with its heap indexing, internal-node post-order, and leaf enumeration. The compiler constructs $\mathrm{FBT}(N)$ by the `CBTree` class at `sprs_core.py:380--482`; the class name is historical, the implementation constructs a full binary tree.

B. The ABI: Operand Binding and Leaf Binding

The ABI is a contract between compiler and fabric comprising two disciplines.

Definition 4 (Operand binding)**.** For every internal node v of $\mathrm{FBT}(N)$, the compiler emits exactly one `COMPUTE` instruction on the PE hosting v, with

$$\texttt{addr\_a} = \mathrm{dmem}(\mathrm{left}(v)), \tag{4}$$

$$\texttt{addr\_b} = \mathrm{dmem}(\mathrm{right}(v)), \tag{5}$$

where $\mathrm{dmem}(\cdot)$ is a per-PE DMEM address assigned by the allocator. The binding is a pure function of v and depends on nothing else.

Definition 5 (Leaf binding)**.** For every leaf $k \in \mathrm{leaves}(\mathrm{FBT}(N))$, the `GENERATE` instruction on the PE hosting k writes, into the DMEM slot bound to k, the 64-bit value $L[\mathrm{idx}(k)]$, where idx is the canonical leaf enumeration of Definition 3.

The fabric's obligation is to execute the two disciplines symmetrically: `COMPUTE` reads the two DMEM slots at a single clock edge, scoreboard-gated, and writes $\mathrm{fadd}_{\mathrm{rtl}}(\text{value}_a, \text{value}_b)$ to the destination slot; `GENERATE` delivers $L[\mathrm{idx}(k)]$ into its destination slot with the scoreboard set. Critically, neither discipline references G, topology, assignment α, mapping μ, refinement r, or timing. These are left free — and are the object of the tournament's optimisation in §VI.

C. Assumptions

Fifteen assumptions carry the two properties: five *compiler obligations* (C-*), which the compiler or the user must discharge, and ten *fabric invariants* (F-*), which are established by inspection of the RTL. Eleven of the fifteen are hypotheses of Claim 1. The four marked † are not: violating one of them can stop a run, but it cannot alter a root value that the fabric does produce, and Claim 1 is conditional on normal termination. Those four are the hypotheses of Property 1 instead.

Compiler obligations:

C-sw (Single-writer allocation for remote destinations). Partition each PE's DMEM into *receive buffers*, the addresses named as `target_addr` by some `SEND` from another PE, and *local temporaries*, the rest. For every PE p and every receive buffer $a \neq 0$, the emitted image contains exactly one write to (p, a), and that write is remote. Local temporaries are exempt: the allocator reuses them (§V-C), and their safety comes from in-order issue rather than from single assignment — see

Remark III-C. The scoping is load-bearing and every use of C-sw below respects it; the two classes are disjoint by construction.

C-bind (Operand binding, Def. 4).
Every `COMPUTE` for internal node v binds the operand pair by definition. Checked on the emitted image by the post-condition pass (§III-I).

C-leaf (Leaf binding, Def. 5).
Every `GENERATE` for leaf k delivers $L[\mathrm{idx}(k)]$ into the DMEM slot bound to k. Codified by the testbench emitter (`emit_sv_testbench`, `sprs_core.py:5533--5549`).

C-mem (Memory feasibility).
For every PE p, the emitted image names at most `DMEM_DEPTH` $= 512$ distinct DMEM addresses on p; and every router id in the emitted adjacency table is below the disconnected sentinel of the adjacency interface's id field (`noc_system.sv`), which the submitted design fixed at 12 bits, i.e. at most 4,094. Neither bound is enforced by the fabric at run time. The fabric truncates DMEM addresses to $\log_2$ `DMEM_DEPTH` bits (`btree_fsm_fast.sv:82`) and `SEND` target addresses to ten (`btree_fsm_fast.sv:629`), so an image over budget aliases two addresses onto one slot silently; router ids at or above the sentinel alias onto low ids, which is what stopped one instance of the suite from ever settling (L0). Both halves are the compiler's to discharge, and both are cheap: peak occupancy is a quantity the allocator already maintains, so the DMEM half is an $O(N)$ read-off checked before emission (Remark III-C), and the router-id half holds by construction once the emitter sizes the id field to the instance, with an elaboration-time range check in the fabric (`gen_adj_width_check`, `noc_system.sv`) failing the build rather than aliasing if it does not.

† **C-route (Routing-table soundness).**
The emitted routing tables encode destination-correct, loop-free paths on the declared topology. Satisfied by construction for each topology family. We claim destination-correctness and loop-freedom only; the emitted tables are minimal-distance rather than dimension-ordered, so we do not claim an acyclic channel-dependency graph.

Remark (The C-mem margin, and what lies past it). C-mem is the one hypothesis of Claim 1 that the campaign came close to violating, so we state the margin rather than the fact of compliance. Peak DMEM occupancy over the validated configurations reaches 491 of 512 slots, a margin of 21. Past the ceiling the claim fails, and it fails silently: on a constructed $\mathrm{FBT}(512)$ instance with $G = 2$ and all leaves on one PE, peak occupancy is 522 slots, 11 emitted addresses exceed the depth, and the run terminates normally — no `P_ERROR`, no deadlock — returning a wrong root, and two *different* wrong roots under two different packet-delivery orders. Aliasing defeats the presence gate rather than tripping it, because the aliased slot has been written by something. The fabric cannot detect the condition, which is why C-mem is a hypothesis of Claim 1 rather than a property of the machine, and why we report the margin. The compiler can: the emitter now carries a post-condition over the instruction words it is about to write out and raises `EmitRefused` rather than emit an image whose addresses would alias, so the failure above is a recorded compile failure in the released tool and a wrong root only in the version we measured it on. No configuration in the campaign trips that post-condition, which is the same statement as the margin. A checker (`tests/checkers/test_emit_refuses_aliasing.py`) guards it, and its own negative control confirms the refusal is driven by the addresses rather than by a small depth.

Remark (Why address reuse does not break the gate). The allocator of §V-C returns a child's slot to a free list once the parent has consumed it, so a local temporary can be written more than once per program. Read against a scoreboard bit that is never cleared, that looks unsafe: a second writer would find the bit already set, and a reader could not distinguish the new value from the old. Reuse is also not a corner case that Claim 1 could be restricted away from — over 562 compiled configurations we measured 108,607 recycled addresses and 397,422 reuse events, and *every* configuration recycles.

It is safe, and there are exactly two cases, distinguished by how many readers of the old value are still outstanding when the slot is rewritten. (a) In 31.3% of reuse events there are none: the last consumer of the old value has already retired and the slot was returned to the free list, so there is nothing to protect. (b) In the remaining 68.7% there is exactly one, and it is the very `COMPUTE` that overwrites the slot; under F-inorder that instruction samples both operands in DE strictly before it writes back in EX. Over the same configurations no reuse event has a remaining reader other than the overwriting instruction, and none has two, so (a) and (b) are exhaustive. The ordering is supplied by program order on a single PE, not by the scoreboard. The scoreboard exists for the one case program order cannot cover: a value written by a *different* PE, whose arrival time is a function of routing and contention. Those destinations are precisely the receive buffers, and those are never recycled.

Case (b) has a sub-case that program order alone does not cover, and it is the reason F-gci-haz and F-arb appear in Claim 1's hypotheses. `GENERATE` is asynchronous: the instruction only asserts `gci_start` in EX, and the GCI agent commits the leaf value 4 or more cycles later (`noc_ni.sv:26`), against a compiler-emitted gap of 6 cycles at minimum and 12 at the median. 35.4% of `GENERATE`s (29,097 of 82,084 in the instances we instrumented) target a *recycled* slot, and every one of them is followed by a read of that slot. For those the scoreboard bit is already 1, set by the previous writer, so F-sb-gate cannot stall the consumer and cannot be what makes them safe. The in-flight interlock F-gci-haz is, and F-arb's retry latch is what stops a colliding PIU write from discarding the pending GCI write. Remove either and the consumer reads the stale value, with no stall and no diagnostic.

The separation of the two address classes is structural rather than incidental. The allocator assigns receive buffers in a

second pass, from fresh addresses above the local high-water mark, and never from the free list. We verified both facts directly on the emitted images of the same 562 configurations: the set of recycled addresses and the set of remotely-written addresses never intersect, and no operand is ever read after the write that recycles its slot (zero such events out of 397,422).

Fabric invariants:

F-sb-mono (Scoreboard monotonicity).
For every receive buffer a, `scoreboard[a]` transitions $0 \to 1$ at most once per program, at the clock edge of the unique remote write to a, and stays set until the next program-level reset. As with C-sw, the scope is receive buffers: a local temporary's bit is set by its first writer and stays set across every reuse, which is why local temporaries are covered by F-inorder and F-gci-haz below rather than by the scoreboard.

F-sb-gate (Scoreboard gating).
`COMPUTE` stalls DE→EX until both `scoreboard[addr_a]` and `scoreboard[addr_b]` are 1; `SEND` stalls until `scoreboard[addr_a]` is 1 (`scoreboard_hazard`, `btree_fsm_fast.sv:514--521`).

F-fw (DMEM write-to-read forwarding).
A DMEM read at cycle t returns the value committed by the unique write to that address at cycle $\leq t$, even across the adjacent-cycle race. Realised by a two-level address-match forwarding network.

F-inorder (In-order single-PE issue).
Each PE issues its program in order through IF→DE→EX, one instruction at a time, and an instruction samples both operands at the DE read port strictly before it writes its result back in EX (`btree_fsm_fast.sv:548--558` and `:608--621`). This is what makes local-temporary reuse safe (Remark III-C); the scoreboard cannot, because a recycled slot's bit is already set.

F-gci-haz (In-flight `GENERATE` interlock).
From the cycle a `GENERATE` asserts `gci_start` until the GCI agent commits its value, DE stalls any `COMPUTE` or `SEND` naming the pending destination as an operand (`gci_data_hazard`, `btree_fsm_fast.sv:523--534`). The RTL comment calls this belt-and-braces with the scoreboard; it is not. It is the sole protection for the 29,097 `GENERATE`s that target a recycled slot, where the scoreboard bit is already set.

F-arb (Write arbitration).
The DMEM write arbiter is strict-priority PIU $>$ GCI $>$ FSM over the three write agents, with a GCI retry latch that re-asserts the GCI write until accepted (`btree_fsm_fast.sv:364--378`, `:419--421`); no write is silently dropped. Necessary for the root value, not only for liveness, on the recycled-destination path (§IV-G).

F-fadd (FADD purity).
`fp64_add.sv` is combinational and stateless (zero `always_ff`); its output is a pure function of the two 64-bit inputs sampled at the EX clock edge.

† **F-credit (Credit conservation).**
For every link and every interval, $\#\text{credit\_returns} \leq \#\text{buffer\_reads}$ with a bounded wait. Realised by counter-serialised credit pulses that eliminate a multi-VC credit-loss bug present in an earlier design.

† **F-misroute (Misroute drop).**
Packets arriving at a PE whose `dest_gpu_id` header field does not match the local PE are dropped, not silently consumed. Converts a C-route violation from silent corruption into a liveness failure detected by F-watchdog.

† **F-watchdog (Liveness guard).**
A stall exceeding WATCHDOG_MAX $= 20{,}000$ cycles or an EX stall exceeding TIMEOUT_MAX $= 10{,}000$ cycles raises `P_ERROR`; invalid opcodes raise `P_ERROR` immediately.

What the list omits, and why. Three conditions that a reader might expect here are absent, and each absence is a decision we can defend. *Compilation determinism* is not a hypothesis: Claim 1 quantifies over every image the compiler can emit, and a statement true of all of them does not require the compiler to be a pure function of its seed. Fixed seeds and insertion-ordered dictionaries make the campaign re-runnable (§VI); they do not carry the claim. *Partition connectedness* is not a hypothesis either, although 2,061 of the RTL-validated configurations (73.0%) have partitions that are not connected subtrees. Operand binding is by node identity and every cross-PE child gets a dedicated receive slot, so the emitted binding is $(\text{left}(v), \text{right}(v))$ for *any* assignment; we checked this statically on 1,010,895 `COMPUTE` and 731,982 `SEND` instructions across 681 configurations and found zero violations (§V-B). What disconnection actually costs is DMEM occupancy, and that is what C-mem bounds. Finally, the *identity slot* `DMEM[0]` is reserved but unread in this workload — $\text{FBT}(N)$ has no single-child internal node (§III-H) — so no assumption about its contents is load-bearing here.

Remark. The partition is not cosmetic. The C-* obligations can be checked by a compiler-side post-condition on the emitted image, independently of any fabric execution — they are compile-time verifiable, and C-mem joins them precisely because peak occupancy is a property of the image rather than of a run. The F-* invariants are properties of the fabric's behaviour, established once by RTL inspection and independently of any particular compilation. Separating the two means that a failure of Claim 1 can be diagnosed precisely: either the compiler violated an obligation (a bug in SPRS) or the fabric violated an invariant (a bug in the RTL), but not both.

D. Claim 1 and the Fail-Stop Property

Claim 1 (Cross-topology bitwise invariance)**.** *Let $N \in \mathbb{N}$, let $L \in (\text{bits}_{64})^N$ be a fixed leaf-value vector, and let $\mathcal{T}$ be the family of flat direct-network topologies realisable by adjacency tables consistent with `noc_system.sv`. Assume C-sw, C-bind, C-leaf, C-mem and F-sb-mono, F-sb-gate, F-fw, F-inorder, F-gci-haz, F-arb, F-fadd. Let G be a worker*

count and $\tau \in \mathcal{T}$ a topology for which `build_schedule` *emits an image satisfying* C-mem*, let α, μ, r be any valid assignment, mapping and refinement, and let s be the per-PE instruction schedule* `build_schedule` *emits for them. If the run terminates normally —* `all_done` *asserted, no* `P_ERROR`*, no watchdog — then the observed 64-bit root, the result of the root PE's final* `COMPUTE`*, satisfies*

$$R_{\text{observed}}(N, L, G, \tau, \alpha, \mu, r, s) \;=\; R_{\text{canonical}}(N, L). \quad (6)$$

Three features of the statement are deliberate and a reimplementer should read them as binding. *First*, C-mem is a compilability precondition, not a decoration: the claim asserts nothing about a configuration whose image exceeds the per-PE DMEM budget or whose router count exceeds what the adjacency interface can address. One instance in our suite is in that position, and it is also the one that never settles. *Second*, the claim is a partial-correctness statement. Liveness is Property 1's business, which is exactly why C-route, F-credit, F-misroute and F-watchdog are not hypotheses here: each of them, violated, can prevent a root from being produced but cannot change one that is. *Third*, the observable is the root PE's final `COMPUTE` result, the value latched at `btree_fsm_fast.sv:806` and the quantity every number in this paper reports; we do not state the claim on the root DMEM slot, which no emitted instruction ever reads back.

Under the same assumptions a weaker safety property holds, and its scope is exactly the set of violations that leave some operand *absent*.

Property 1 (Presence-gated fail-stop)**.** *Suppose an assumption violation causes some* `COMPUTE` *to name a DMEM address whose presence bit is never set — because a packet was misrouted and dropped (F-misroute), because a credit was lost and the value never arrived (F-credit), because a routing table names an unreachable destination (C-route), or because an operand was bound to a slot that no agent writes (C-bind). Then no instruction retires on the affected PE, F-watchdog fires after* `WATCHDOG_MAX` $= 20{,}000$ *consecutive non-retiring cycles, and the PE enters* `P_ERROR` *recording* `error_pc` *and* `error_state`*;* `all_done` *is never asserted and no root is produced. An invalid opcode raises* `P_ERROR` *immediately. The fabric therefore never returns a root in these cases.*

The converse bound is equally explicit, and we state it because the property is easy to over-read. The fabric checks presence, not identity: a `COMPUTE` whose operands are both present executes, whatever they contain. Violations that leave every read address present — an operand misbound to a live slot (C-bind), a corrupted leaf value (C-leaf), a duplicate write to a receive buffer (C-sw), an image over the DMEM budget in which two addresses alias onto one written slot (C-mem), or a violation of F-sb-mono, F-sb-gate, F-fw, F-inorder, F-gci-haz, F-arb or F-fadd — terminate normally with a wrong root. The fabric does not detect these, and we do not claim that it does. They are discharged before or outside it: C-bind, C-sw, C-leaf and C-mem by the compile-time post-condition on the emitted image, and all of them by comparison of the final root against the compiler-computed expected value (§III-I).

The injected-fault campaign separates along exactly this boundary, and the separation is evidence we had not previously claimed. Of the 54 address-substitution mutants, 43 repoint an operand at a DMEM slot that no agent ever writes — the harness draws a uniformly random address rather than another node's address — and those fail-stop: in every case we re-simulated, the fabric raised `P_ERROR` with a recorded `error_pc` on each PE, which is Property 1's outcome observed under injected fault rather than argued from the RTL. The remaining address substitutions, and all 54 leaf-corruption mutants, terminate normally with a wrong root and are caught by the expected-value comparison instead. Oracle non-vacuity therefore rests on the mutants whose root reached that comparison — the 54 leaf corruptions plus at most eleven address substitutions — and not on all 107, and C-leaf is the clean counterexample to any stronger reading of Property 1: all 54 leaf-corruption mutants terminate normally, one of them one ulp from the canonical root.

We label these a *claim* and a *property* rather than mechanised theorems because the supporting argument proceeds by RTL inspection, a compiler post-condition on the emitted image, end-to-end root comparison and mutation testing, not by formal proof. We discuss a mechanisation path in §III-I; readers who insist on "theorem" should substitute accordingly.

E. Supporting Arguments

Figure 2 shows the dependency structure of the assumptions and the supporting arguments. We present four arguments, each resting on a specific subset, and then compose them.

a) Argument A1 (Operand binding, from C-bind, C-mem).: `build_schedule` (`sprs_core.py:5218--5222`) sets `child_addrs = [addr(children[0]), addr(children[1])]`, and the tree's `children(v)` method returns $[\text{left}(v), \text{right}(v)]$ by construction (`sprs_core.py:404--408`). The addresses come from `addr_map` keyed on *node identity*, and the allocator gives every cross-PE child a dedicated receive slot above the host PE's local high-water mark, so the emitted binding is $(\text{left}(v), \text{right}(v))$ for any assignment, whether or not the partition is connected. C-mem is what makes that address-level statement a value-level one: distinct addresses stay distinct only while the image fits the DMEM depth, because the fabric truncates every DMEM address to $\log_2$ `DMEM_DEPTH` bits. Under A3 the operand *values* read at those addresses are the final values written.

b) Argument A2 (Single-writer construction, from C-sw, F-sb-mono).: The `DMEMAllocator` (§V-C) walks `su_postorder` and assigns each tree node a DMEM slot, recycling a local slot only after the node's unique parent has consumed it; receive buffers are allocated in a second pass and never recycled. Because $\text{FBT}(N)$ is a tree, each node has exactly one parent and therefore exactly one consumer. This constructs C-sw *in its scoped form*: every receive buffer has exactly one writer, and that writer is remote. Under F-sb-mono the scoreboard bit of a receive buffer therefore transitions $0 \to 1$ exactly once per program, at the clock edge of that unique write. Neither statement is made about local

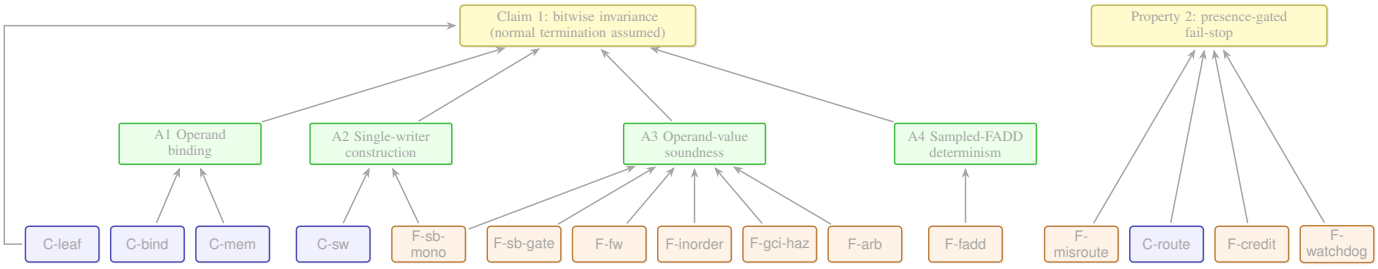


Fig. 2. Assumption dependency structure: five compiler obligations (C-*, blue) and ten fabric invariants (F-*, orange). Eleven feed the four arguments A1–A4 (green) that establish Claim 1; the four at the right establish Property 1, which is why Claim 1 may assume normal termination. C-leaf enters at the composition step rather than through an argument, and C-mem enters A1 as a capacity precondition. C-bind also appears in Property 1's antecedent, in the case where the misbound slot is one that no agent writes; that edge is omitted for legibility. Every node has at least one edge, and every edge is used in the text below.

temporaries, which are recycled by design; those are case (b) of A3.

c) Argument A3 (Operand-value soundness, from F-sb-mono, F-sb-gate, F-fw, F-inorder, F-gci-haz, F-arb).: C-sw partitions every PE's DMEM into receive buffers and local temporaries, so every address a `COMPUTE` reads falls into exactly one of two cases, and the two together are the whole argument.

(a) Receive buffers. Under F-sb-mono, once `scoreboard[a]` is set, `DMEM[`a`]` holds the value of the unique remote write to a. Under F-sb-gate the DE→EX transition of a `COMPUTE` requires both operand bits to be set, so at the EX clock edge both operand slots hold their committed values. F-fw handles the BRAM read-first race: two cascaded forwarding registers catch the back-to-back write-then-read case and the across-one-stall case, which together exhaust the races possible at this pipeline depth.

(b) Local temporaries. Here the scoreboard is silent — a recycled slot's bit is already set — and it does not need to speak. By Remark III-C a recycled slot has either no remaining reader, in which case the old value is dead, or exactly one, the `COMPUTE` that overwrites it, which under F-inorder samples its operands in DE strictly before writing back in EX. The single writer that program order does not order is the asynchronous `GENERATE`: F-gci-haz stalls any consumer naming its in-flight destination until the value commits, and F-arb keeps a colliding PIU write from discarding it. In both cases the value a `COMPUTE` reads is the one its emitted binding names.

d) Argument A4 (Sampled-FADD determinism, from F-fadd).: At the EX-writeback clock edge, `fsm_dmem_wr` commits $\mathrm{fadd}_{\mathrm{rtl}}(\mathrm{dmem\_a}, \mathrm{dmem\_b})$ where the inputs are the stable values of A3. `fp64_add.sv` is combinational with zero `always_ff` registers; its output is a pure function of its inputs.

e) Composition.: Every internal DMEM value at termination is $\mathrm{fadd}_{\mathrm{rtl}}$ applied to the (left, right) children — A1 for the addresses, A3 for the values, A4 for the function — evaluated in post-order by data dependence, with initial values supplied by the C-leaf-enforced `GENERATE` writes and address distinctness supplied by C-mem. The root PE's final `COMPUTE` therefore returns $\mathrm{evaluate}(\mathrm{FBT}(N), L, \mathrm{fadd}_{\mathrm{rtl}}) = R_{\mathrm{canonical}}(N, L)$, a pure function of (N, L). Each of Claim 1's eleven hypotheses is used at least once above: C-bind and C-mem in A1, C-sw and F-sb-mono in A2, F-sb-gate, F-fw, F-inorder, F-gci-haz and F-arb in A3, F-fadd in A4, and C-leaf here.

Remark (On Property 1). The fail-stop argument is more direct, and it is bounded. A C-route violation routes a packet to a PE that is not its destination, where F-misroute drops it; the intended destination's presence bit is never set, its dependent `COMPUTE` stalls, and F-watchdog raises `P_ERROR` after 20,000 cycles. A F-credit violation stalls a link or overfills a receive FIFO, whose write is ignored when full; either way the value never arrives and the same chain fires. A C-bind violation that names a slot no agent writes is the same case once more, and it is the case we have injected. The bound is that this is *all*: a violation leaving every read address present — a C-sw violation, for instance, whose duplicate write finds the bit already set and is a no-op to the gate — produces a wrong root with no fabric-visible symptom, and is caught, if at all, by the compile-time post-condition or by the root comparison.

F. The Mapping/Topology Orthogonality Theorem

Claim 1 asserts invariance over $(G, \mathrm{topology}, \alpha, \mu, r, s)$. The (G, α) slice is where the substantive work lives: changing G changes the number of PEs, and changing α changes which PE hosts which tree node. The $(\mathrm{topology}, \mu, r, s)$ slice is strictly less interesting — once the assignment is fixed, the topology, the worker-to-physical mapping, and the routing and timing only change *where* and *when* data flows, not the operand binding at any COMPUTE. We make this formal.

Proposition 1 (Mapping/topology orthogonality)**.** *Fix N, L, assignment α, refinement r, and the fabric's FADD unit. For any two valid mappings μ_1, μ_2 and any two valid routing tables ρ_1, ρ_2 for (possibly different) topologies in $\mathcal{T}$, the observed root result satisfies*

$$R_{\mathrm{observed}}(\mu_1, \rho_1) \;=\; R_{\mathrm{observed}}(\mu_2, \rho_2) \;=\; R_{\mathrm{canonical}}(N, L). \tag{7}$$

Structural argument. By inspection of `build_schedule` and `generate_instructions` (§V-E): the `COMPUTE` instructions' $(\mathtt{addr\_a}, \mathtt{addr\_b}, \mathtt{dest})$ triples depend only on α and the DMEM allocator's output, neither of which reads μ or the routing tables. The `SEND` instructions' $(\mathtt{dest\_gpu}, \mathtt{target\_addr}, \mathtt{link})$ triples depend on μ

(through the `mapping.get(gpu, gpu)` indirection) and on the routing tables — but these only affect *where* data flows, not the $(\texttt{addr\_a}, \texttt{addr\_b})$ operand bindings at the receiving PE's `COMPUTE`. Under F-sb-gate, the receiving PE's `COMPUTE` reads the DMEM values by address, not by arrival time, arrival port, or arrival VC. Therefore R_{observed} is independent of (μ, ρ). □

Proposition 1 is the cleanest statement of what the ABI buys. The topology and the physical mapping — the two things that change most obviously under cluster reshape — are structurally orthogonal to the bit result. This is not a consequence to be validated empirically; it is a consequence of the code path that emits the instructions.

G. FADD Policy

Our FADD (`fp64_add.sv`, with a bit-exact Python mirror at `sprs_core.py::_fp64_add_bits`) implements a *deterministic subset* of IEEE 754-2019 binary64 addition. We specify the subset fully:

1) *Input FTZ.* Subnormal inputs (exp $= 0$, frac $\neq 0$) are flushed to ± 0 preserving sign.
2) *RNE only.* The round-to-nearest-even predicate is $\text{guard} \wedge (\text{sticky} \vee \text{lsb})$. No other rounding mode is available; there is no latched rounding-mode register.
3) *Output FTZ.* Post-round subnormals are flushed to ± 0 preserving sign.
4) *Signed-zero cancellation.* The result of $x + (-x) = +0$ for all non-NaN x; $(-0) + (-0) = +0$. The latter deviates from IEEE 754-2019 §6.3, which mandates -0; we document the deviation and note it is deterministic and inherits Claim 1.
5) *NaN propagation.* Output is a quieted qNaN whose sign equals $\text{sign}(a)$, exponent is `0x7FF`, bit 51 is forced to 1 (the quiet bit), and bits [50:0] copy the payload of a if a is NaN, else of b. Narrower than IEEE 754 (which allows any qNaN payload) but deterministic.
6) *Infinity arithmetic.* $\pm\infty + \pm\infty$ (same sign) $= \pm\infty$; $\pm\infty + \mp\infty$ returns the canonical qNaN `0x7FF8_0000_0000_0000`; $\pm\infty +$ finite $= \pm\infty$.

We refer to (1)–(6) collectively as *FTZ+RNE semantics*. Invariance (Claim 1) depends only on fadd_{rtl} being a pure function of its inputs; it does not depend on any particular IEEE 754 feature.

Compatibility with reference implementations. FTZ+RNE diverges from strict IEEE 754 on three regimes: subnormal inputs, post-round subnormal outputs, and signed-zero cancellation of two negative zeros. Workloads whose reference implementation (NumPy, MATLAB, a reference BLAS) produces or depends on subnormals will see SPRS diverge from reference at the affected bits. This is a scope decision, not an oversight: a subnormal-aware alternative `alu_op` slot is implementable as a one-instruction ISA extension that inherits Claim 1 by construction. We flag the extension for deployments requiring reference-library bit-parity.

The subset, checked rather than asserted. Items (1)–(6) are a specification, and the adder we first submitted did not meet item (2): on effective subtraction it was not round-to-nearest-even. We found that by scoring the adder against references external to the design — exact rationals, and the host FPU — rather than against its own Python mirror, repaired the hardware and the mirror together, and re-verified. §IV-J gives the defect, the repair and the evidence; L13 gives the two independent reasons why neither the campaign nor the mirror could have found it. What items (1)–(6) state is therefore what the shipped adder now measures: zero disagreements with correctly-rounded IEEE 754 addition over 4,644,000 vectors inside the documented scope (normal operands, normal non-zero result).

H. Edge Cases

A few edge cases deserve explicit treatment.

$N = 1$. $\text{FBT}(1)$ is a singleton (one leaf, zero internal nodes). The "result" is $L[0]$; the compiler emits one `GENERATE` and no `COMPUTE`. The argument is vacuously true.

$N = 0$. Disallowed at the compiler front end.

Single-child edge. By construction, $\text{FBT}(N)$ with $2N - 1$ nodes has no single-child internal nodes. The identity slot `DMEM[0]` is reserved but unused in the present workload; it exists so that future relaxations (asymmetric trees, variable-arity reductions) can use an identity operand without changing the ISA.

Integer and bitwise reductions. Claim 1 requires only a deterministic binary operation; it does not require FADD specifically. Integer ADD, MAX, MIN, bitwise operators inherit the claim by extension via a different `alu_op` code. This is a structural extension, not an empirical generalisation.

I. Mechanisation and Runtime Checks

Our current strategy discharges A1–A4 through a combination of *compile-time* post-condition assertions, *simulation-time* SystemVerilog assertions, and *empirical* mutation testing.

a) Compile-time checks.: C-bind, C-sw, C-leaf and C-mem are decidable on the emitted IMEM without executing anything:

- For every internal v and its emitted `COMPUTE`, $\texttt{addr\_a} = \texttt{addr\_map}[\text{pe}(v)][\text{left}(v)]$ and $\texttt{addr\_b} = \texttt{addr\_map}[\text{pe}(v)][\text{right}(v)]$.
- For every PE p, $|\{(a, \texttt{dest}) : \texttt{dest} = a\}| = |\texttt{addr\_map}[p]|$ (single-writer).
- For every leaf k, the `GENERATE` writes $\texttt{gci\_buffer}[\text{local\_idx}(k)] = \text{make\_leaf\_val}(\text{idx}(k))$.
- For every PE p, $\max_a \texttt{addr\_map}[p][a] < 512$ (C-mem).

The campaign path did not run these checks inside the compile loop; we ran them afterwards over every released configuration, and §V-E reports the counts. C-mem is the exception in the released compiler, where it is a precondition of emission rather than an audit of the output (Remark III-C). C-route has no companion check beyond destination-correctness and loop-freedom by construction, and §IV-H states what that does and does not establish.

b) Simulation-time checks.: Each emitted testbench carries the canonical root as a compile-time `EXPECTED` constant and compares the fabric's `all_done` result against it (`sprs_core.py:5482, 5573`), which is the `PASS`/`FAIL` verdict every number in §VII is built on. We ship no SystemVerilog assertion suite, and no claim here rests on one.

c) Mutation testing.: To establish that the oracle harness is non-vacuous (i.e., would detect a real divergence were one to occur), we inject two mutation classes:

- *Address substitution*: in one randomly chosen `COMPUTE`, replace `addr_b` with the DMEM address of a different tree node. Breaks Argument A1 while preserving A2–A4.
- *Leaf corruption*: in one randomly chosen `GENERATE`, flip one mantissa bit.

We *do not* use operand-swap as a mutation: FADD is commutative at the bit level under FTZ+RNE ($\mathrm{fadd}_{\mathrm{rtl}}(a,b) = \mathrm{fadd}_{\mathrm{rtl}}(b,a)$ for all (a,b)), so operand-swap is bit-identical and would inflate the false null-detection rate spuriously. Mutation detection rates are reported in Table V (§VII-C).

d) Path to mechanisation.: A bounded Alloy [34] model of the scheduler plus scoreboard (scope $N = 16, G = 4$, $\sim$250 lines) would mechanise Arguments A1–A3 at finite scope. A HardCaml or Kami [35] embedding of `btree_fsm_fast.sv` and `fp64_add.sv` would mechanise A4. We report these as in-progress supplementary artefacts and do not claim them here.

IV. The Fabric: NoC, ISA, Pipeline

This section describes the fabric that realises the F-* invariants. The guiding principle is that topology is a runtime configuration of adjacency and routing tables (§IV-B), the ISA is small enough to audit for ABI adherence (§IV-D), the scoreboard is the single point of synchronisation across the three agents that write DMEM (§IV-E), and every assumption violation terminates the program at a diagnostic state rather than producing silent corruption (§IV-F).

A. Design Goals

The fabric supports a cluster of up to 4096 PEs in a single Verilog instantiation whose topology is determined by the adjacency and routing tables loaded at boot. The binding ceiling is the width of the configuration interface, not a parameter: adjacency writes carry a 12-bit router id whose top code is the disconnected sentinel (`noc_system.sv:67--70`), so router ids are addressable up to 4,094 and a configuration above that aliases silently (§IV-I). `MAX_ROUTERS` in `noc_pkg.sv` is declared once and referenced nowhere, as is `MAX_PORTS`. Router radix is likewise not fixed by the design: it is the per-instance parameter `PORTS_PER_RTR`, which the compiler emits and which ranges 3–66 across our suite (§IV-I). The reduction ISA has four live opcodes, one reserved for future barrier/reset semantics. All determinism-critical mechanisms — scoreboard, forwarding network, DMEM write arbiter, credit accounting — are simple enough to fit in a two-page RTL inspection.

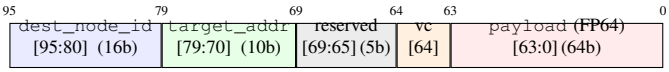


Fig. 3. 96-bit single-flit packet format. Bit `[64]` is read by the router as a VC-steering bit but is written as a constant zero by the only packet-forming assignment in the design, so every packet travels on `VC0` and routing latency is value-agnostic by construction (§IV-H). The package headers document `[69:64]` as six reserved bits and are stale with respect to the router's use of bit `[64]`.

B. Topology as a Runtime Parameter

Two per-router configuration tables parameterise the topology.

The *adjacency table* (`adj_entry_t` in `noc_pkg.sv`) maps $(\text{router}, \text{port}) \rightarrow (\text{target\_router}, \text{target\_port}) \mid$ `ADJ_DISCONNECTED`, set by the compiler via a write-enable interface before `program_start`. The *routing table* (`route_table` in `noc_router.sv`) maps $(\text{router}, \text{destination}) \rightarrow$ output_port, also compiler-loaded. A single `noc_system` module with `N_NODES` and `N_ROUTERS` parameters supports the full family $\mathcal{T}$. Port 0 is reserved for the local network interface on every router — the one structural convention independent of topology.

The consequence is that *topology is data, not RTL*. Adding a new topology requires implementing a Python classmethod that emits adjacency and routing tables; no Verilog change is needed. The compiler at `sprs_core.py:487--695` provides six topology families (linear, ring, 2D mesh, 2D torus, fat-tree, hypercube) with analytical shortest-path computations at $O(1)$ per pair; a BFS fallback handles custom topologies at $O(G)$ per source.

C. Packet Format

The 96-bit single-flit packet format carries exactly one FP64 value per packet: a 16-bit `dest_node_id`, a 10-bit `target_addr`, five reserved bits, the VC-steering bit $[64]$, and the 64-bit payload (Fig. 3).

The router steers on bit $[64]$ (`noc_router.sv:118`), but the compiler and the FSM never set it: the field is a literal `6'b0` in the single packet-forming assignment (`btree_fsm_fast.sv:629`). VC selection is therefore constant rather than merely header-dependent, so buffer occupancy cannot be perturbed by the payload and the cycle counts we report are value-agnostic by construction. §IV-H states what that costs on cyclic topologies.

D. Reduction ISA

A four-bit opcode addresses sixteen slots; four are live (Table II), eleven are reserved for trap, one is earmarked.

`DMEM[0]` is reserved as an identity slot: DMEM is initialised to all-zero and `scoreboard[0]` is set at reset (`btree_fsm_fast.sv:183--186, 709`), and the allocator reserves address 0 (`sprs_core.py:5098`). Zero is the identity for ADD, FADD, OR and XOR; MAX, MIN and AND would need a different initialiser. This realises F-id for the operations we run and supports compiler emission of identity-preserving half-reductions; the slot is unread in the reported workload (§III-H).

TABLE II
THE FOUR-OPCODE REDUCTION ISA. OP_RESERVED IS RESERVED FOR A FUTURE BARRIER/RESET_SB INSTRUCTION ENABLING MULTI-REDUCTION PROGRAMS. OPCODES $\geq$ 4'h4 (OTHER THAN THE RESERVED SLOT) TRAP TO P_ERROR.

Mnemonic	Code	Effect
NOP	4'h0	Consume one slot.
GENERATE	4'h1	Fetch a leaf into DMEM[dest] via the GCI interface; set scoreboard[dest].
COMPUTE	4'h2	Read DMEM[addr_a], DMEM[addr_b] (scoreboard-gated), write alu_op(a, b) to DMEM[dest], set scoreboard[dest].
SEND	4'h3	Enqueue an RDMA-push packet with payload DMEM[src] to (dest_gpu, target_addr) on link.
OP_RESERVED	4'h4	Reserved for future BARRIER/RESET_SB.

E. Pipeline and DMEM Forwarding

Each PE hosts a three-stage pipeline:

- IF: registered IMEM read (1-cycle latency), PC management, combinational decode of opcode, aux, dest, addr fields.
- DE: scoreboard hazard check, DMEM read issue. Stalls if either scoreboard bit is 0 (F-sb-gate).
- EX: combinational ALU (FADD and friends), DMEM writeback or TX FIFO enqueue. Stalls on SEND backpressure or DMEM write collision.

The DMEM read path uses a two-level write-to-read forwarding network (the forwarding network in btree_fsm_fast.sv), realising F-fw. Level 1 forwards a cycle-N write to a cycle-$(N+1)$ read of the same address — the back-to-back-COMPUTE case. Level 2 forwards across a single intervening stall, catching the case where the one-cycle forward expired before the read settled. The two levels together exhaust the possible races given the pipeline depth and the scoreboard's one-cycle gate.

The scoreboard is a DMEM_DEPTH-wide bit vector set whenever *any* of three write ports commits: PIU (an arriving RDMA-push packet), GCI (a completed leaf fetch, which is also how a GENERATE commits), or the FSM COMPUTE writeback (btree_fsm_fast.sv:365--382, 706--721). F-sb-mono guarantees each bit transitions $0 \rightarrow 1$ at most once per program; the scoreboard is the single point where "data present at address a" is decided, independently of which port delivered the data. This is what makes the scoreboard-gated reads structurally race-free (Fig. 4).

F. Fail-Stop Discipline

The fabric implements a three-way safety net that makes Property 1 provable rather than asserted.

PIU autonomous write and misroute drop. The PIU (the PIU block in btree_fsm_fast.sv) drains RX FIFOs on a priority basis. On each drained packet it examines bits [95:80] (dest_gpu_id). If equal to the local PE's id, it writes the payload to DMEM[pkt[79:70]] (target_addr from the header) and sets the scoreboard bit. If not equal — a routing-table bug, a corrupted packet, a torn adjacency — it drops the packet, returns credit, and asserts piu_misroute for one cycle. This realises F-misroute. The drop and the credit return are real hardware; the flag itself is an internal signal with no reader and is not a module port, so a misroute is observable only through the liveness failure it causes.

The consequence for determinism is stronger than a silent-failure assumption. A bad routing table causes the destination's scoreboard bit never to be set; the destination's dependent COMPUTE stalls forever; F-watchdog (below) fires and raises P_ERROR. A routing-table violation cannot produce a silently-wrong root result — the fabric converts the violation into a liveness failure detected and localised by the watchdog.

Watchdog + stall timeout + invalid-opcode trap. The fabric maintains three error detectors in the P_RUN state (the stall logic in btree_fsm_fast.sv):

- Watchdog: fires after WATCHDOG_MAX $= 20{,}000$ cycles with no instruction retired.
- Stall timer: fires after TIMEOUT_MAX $= 10{,}000$ cycles of continuous EX stall.
- Invalid-opcode trap: any opcode $>$ 4'h3 (other than the reserved slot) triggers P_ERROR immediately.

Each raises error_pc and error_state for post-mortem diagnosis. The termination outcomes are therefore (i) correct result, (ii) P_ERROR with diagnostic context, or (iii) watchdog timeout with the same diagnostic; no configuration in the campaign produced a fourth. The fabric does not guarantee that a fourth is impossible — Remark III-C gives the one construction, an image over the C-mem budget, that defeats the presence gate instead of tripping it.

G. Write Arbitration and Retry Latches

The three DMEM write ports demand the same resource; their arbitration (Fig. 4, bottom) realises F-arb.

The arbiter is strict-priority PIU $>$ GCI $>$ FSM (the DMEM write arbiter in btree_fsm_fast.sv). PIU always wins — maintaining the RDMA-push semantic that incoming data is never blocked. The GCI write carries a retry latch (gci_write_held in btree_fsm_fast.sv): if GCI completion (gci_done) collides with a PIU write in the same cycle, the arbiter drops the GCI write but the retry latch captures the pending value and re-asserts gci_dmem_wr until the arbiter accepts it. The FSM COMPUTE writeback loses to both agents and is stalled via ex_dmem_collision when necessary.

This matters for determinism in a subtle way. Without the GCI retry latch, a PIU/GCI collision would silently drop the leaf value; the destination's scoreboard would never be set by GCI (PIU wins so the bit is set, but to the wrong value); the next COMPUTE would read a wrong leaf and produce a wrong root — violating A1 (sampled-read determinism) despite all other invariants holding. The retry latch is a narrow safeguard against a specific timing race; we name it explicitly because it is necessary-for-correctness, not merely nice-to-have.

H. Flow Control, Routing, and Deadlock

Each input port carries NUM_VCS $=$ *two* virtual channels (noc_pkg.sv), and every emitted testbench sizes

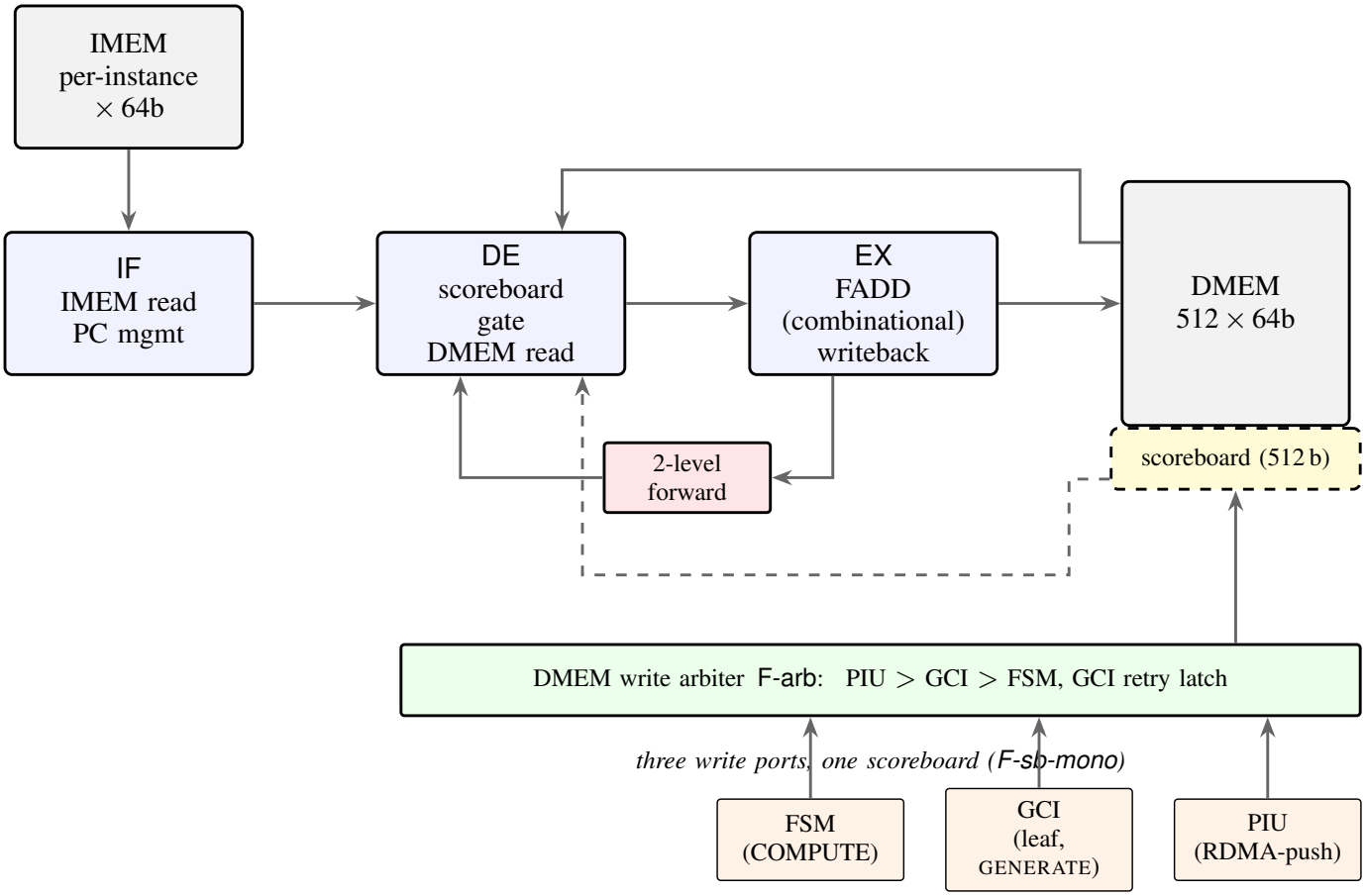


Fig. 4. PE microarchitecture. The three-stage IF–DE–EX pipeline is scoreboard-gated at DE; the scoreboard is a single source of truth for "data present at address a" across the three agents that write DMEM (PIU, GCI, FSM `COMPUTE` writeback). A `GENERATE` is committed by the GCI port, not by a fourth agent. A two-level write-to-read forwarding network (red) handles the BRAM read-first race for back-to-back writes and across-one-stall writes. The DMEM write arbiter (green) enforces the strict-priority PIU > GCI > FSM discipline of F-arb; the GCI retry latch re-asserts the pending write until accepted. No write is silently dropped.

router FIFOs at 16 flits, overriding the RTL default of 8 (`.RTR_BUF_DEPTH(16)`, `sprs_core.py:5430`). Per-link credit-based flow control prevents ingress overflow: `link_tx` decrements a per-link credit counter on each transmission and increments it on each credit-return pulse, with counter conservation $\sum$pulses $= \sum$reads (F-credit). Credit return is serialised through a pending-credit register that accumulates the reads of a cycle and drains one pulse per cycle (`noc_router.sv:192,210`); it is two bits wide and wraps if a backlog of four ever accrues, which requires two virtual channels to be read in the same cycle. In this campaign it never does, for the reason the next paragraph gives, and its value is provably zero in every reported cycle.

One channel, and no escape. The router reads bit [64] as a VC selector, but nothing writes it: the field is a constant in the only packet-forming assignment in the design, the ISA has no VC field, and the compiler contains no virtual-channel logic. Every packet in every reported run travels on `VC0`. Nor could the second channel serve as an escape channel if the compiler did assign one: the router's egress buffer is a single FIFO per port shared across VCs (`noc_router.sv:142--165`), so the two channels are coupled at the output whatever the header says.

What discharges C-route, and what does not. The compiler computes routing tables by greedy minimal-distance search over the adjacency list (`sprs_core.py:667--686`); they are not dimension-ordered. What that construction gives is what C-route claims (§III-C): every emitted path terminates at the destination named in the packet header and visits no router twice. It does not give an acyclic channel-dependency graph. Built directly from the emitted tables, the CDG contains cycles on the ring instances (cycle length G, the textbook bidirectional-ring cycle) and on the torus instances (cycle length 6–32); it is acyclic on linear, mesh, hypercube and fat-tree at every suite size. We therefore claim no deadlock-freedom property for cyclic topologies. Deadlock is possible in principle, and the fabric converts it into a fail-stop rather than a wrong answer: a deadlocked PE retires nothing, so F-watchdog fires and Property 1 applies — `P_ERROR` with `error_pc` and `error_state`, and no root. The campaign's non-settling runs are accounted for in §VII-C; none of them, and none of the settled runs, produced a finite root differing from the canonical one. A compiler-side dimension-ordered pass with a VC-escape assignment, and a per-VC egress buffer to make that escape channel real, are the two changes a production deployment would need; neither can change a bit

TABLE III
MEASURED ROUTER RADIX ACROSS THE 40-INSTANCE SUITE, READ FROM THE PPR LOCALPARAM OF EACH EMITTED TESTBENCH. RADIX IS A TOPOLOGY PROPERTY; MAX_PORTS IN NOC_PKG.SV IS VESTIGIAL AND ENFORCES NOTHING.

Topology	Radix rule	Radix P	$\lceil \log_2 P \rceil$
linear	2 + local	3	2
ring	2 + local	3	2
mesh	4 + local	5	3
torus	4 + local	5	3
hypercube	dim +1	3–11	2–4
fat-tree	max(5, k+2)	5–66	3–7

result, only whether the run terminates.

A second silent-drop channel has the same shape. The emitted testbenches give the compute node's link to the network interface 16 credits against an 8-entry injection FIFO (.NI_BUF_DEPTH(8)), so a sufficiently long injection burst would overflow the FIFO and the packet would be discarded without a flag. The scoreboard bounds the damage exactly as above: a lost SEND leaves the destination's presence bit clear, the dependent COMPUTE stalls, and the watchdog fires. A drop can only cost termination, never correctness, which is why no passing run can contain one.

Round-robin allocation at each output port advances only on a successful grant (noc_router.sv): rr_ptr[i] increments iff obuf_wen[i] this cycle. Without this guard, RR advance on empty cycles would allow unfairness; with it, every contending $(port, vc)$ candidate is served within NUM_CANDIDATES cycles of becoming ready.

I. Resource Envelope

Router cost is dominated by radix, and radix is a property of the topology, not a constant of the design. MAX_PORTS is declared in noc_pkg.sv (and again, with a different value, in btree_pkg.sv) but is never referenced by any module; the live quantity is PORTS_PER_RTR, which the compiler emits per instance. We read it back out of every generated testbench rather than assume it: Table III reports what the suite actually instantiates, and those ports are real — on every topology at every suite size, the number of ports the emitted adjacency table actually wires equals PORTS_PER_RTR.

The span is 3–66 ports. Low-diameter mesh and torus routers stay at $P = 5$, hypercube reaches $P = 11$ at $G = 1024$, and fat-tree reaches $P = 66$ at $G = 4096$ (maxp_mod_fat, 4,192 routers). Because the crossbar term is $O(P^2)$, a single fixed-radix estimate is not merely imprecise across topologies — it is wrong by more than an order of magnitude at the top of the range. Table IV therefore reports the envelope as a function of P, holding the route-table depth at $G = 4096$ so that the columns isolate the effect of radix alone.

A fat-tree router is 20× a mesh router (750,976 bits against 37,728), and 56% of that is crossbar. Aggregated over the fabric the gap is starker still: $\approx$ 147 Mbit of router storage at $P = 5$ over 4096 routers, against $\approx$ 3002.3 Mbit for maxp_mod_fat — the regime where a flat crossbar stops being the right structure and a hierarchical or input-queued switch is the standard answer. We report the flat figure because it is what we simulated, not because it is what we would build at that radix.

TABLE IV
PER-ROUTER RESOURCE ESTIMATE AS A FUNCTION OF RADIX P, AT A COMMON ROUTE-TABLE DEPTH $G = 4096$ AND THE FIFO DEPTH THE CAMPAIGN INSTANTIATES (BUF_DEPTH = 16, NOT THE RTL DEFAULT OF 8). A RESEARCH-PROTOTYPE ENVELOPE (STORAGE-EQUIVALENT BITS; THE CROSSBAR ROW PRICES MUX STORAGE-EQUIVALENT, NOT REGISTERS); NO POST-PLACE-AND-ROUTE DATA.

Component	**Per-router bits at radix P**		
	$P = 5$ mesh/torus	$P = 11$ hypercube	$P = 66$ fat-tree
Routing table ($G\lceil \log_2 P \rceil$)	12,288	16,384	28,672
VC ingress FIFOs ($P{\cdot}2{\cdot}16{\cdot}96$)	15,360	33,792	202,752
Egress FIFO ($P{\cdot}16{\cdot}96$)	7,680	16,896	101,376
Crossbar ($P^2{\cdot}96$)	2,400	11,616	418,176
Per-router subtotal	**37,728**	**78,688**	**750,976**

Per-PE memory is independent of topology, but only one half of it is fixed. DMEM is 512×64 on every instance (sprs_core.py:5384); IMEM is not, because the emitter elaborates CN_IMEM to the next power of two above that instance's longest per-PE program — 64 to 2048 entries across the simulated suite (§VI-E). We price the 512×64 case for both, 65,536 bits per PE and $\approx$ 256 Mbit over $G = 4096$, and it should be read as the design target rather than as a measurement of what every run instantiated. This is the row Appendix A's 64-bit instruction word sizes.

Two caveats bound the estimate. It counts storage-equivalent bits only — link drivers, clock distribution, arbiter logic, and I/O are excluded — and it is an RTL-level count, not a synthesis result, from a design whose ALU is combinational (L0c).

J. Known Limitations

To set the scope of the present implementation honestly:

Head-of-line blocking. The router is store-and-forward with a single FIFO per VC; under the tree-reduction traffic pattern, root-incident links queue up to $\lceil \log_2 G \rceil$ merge waves. At $G \geq 1024$ on ring and torus, absolute cycle counts slew because of this. Wormhole forwarding is a planned extension.

Single clock domain. No clock-domain crossings. Multi-chiplet deployment at $G = 4096$ would require asynchronous FIFOs on inter-chiplet links, changing credit round-trip latencies but not the bit result.

Validated at one point of a parameterised design. The fabric instantiates two virtual channels, two compute-node links, and $k/2$ core switches per fat-tree pod; the compiler uses one of each. The unused halves are where the defects are. The credit backlog register of §IV-H can only overflow on a two-VC read; the network interface's injection race can only fire on the second link; and for $k \geq 4$ the fat-tree emitter assigns pod-to-core ports above the declared radix, which the fabric discards, so the elaborated network uses a single core switch. Every emitted route is valid on that network and the surplus cores carry no traffic, but the analytic lower bound Ω (§VI-A)

min-cuts the intended edge set, so the fat-tree Ω ratios we report are computed against a more capable network than the one simulated — an understatement, not a flattery. This is the expected consequence of validating a parameterised design at a single point in its parameter space, and we state it as a general caveat on the RTL rather than as four separate errata.

Effective subtraction in `fp64_add`: a defect, repaired. The adder we first submitted did not implement round-to-nearest-even on effective subtraction. When the operand signs differ the aligned mantissas are subtracted, and the bits shifted out during alignment form a tail strictly between zero and one guard unit; the design recorded that tail in the sticky bit but never borrowed it, so the raw difference was too large by exactly the tail and the *addition* rounding rule $\mathrm{guard} \wedge (\mathrm{sticky} \vee \mathrm{lsb})$ was then applied to it, rounding up a value lying below the midpoint. The error is always $+1$ ulp. Scored against the host's correctly-rounded addition it fires on 46.8%–50.4% of opposite-signed pairs whose exponents differ by 5 to 52 binades, and on *every* such pair at a difference of exactly 53.

The repair takes three coordinated edits, and that is the part worth reporting. They are: split the alignment tail into its most significant bit and the rest; borrow the tail from the difference; and rescale the borrowed remainder across the one-bit renormalising left shift that follows a cancellation, which can carry it past a whole guard unit. Stopping after the borrow — the obvious one-line fix — leaves a -1 ulp residual on 1.3% of effective subtractions (979 of 76,000 in a three-variant sweep), concentrated at exponent separations of 2–10 binades. A partial repair here looks plausible and is wrong, which is why what establishes correctness is the verification rather than the patch.

On a two-directional binade sweep plus a uniform random sample the repaired adder agrees with correctly-rounded IEEE 754 addition on all 160,800 vectors (zero errors), and the repaired software oracle agrees with it bit for bit on the same set (zero mismatches); over the wider host-FPU sweep the figure is zero of 4,644,000. The negative control fires: the as-submitted pair fails the same test on 15,717 of 62,320 vectors, every failure $+1$ ulp on the effective-subtraction path. That control also makes the second half of the finding measurable. `_fp64_add_bits` is a line-for-line transcription of `fp64_add.sv` rather than an independent model, so it carried the identical defect and was repaired in lockstep — and on the as-submitted pair the two agree bit for bit on all 62,320 vectors while both are wrong on 15,717 of them. A differential check between an implementation and its own transcription passes exactly where it is least informative; L13 draws the lesson, and this is its measured instance.

No number in this paper moves, and we checked that rather than argued it. Leaf values come from (9), which is strictly positive at every index, so no FADD in any reported run has operands of differing sign: replaying the emitted reduction order under the as-submitted and the repaired oracle over 53 worker counts from 2 to 16,384 moves zero roots and encounters zero mixed-sign additions, while the guard's own injected-adder control detects two of two.

Soft-enforced single-writer. C-sw is discharged by a compile-time post-condition over the emitted IMEM, not by hardware refusal of a second write to an already-set scoreboard slot. A bug that silently corrupted the allocator would be caught by the static audit of §V-E but not by the fabric. A one-line RTL extension — "raise `P_ERROR` on a scoreboard-set transition from 1 to 1" — would harden this into a fabric-side guarantee; we plan it for the next revision.

V. The SPRS Compiler

This section describes how the compiler discharges the C-* obligations, and — because a reviewer of an earlier draft was right to ask — which parts of it actually produced the results in this paper. One pass, SU-ordered DMEM allocation (§V-C), does the load-bearing work: it *constructs* single-writer allocation and operand binding rather than assuming them. The remaining stages are standard heuristic ensembles whose algorithmic choices matter for cycle counts but not for the bit result.

A. Compilation Pipeline

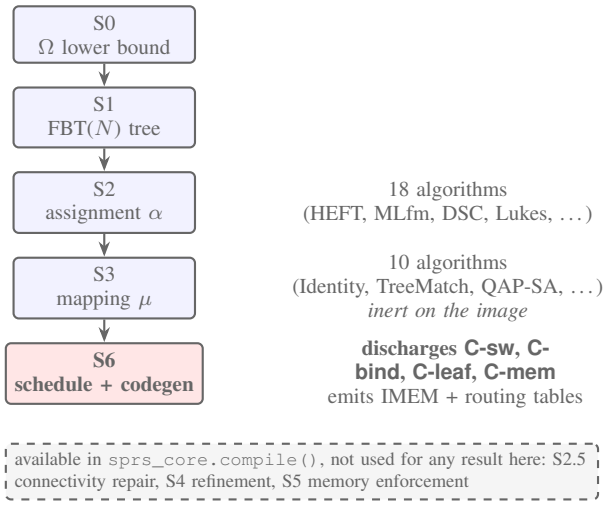
SPRS compiles a triple $(N, G, \text{topology})$ into a per-PE IMEM image plus per-router adjacency and routing tables. Every image reported in this paper was produced by five stages:

- **S0** Composite lower bound Ω (`compute_lower_bound`) — computed once per instance and cached; a reported metric (§VI-A) rather than a step of image construction.
- **S1** $\mathrm{FBT}(N)$ instantiation (`CBTree.__init__`).
- **S2** Assignment α : tree nodes $\to$ GPUs, selected from 18 algorithms (Appendix B, Tab. VIII).
- **S3** Mapping μ : GPUs $\to$ physical PEs, from 10 algorithms. *Inert on the emitted image*: `SEND` carries the logical GPU id and IMEM is loaded by logical index, so μ reaches the image only through the hop counts the cost model uses to place NOPs. Recompiling five representative points under all ten mappings yields one to three distinct images, differing only in NOP padding.
- **S6** ISA emission with cycle-accurate NOP insertion and GCI-gap peephole compression, then testbench emission (`build_schedule`, `generate_instructions`, `emit_sv_testbench`).

Three further passes exist in the released compiler and produced none of the results reported here. Connectivity repair (`repair_connectivity`, S2.5) and memory enforcement (`enforce_memory`, S5) are called only from `sprs_core.compile()`, which the campaign path does not use; refinement (S4: 12 base methods chainable to 122 configurations) was not exercised in the hardware campaign, which fixes refinement at the identity and enumerates the (α, μ) face exhaustively instead (§VI-E). §V-B reports what running S2.5 would have cost, and why we report the unrepaired pipeline.

All three algorithmic search stages preserve the ABI by Argument A1 (§III-E) — including the refinement stage the campaign did not run, which permutes placement and schedule order but never an operand pair. Every valid triple (α, μ, r)

yields an ABI-compliant IMEM, so every configuration in the $\sim$22,000-point space produces the same 64-bit root result by Proposition 1. That is a structural consequence; §VII-C verifies 2,822 images of it directly.



B. Disconnected Partitions and Memory Pressure

Twelve of the eighteen assignment algorithms — HEFT, DSC, DFSseed, Chrétienne, CPOP, CPFD, ClHEFT, MLfm at small G, and ALNS-refined variants of each — can hand a PE a set of tree nodes that is not a connected subtree of $\mathrm{FBT}(N)$, and 2,061 of the 2,822 RTL-validated configurations (73.0%) are of that kind. Operand binding is unaffected, for the reason given in §III-C: `build_schedule` binds children by node identity and the allocator gives every cross-PE child a dedicated receive slot above the local high-water mark, so the emitted binding is $(\mathrm{left}(v), \mathrm{right}(v))$ for *any* assignment. That is why the invariance result quantifies over the raw output of all eighteen algorithms rather than over a repaired subset.

What disconnection costs is DMEM. A node whose parent is remote never returns its slot to the free list, so live values accumulate: over 681 compiled configurations, peak per-PE occupancy has median 10 slots (max 102) on the 244 connected partitions and median 25 (max 491) on the 437 disconnected ones, against the 512-entry budget. The margin at the tail is 21 slots, and past the ceiling the failure is silent (Remark III-C) — which is why the allocator's $O(N)$ peak-occupancy count is the check that matters, and why C-mem rather than connectedness is the hypothesis of Claim 1.

The repair pass is therefore available but not used. `repair_connectivity` reassigns orphaned nodes to their parent's PE, iterating to a fixed point; run on the 437 disconnected campaign assignments it converges within its 20-pass cap every time, and it does remove the occupancy pressure. It also destroys the load balance the assignment search exists to produce: it moves a median 50.8% of all tree nodes (max 96.9%), inflates the peak per-PE node count by a median factor of 1.86 and by up to 304×, and leaves 82 of the 437 with more than 512 instructions on a single PE against exactly one before repair. It is not even monotone in the quantity it is invoked for: in 33 of the 437 it *raises* peak DMEM. Connectedness is a correlate of memory pressure, not a guarantee of it, so we report the unrepaired pipeline and check the bound directly.

C. DMEM Allocation as Single-Writer Constructor

The `DMEMAllocator` class (`sprs_core.py:5087--5132`) walks `su_postorder` (Sethi-Ullman-refined post-order, sibling-reordered for register pressure). It reserves address 0 on every PE as the identity slot; at each node v on PE $p = \alpha(v)$ it returns to p's free list the slot of every child of v already resident on p, then gives v the head of that free list if it is non-empty and otherwise the next fresh address, advancing `peak_local`$[p]$. A final pass gives every cross-PE edge a persistent receive-buffer slot on the parent's PE, above that PE's local high-water mark, which is never freed. (Pseudocode is in the companion.)

Three properties of that walk give C-sw in the scoped form Argument A2 uses (§III-E): $\mathrm{FBT}(N)$ is a tree, so every node has exactly one consumer; SU-postorder visits children before the parent, so a local slot is freed only after its consumer has read it; and receive slots are allocated in a separate pass and never freed, because the scoreboard-gated `COMPUTE` needs the address to be stable. Single writing is therefore *constructed* rather than assumed — an allocator working in pre-order, or freeing slots before the consumer ran, would break it. The same walk yields C-mem at no extra cost: `peak_local`$[p]$ plus the receive slots is exactly the number of distinct addresses the image will name on p, so the capacity obligation is an $O(N)$ read-off of a quantity the allocator already maintains (§V-B).

D. Schedule Construction

`build_schedule` (`sprs_core.py:5135--5239`) walks `su_postorder` and maintains three per-PE clocks: `gpu_time[g]` (when PE g is next free), `gci_free[g]` (when g's GCI is free to issue the next GENERATE), and `node_ready[n]` (when node n's value is available).

A `GENERATE` on PE g cannot start until $\max(\texttt{gpu\_time}[g], \texttt{gci\_free}[g])$; its data is ready at $t + \texttt{gci\_latency} + 2 = t + 6$. The constant 6 is measured from the RTL: `gci_latency = 4` is the state-machine count from `gci_start` to `gci_done`, plus 1 cycle for the pulse to propagate, plus 1 cycle for DMEM commit. This is *RTL-proven*: the software cost model uses the same constant, and the hardware-observed cycle count matches.

A cross-PE child emits a `SEND` on the child's PE; the parent's `COMPUTE` waits on arrival via scoreboard, not via wall-clock estimation. This decouples estimation error in the cost model from correctness: even if the cost model mispredicts arrival time, the scoreboard gate ensures the parent `COMPUTE` reads the right value — it just stalls for longer.

The operand-binding line is the one that realises C-bind:

```
child_addrs = [addr_map[gpu][c]
               for c in tree.children(v)]
```

where `tree.children(v)` returns $[\mathrm{left}(v), \mathrm{right}(v)]$ by construction. The `addr_map` is the allocator's output; `gpu` is $\alpha(v)$; the children's addresses are bound before the compute executes.

E. Codegen and ABI Verification

The instruction generator (`generate_instructions`) emits cycle-accurate NOPs before each op to advance the instruction index to the scheduled cycle, then the op itself. A two-pass peephole (`_peephole_nop_compress`) collapses NOP runs while preserving the GCI-serialisation gap, and strips trailing NOPs.

C-sw, C-bind and C-leaf are post-conditions on the emitted IMEM (§III-I), checkable without executing anything. The campaign path did not run them in the compile loop, so we discharged them afterwards over the released images: 1,010,895 `COMPUTE` and 731,982 `SEND` instructions across 681 configurations, zero violations, no missing or out-of-range address, and no receive buffer with two writers. We report the check this way — as a static audit of the artifact rather than as a compiler feature that never fired — because that is what was run.

F. Orthogonality of the Compiler Variation Space

By Argument A1 and Proposition 1, compiler variation — the choice among 18 S2, 10 S3, and 122 S4 algorithms — affects *which PE hosts a node* and *when a `COMPUTE` fires*, not *which operand pair is bound*. The campaign (§VI-E) therefore optimises over a space structurally incapable of affecting the bit result. For the mapping axis the statement is stronger and correspondingly less interesting: μ does not reach the emitted image at all except through NOP padding (§V-A), so invariance across S3 is close to tautological, and the evidential weight of the campaign sits on the assignment axis and on reshape across $(G, \text{topology})$. Empirically, every one of the 2,822 HW-validated configurations produces the same 64-bit root result at its leaf count (Table V); they differ only in the cycle count. The remainder of the $\sim$22,000-point space is covered by the argument above rather than by measurement.

VI. Evaluation Methodology

Evaluation proceeds along two axes: *structural* validation of Claim 1 (same bit result across all configurations at the same N), and *performance* characterisation of the same-fabric overhead (cycle count of ABI-compliant schedules relative to non-deterministic schedules on the identical substrate). Both rest on a single cycle-accurate simulation pipeline (Vivado xsim 2025.2).

A. Lower Bound Ω as a Secondary Metric

We compute a composite lower bound Ω on any schedule on the fabric (`compute_lower_bound`, `sprs_core.py:1015`):

$$\Omega = \max\big(\mathrm{lb}_{\text{span}},\ \mathrm{lb}_{\text{comm}},\ \mathrm{lb}_{\text{gather}},\ \mathrm{lb}_{\text{bis}},\ \min_{K\in[K_{\min},K_{\max}]} \max\big(\mathrm{lb}_{\text{gen}}(K),\ \mathrm{lb}_{\text{instr}}(K)\big)\big), \quad (8)$$

where K is the number of PEs a schedule actually populates: $K_{\min} = 1$ and $K_{\max} = \min(G, 2N-1)$, since no schedule can use more PEs than the fabric has or than the tree has nodes. Two of the six terms depend on K — both lb_{gen} and $\mathrm{lb}_{\text{instr}}$ shrink as work spreads over more PEs — so a bound valid against *every* schedule must take the most favourable K available, hence the inner minimisation. Taking a maximum there would assert a bound that some legal schedule beats, which is not a lower bound at all. The four K-independent terms bind regardless and enter the outer maximum directly.

The six constituents are, informally: $\mathrm{lb}_{\text{span}}$, the zero-communication critical-path depth; $\mathrm{lb}_{\text{gen}}(K)$, the GCI serialisation bottleneck; $\mathrm{lb}_{\text{instr}}(K)$, the instruction-throughput bottleneck; $\mathrm{lb}_{\text{comm}}$, a single cross-PE hop floor; $\mathrm{lb}_{\text{gather}}$, the root PE's critical path; lb_{bis}, link serialisation at the topology bisection.

Ω bounds any schedule — deterministic or not — and is therefore not a valid cost-of-determinism denominator by itself. We report Ω as an absolute-performance diagnostic in Appendix D but not as the headline overhead metric.

B. Non-Deterministic Baseline on the Same Fabric

The natural cost-of-determinism metric is the *same-fabric overhead* η: the ratio of SPRS's cycle count to that of the best non-deterministic reducer compiled onto the identical substrate — same NoC, same ALU, same memory hierarchy, same topology — obtained by relaxing C-bind to allow arbitrary operand binding at each internal node. Two such baselines are natural: *ND-Ring*, a linear worker-ring traversal whose reduction order follows the physical ring, and *ND-RecDouble*, recursive doubling in $\lceil \log_2 G \rceil$ merge stages.

We did not build these baselines, and we report no measured η. We also decline to substitute an analytical one. We built such a model — the same cost constants that produce Ω, applied to a ring and to recursive doubling — and it returns $\hat{\eta} \approx 1$ on every instance in the suite. An earlier version of this paper read that as a modelling artefact, on the grounds that a cost model indexed by depth and hop count cannot see which operands pair at each level. That reading was wrong, and the reason is structural rather than numerical.

The ISA's `COMPUTE` is two-input: it names `addr_a` and `addr_b` and nothing else (Table II), and the datapath carries no third operand port. A partial sum of arity $k > 2$ is therefore not expressible on this fabric at all, so every reducer over N leaves has a dependence chain of at least $\lceil \log_2 N \rceil$ combines — which is exactly the depth of $\mathrm{FBT}(N)$ (Definition 2). Measured against the strongest same-fabric alternative, a balanced local tree over each PE's $\lceil N/G \rceil$ leaves followed by $\lceil \log_2 G \rceil$ merge rounds, SPRS is never deeper on any of the 39 hardware-validated instances and is one level shallower on three of them. The tree-depth term cannot be positive here.

At power-of-two shapes the statement is stronger than an inequality on depth. Cutting $\mathrm{FBT}(N)$ at level $\log_2 G$ yields exactly G complete subtrees of exactly N/G leaves, with exactly $G-1$ internal nodes above the cut — which is ND-RecDouble's own decomposition into a local phase and $\lceil \log_2 G \rceil$ merge stages. The two schedules are then DAG-isomorphic: the same instruction multiset, the same dependences, the same PE assignment, differing only in which leaf sits at which position. We verified that construction directly on all 88 pairs with $N = 2^a$, $a \in [3, 13]$, and $G = 2^b$, $b \in [1, a]$, with no exceptions. Both N and G are powers of two on 31

of the 39 hardware-validated instances, so on those $\eta = 1$ is a theorem and no experiment can return anything else. The analytical model returns $\hat{\eta} \approx 1$ because that is the answer, not because it is blind to the difference; we still decline to quote it as a measurement, because it is the compiler's own cost model and a ratio computed from it would be circular whatever value it took.

A baseline would say something on the remaining 8 instances, whose N or G is not a power of two and where the isomorphism does not hold. Building one there is not a small change: relaxing C-bind means re-deriving the DMEM allocator's single-writer discipline and the scoreboard gating around a schedule that no longer has a canonical operand pairing, and the resulting numbers would only be meaningful if the baselines were optimised with the same effort we spent on SPRS — otherwise η measures our relative implementation effort rather than the ABI. We flag that residual in §X (L1b) and price the ABI structurally instead (§VII-E).

We note also that η, even if measured, would not isolate the cost of determinism against a bandwidth-optimised production collective on NVLink or Infinity Fabric silicon. Those systems are out of scope; such a comparison would conflate the ABI cost with the much larger gap between a research prototype and production silicon.

C. Software-Only Baseline

To contextualise the fabric-level mechanism against software-level mitigations, we also compile a third baseline, *SW-FixedTree*, a software-only scheme emulating NCCL tree all-reduce forced to the same tree shape across all $G \in \mathcal{T}$. Because NCCL tree shape is a function of G at the rank level, SW-FixedTree cannot be implemented without either (a) hardware support (which is what SPRS provides), or (b) worst-case padding to the topology-specific tree depth. We report the analytical overhead of (b) as an upper bound on this software-only alternative (Appendix D). A third route — replacing the library's tree outright with a fixed one, rather than reshaping NCCL's — is not this baseline; it is the software export of §VIII-D, which is built, run against stock `ncclAllReduce` and priced there.

D. Instance Suite

Forty instances span $N \in \{8, 10, 16, 32, 50, 100, 128, 200, 500, 512, 1024, 1500, 2048, 3000, 4096, 8192\}$, $G \in \{2, 4, 5, 7, 8, 10, 12, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096\}$ and six topologies (linear, ring, 2D mesh, 2D torus, fat-tree, hypercube). Multiple $(G, \text{topology})$ points at the same N directly exercise the invariance claim (Table V). Per-(N-bucket, topology) cell counts are in Appendix D; some cells are thin due to convenience sampling, a limitation we discuss in §X.

Feasibility bound. DMEM is fixed at 512 entries per PE; IMEM is not. Each testbench elaborates `CN_IMEM` to the next power of two above its own longest per-PE program (64 to 2048 entries here, `sprs_core.py:5380`). 278 of the 7,200 compilations exceed the 512-instruction reference budget and are flagged `E_IMEM_OF`; they were emitted and simulated anyway, and 113 of them are among the 2,822 validated configurations. They cluster at both ends of the N/G range: high N/G piles leaves onto too few PEs (120 flagged configurations each on `large_extreme_1d` and `vlarge_extreme_1d`), while at $N/G = 2$ a PE with few local nodes has many remote children to land. Instruction-memory depth cannot change the root, so the 512-entry budget should be read as the deployment target priced in §IV-I, not as the depth every simulation used.

Leaf values are generated by `make_leaf_val` (`make_leaf_val` in `sprs_core.py`):

$$L[i] \;=\; (i{+}1){\cdot}1000.0 + \big((i{\cdot}37{+}13) \bmod 100\big)/100 + 0.007. \tag{9}$$

The formula is chosen so that every leaf carries a non-trivial fractional part: the additive $((37i{+}13) \bmod 100)/100{+}0.007$ term guarantees a populated mantissa, so each FADD exercises the guard, round and sticky bits rather than reducing to an exact power-of-two addition. Magnitudes span 10^3 to 8×10^6 across the suite, so partial sums accumulate across a wide exponent range and reassociation is observable in the low-order bits.

We deliberately do not claim that reordering *always* changes the result. It does not: at these magnitudes a sufficiently small perturbation is absorbed by rounding, which is precisely what the mutation protocol of §VI-F has to work around. What the value distribution buys is that a bit mismatch between fabric and oracle is a determinism violation rather than an artefact of degenerate inputs; the converse — that every violation must show up as a mismatch — is established by mutation testing, not by the choice of leaf values.

E. Campaign Protocol

The per-instance configuration space is $18 \times 10 \times 122 = 21{,}960$ (α, μ, r) triples; simulating it exhaustively is infeasible ($\approx$880,000 xsim runs). We fix refinement at the identity and enumerate the remaining 18×10 (α, μ) face *exhaustively* on every instance — 180 of the 21,960 triples per instance, 0.82% of the space, 7,200 compilations, of which 6,777 emit a program image (390 size-gated, 33 evaluation timeouts). Refinement is fixed at `None` throughout and is covered by the orthogonality argument of §V-F rather than by measurement. The campaign is one pass: compile, deduplicate, simulate.

Testbench-hash dedup. Distinct (α, μ) pairs frequently produce identical program images up to PE relabelling. A SHA-256 hash of the emitted testbench collapses the 6,777 images to 2,920 distinct ones (56.9%); followers inherit the leader's cycle count. This is not merely a compute saving: identical images *tautologically* produce identical bit results, so dedup is what makes each remaining xsim run informative.

No shortlist. Every distinct image was queued for xsim — 12 to 142 per instance, median 75, covering 152 of the 180 pairs — so no selection of any kind stands between the compiler and the invariance evidence. The one exception is `maxp_mod_fat`, dropped from the HW-validated suite: five of its 76 images were attempted and each exhausted the 8 h simulator budget; the other 71 were not run. The budget was not the binding constraint, and §X (L0) reports what

was. A top-K-by-`makespan` shortlist would have missed a median 20% of the true top-K by measured cycles at $K = 15$ (Appendix E), which is why we did not use one.

Campaign accounting. On the 39 remaining instances 2,844 distinct images were attempted, 2,822 settled and passed, and 22 did not settle. Twenty-one of the 22 are fabric fail-stops — their retained per-PE counters stop at a last retirement plus exactly `WATCHDOG_MAX` = 20,000 cycles, far inside the testbench budget — and are the campaign's only observation of Property 1 on unmutated compiler output; the twenty-second is a simulator wall-clock kill. The campaign cost 1,003.3 core-hours: 821.5 on passing runs, 122.9 on runs that did not settle (retries included), 58.9 on `maxp_mod_fat`.

F. Mutation-Testing Protocol

To confirm that the oracle harness can detect bit divergence (i.e., that the zero-divergence result is non-vacuous), we inject two mutation classes per instance and record the detection rate:

Address substitution.
In one randomly chosen `COMPUTE`, repoint `addr_a` at a different DMEM address. Breaks Argument A1 while preserving A2–A4.

Leaf corruption.
Perturb one leaf value by the smallest single-bit flip that provably moves the canonical root. Breaks C-leaf.

The escalation in the second class is necessary, not incidental, and it is the same phenomenon the paper is about. Flipping the mantissa LSB is frequently *not* observable at the root: at these operand magnitudes the perturbation falls below the round bit of the running sum and RNE discards it, leaving the canonical root unchanged. A fabric reporting `PASS` on such a mutant is correct to do so, and counting it as a missed detection would understate the oracle. We therefore raise the flipped bit position until the software oracle confirms the root has actually moved, so that every counted mutant is a test of detection rather than a test of whether rounding happened to preserve the value.

We deliberately do not use operand-swap as a mutation: FADD is commutative at the bit level under FTZ+RNE ($\mathrm{fadd}_{\mathrm{rtl}}(a,b) = \mathrm{fadd}_{\mathrm{rtl}}(b,a)$ for all 64-bit (a,b)), so operand-swap is bit-identical and would spuriously inflate the false null-detection rate.

G. Metric Convention

`hw_cycles` is the primary metric: `hw_cycles` $= \max_p$ `perf_total_cycles`$[p]$ as measured by per-PE performance counters in Vivado xsim. `makespan` from the software cost model is a diagnostic only. Summary statistics are reported as medians with observed ranges rather than as interval estimates: the per-instance samples are not independent draws from a population — they enumerate a deterministic configuration space — so a resampling interval would describe our sampling of that space rather than any underlying uncertainty. Per-topology breakdowns are reported where cell counts permit; thin cells are labelled.

VII. Evaluation Results

A. Setup

Vivado xsim 2025.2 on Linux; ten SystemVerilog sources in `rtl/` (3,738 lines). The tournament runs on a 72-thread Intel Xeon w9-3475X workstation with 503 GB of RAM and RNG seeds pinned at 42. Per-algorithm timeout 1800 s; per-simulation wall-clock cap 28,800 s. Per-instance xsim wall-clock is in Table X (Appendix D).

Every cycle figure reported in this paper comes from a single campaign of 2,822 passing runs consuming 821.5 core-hours, at a median of 99 s per testbench; the five largest instances account for 90.0% of that total and the single longest run took 5.51 h. An earlier campaign over the same testbenches is not used for any reported number: it recorded PE 0's performance counter rather than the maximum across PEs, which understates end-to-end latency whenever the reduction root lands elsewhere. That defect moved 50.5% of cycle counts and changed the *minimising configuration set* on 38 of 39 instances, leaving it disjoint from the corrected set on 32, so the measurements were repeated in full rather than corrected in place. We report it that way round because "the winning configuration" is not a well-defined object on this data (§VII-F). Hardware validation covers 39 of the 40 instances (§X).

B. The Premise, Measured

The ABI is worth its cost only if reshaping a production collective really does move the bits. Earlier versions of this paper argued that from the arithmetic and from the libraries' own documentation; we have since measured it. On a single node with two NVIDIA RTX PRO 6000 Blackwell GPUs over PCIe (no NVLink) and 72 CPU cores, we reduced identical input *bytes* at fourteen rank counts from 1 to 64 under PyTorch 2.11.0's distributed collectives [2], holding the data, the seed and the code fixed so that only the shape of the job changed. Leaf i's value is a pure function of i, so reshaping alters who holds which leaf and in what order the partial sums combine, and nothing else.

The fourteen configurations returned fourteen distinct 64-bit results, in every one of twelve (leaf count, precision, value-profile) cases; in the six `fp64` cases, for which the single-process canonical reduction was also computed, none of the fourteen matched it. Spreads reach 5,120 ulp, a maximum relative spread of 8.1×10^{-13}, and at least 1,023 of the 1,024 elements of the result vector move in every case; pooled over every configuration swept, 78.6% of `fp64` configuration pairs and 90.0% of `fp32` pairs disagree. Repeated runs at any *fixed* configuration were bitwise identical, five times out of five, with no disagreement between ranks inside a run, and an independent re-run of the sweep under a different bookkeeping harness reproduced 168 of 168 common cells bit for bit. Reshape moves the answer; noise does not. That decomposition — determinism at a fixed shape, divergence across shapes — is the failure the ABI is built to remove, and §VII-C is the same experiment on a fabric that holds the shape fixed by construction.

Three scope conditions belong with the measurement rather than under it. The rank-count sweep runs on the CPU backend, because NCCL requires one rank per device and this node carries two GPUs, so partitions above $G{=}2$ could not be placed on them. At $G{=}2$ the NCCL algorithm-by-protocol axis (seven configurations) and the channel-count axis (three) each returned a single root; that is structural and not an empirical null, because at two ranks the cross-rank reduction is one floating-point addition per element and no schedule can reassociate it, and we do not generalise it to larger G. At that same partition the CPU and GPU roots differ in all twelve cases, so device placement moves the bits as well as partitioning does. This measures reduction order on one node; it is not a benchmark of any library and no timing claim is drawn from it.

C. Cross-Topology Bitwise Invariance

We state the paper's primary empirical result in the narrowest form the campaign supports. Across the 2,844 distinct ABI-compliant program images obtained by compiling the complete 18×10 assignment-by-mapping grid on the 39 hardware-validated instances — sixteen leaf counts $N \in [8, 8192]$, requested worker counts $G \in [2, 2048]$, six direct-network topologies — 2,822 simulations terminated, and in every one of them the root PE's final `COMPUTE` result was bit-identical to the canonical root $R_{\text{canonical}}(N, L)$ the software oracle predicts for that N. The remaining 22 did not settle (§VI-E); no configuration in the campaign produced a finite root differing from $R_{\text{canonical}}(N, L)$.

Two scope conditions belong inside that sentence rather than in a footnote. The campaign evaluates one leaf vector per N and the identity refinement throughout. And eight of the sixteen leaf counts are represented by a single requested $(G, \text{topology})$ point: at those N invariance is witnessed across assignment, mapping and schedule, but not across reshape. 2,403 of the 2,822 configurations lie at leaf counts carrying two or more points, and that is where the reshape evidence sits.

Where reshape is exercised it is not marginal. At $N{=}128$ the same root survives six $(G, \text{topology})$ points spanning $G{=}4$ on a linear chain to $G{=}128$ on a torus — a $32\times$ change in worker count across five distinct topologies — and at $N{=}512$ and $N{=}1024$ it survives six and five points respectively. The 2,844 images are moreover distinct *programs*, not repeated runs of one program: the campaign deduplicates by SHA-256 over the emitted testbench and simulates one representative per hash class (§VI-E), so two images in the count differ structurally, not merely in the compiler triple (α, μ, r) that produced them. Invariance therefore holds simultaneously under reshape and under complete replacement of the assignment and mapping algorithms.

The campaign has since been re-executed from source. All 1,326 passing configurations at $G \leq 64$ — every one of the twenty-five suite instances at that scale, spanning all six topologies — were recompiled and re-simulated under both route-table loading paths (§X, L0b), and on the 1,306 points that completed under both, the two paths agree with each other and with the software oracle on the same 64-bit value; each of the twenty-five instances yields exactly one distinct root across all of its pipelines. The released results file records verdicts and cycle counts but not root words, and the RTL re-simulated here is the current tree rather than the exact one that produced that file, so this is a re-execution that self-agrees and re-derives the oracle's prediction — not a bitwise diff against the originally released values. We state it in those terms.

Table V reports the result. All HW-validated configurations at each N produce the same 64-bit root, exactly matching the software oracle, and the divergence column is zero as predicted by Claim 1. Its rows are not equally informative, and §III-A measures why: at the smallest leaf counts a large majority of alternative association orders reproduce the canonical root anyway, so agreement there is close to free and the weight of the evidence for Claim 1 sits at the large-N rows.

A zero-divergence column is only meaningful if the oracle can report a non-zero one, which is what the injected-fault campaign is for (§VI-F). Reporting it requires more care than a single rate. The harness retains one boolean per mutant, and the testbench prints `[FAIL]` both on a value mismatch and from its `!all_done` branch on any run that fails to complete; the two are not distinguishable in the recorded output. Reconstructing the mutations against regenerated images separates them. Of the 53 address-substitution verdicts, in at least 42 the value comparison never ran: the substituted address is one no agent ever writes, so the dependent `COMPUTE` stalls on the presence gate and the flag is a fail-stop, not a detection. Oracle non-vacuity therefore rests on 61–65 of the 107 verdicts — every leaf corruption, plus the address substitutions that happened to reach a live slot — and not on all 107, as we previously reported.

Read the other way round, the same fact is an asset rather than a concession: 43 of the 54 address substitutions fail-stopped instead of corrupting, and in every one we re-simulated the fabric raised `P_ERROR` with a recorded `error_pc` on every PE. That is Property 1's halt outcome observed under injected fault rather than argued from the RTL, and §III-D sets out why the presence gate draws the boundary exactly there.

Extrapolation beyond the measured range. Bit-result invariance at untested G rests on the structural argument of §III-D: operand binding (A1) and scoreboard gating (A3) remain total functions of N and L as G grows, so adding PEs changes the *location* of FADD operations but not the operand pairs they receive. This is not a formal induction on G: adding a PE introduces new routing paths, new VC contention, and new credit-return interactions that may affect cycle counts. What carries is the bit-result invariance under the fifteen assumptions; absolute cycle counts at large G are cycle-accurate simulation artefacts, not silicon measurements, and we make no absolute competitiveness claim.

Coverage of the injected-fault campaign. Mutants were injected across all sixteen leaf-count classes, spanning N from 8 to 8192, so no row of Table V rests on an oracle that was never exercised. One further mutant, on the largest instance, produced no verdict at all and is excluded rather than counted either way. The leaf-corruption column additionally

TABLE V
CROSS-TOPOLOGY BITWISE INVARIANCE. EACH ROW AGGREGATES ALL HW-VALIDATED SPRS CONFIGURATIONS AT THE GIVEN N OVER ALL SAME-N $(G,$ TOPOLOGY$)$ PAIRS AND ALL COMPILER TRIPLES. "DIVERGENT" IS BY CLAIM 1 EXPECTED TO BE ZERO. THE TWO MUTATION COLUMNS COUNT INJECTED MUTANTS THAT THE HARNESS *flagged* (§VI-F); AS DISCUSSED BELOW, A FLAG IS A VALUE MISMATCH FOR LEAF CORRUPTION BUT USUALLY A FAIL-STOP FOR ADDRESS SUBSTITUTION, AND ONLY THE FORMER BEARS ON ORACLE NON-VACUITY.

N	compiled $(G,$ topo$)$	HW-val. $(G,$ topo$)$	HW configs (passing)	64-bit root result	Div.	Addr-sub detected	Leaf-corr. detected
8	2	2	28	`0x40E1948E978D4FDF`	0	6/6	6/6
10	1	1	17	`0x40EADBA0A3D70A3E`	0	3/3	3/3
16	1	1	21	`0x41009A44BC6A7EFA`	0	3/3	3/3
32	3	3	102	`0x41201D1FCED91687`	0	6/6	6/6
50	1	1	25	`0x413374911999999A`	0	3/3	3/3
100	1	1	33	`0x415343B08CCCCCCD`	0	3/3	3/3
128	6	6	371	`0x415F7E8FF9581062`	0	6/6	6/6
200	1	1	44	`0x41732B4046666667`	0	3/3	3/3
500	1	1	98	`0x419DDCAB2C000000`	0	3/3	3/3
512	6	6	456	`0x419F4FA404418938`	0	6/6	6/6
1024	5	5	425	`0x41BF47D2026872B1`	0	2/2	2/2
1500	1	1	104	`0x41D0C665F8400000`	0	2/2	2/2
2048	4	4	437	`0x41DF43E900FBE76D`	0	2/2	2/2
3000	1	1	77	`0x41F0C4F764200000`	0	2/2	2/2
4096	3	3	331	`0x41FF41F48085A1CB`	0	2/2	2/2
8192	3	2	253	`0x421F40FA40427EFA`	0	1/1	2/2

All hex values are 16-digit 64-bit, positive-signed, with exponents consistent with the magnitudes produced by (9). "Compiled" counts the distinct $(G,$ topology$)$ points at that N for which an ABI-compliant image was produced; "HW-val." counts those carried through RTL simulation. The single shortfall is N=8192, where `maxp_mod_fat` (G=4096, fat-tree) could not be simulated; the reason is not a wall-clock budget (§X).

depends on the bit-escalation protocol of §VI-F: without it the column would measure how often FP64 rounding absorbs a perturbation, not how often the oracle detects one.

D. No Silicon Measurements

Every cycle figure in this paper comes from cycle-accurate RTL simulation. We ship Artix-7 (Nexys A7) bring-up wrappers for $G \in \{2, 4\}$, but synthesising them additionally requires a Vivado VIO IP core and board constraints that we do not redistribute, and we hold no board logs; we therefore report no board-measured cycle count and make no silicon-timing claim. This is a reporting gap, not a soundness gap: the invariance result of §VII-C is a statement about bit values, which xsim resolves exactly, and Claim 1 is argued structurally rather than by extrapolation from a boundary point.

E. The Cost of the ABI

The natural way to price the ABI would be to run, on this same fabric, the best *non*-deterministic reducer we could construct — one free to reassociate and to reshape its tree per $(G, \text{topology})$, though not to pick k-ary partial sums, which this ISA cannot express (§VI-B) — and report the ratio η of the two makespans. We did not build that baseline, and we therefore report no measured η. We say so plainly because the alternative — quoting a ratio against a reducer that does not exist — would not be a measurement.

One caveat attaches to every cycle number in this subsection and is not repeated below: all of them come from cycle-accurate RTL simulation of a single-chip fabric, so they price the schedule rather than the wire. Two further caveats stood here in an earlier version and have since been discharged by measurement rather than by argument: the route-table bypass that every run uses contributes exactly zero uncertainty to these cycle counts (§X, L0b), and the premise that reshaping a production collective changes its result is now measured (§VII-B). What is still not measured is η itself, and that is a decision rather than an omission: the only quantity obtainable without building the baseline is the compiler's own cost model, which §VI-G admits only as a diagnostic, so a ratio computed from it would be circular whatever it said. That the model returns $\hat{\eta} \approx 1$ is nonetheless the right answer on most of the suite, and §VI-B establishes it by construction rather than by model.

What can be said without a baseline is stronger than it first appears. The ABI's cost is not an empirical accident to be discovered by benchmarking; it is a consequence of the contract, and the contract is small enough to price analytically. We give the decomposition below, then measure the one term of it that this suite can isolate.

1) Structural decomposition: Four constraints, each a direct consequence of the ABI rather than of our implementation, separate SPRS from an unconstrained reducer.

1) *FBT tree depth — which costs nothing here.* The ABI fixes $\mathrm{FBT}(N)$, so the dependence chain is $\lceil \log_2 N \rceil$ deep regardless of G. A reducer free to form partial sums of arity k would be $\log_k N$ deep, and off this fabric that is a real saving; on it, arity $k > 2$ is not expressible, because `COMPUTE` names two DMEM addresses and the datapath has no third operand port. Every same-fabric reducer is therefore at least $\lceil \log_2 N \rceil$ deep, which is exactly $\mathrm{FBT}(N)$'s depth, and this term is at most zero on all 39 hardware-validated instances (§VI-B).
2) *Operand-binding forbids reassociation.* Fixing $(\mathrm{left}(v), \mathrm{right}(v))$ at every internal node forbids

regrouping operands by locality. An unconstrained reducer may sum PE-local values first and cross the network once; SPRS must cross wherever the tree says to cross, even when a locality-adjacent regrouping would be cheaper.

3) *GCI serialisation.* Consecutive `GENERATE`s on one PE are spaced by $\texttt{gci\_gap} = \texttt{gci\_latency} + 2 = 6$ cycles. A PE holding $\lceil N/G \rceil$ leaves therefore spends at least $6\lceil N/G \rceil$ cycles producing them, and this floor grows with the leaves each PE holds, so it binds hardest at *high* N/G — the regime in which the reduction has the most work to overlap it with, and still cannot. Unlike term 2, this term is an artefact of *our* GCI, not of the ABI, and a multi-channel leaf interface would remove it.
4) *Root-gather fan-in.* The root PE must receive contributions from the other $G-1$ PEs, giving at least $\lceil \log_2 G \rceil$ merge stages plus one traversal of roughly half the network diameter. This is $\mathrm{lb}_{\mathrm{gather}}$ in (8) and it binds any reducer, deterministic or not.

Term 2 is the price of reproducibility and cannot be engineered away without abandoning the contract; term 1 is that price in a form this fabric cannot charge, and reports zero. Terms 3 and 4 are shared with any reducer on this fabric; term 3 is ours to fix, term 4 is nobody's.

2) Where the champion's cycles go: Term 3 can be measured directly rather than through a ratio, and that is how we now report it. The GCI floor $6\lceil N/G \rceil$ accounts for at most 30% of the champion makespan on every one of the 7 instances with $N/G \le 2$, and for at least 43% on every one of the 14 instances with $N/G > 8$, rising to 64–74% on the 5 instances with $N/G \ge 32$ (Spearman $\rho = +0.77$, $n = 39$). The two groups do not overlap. Since $6\lceil N/G \rceil$ assumes a perfectly balanced leaf assignment, and any real assignment puts at least $\lceil N/G \rceil$ leaves on the busiest PE, these are *lower* bounds on the true share. Leaf-generation serialisation is therefore the dominant term at high N/G, which is the quantitative backing for the remark under term 3 that a multi-channel leaf interface would remove it: on this suite that single interface change is the highest-value modification available to the fabric, and it costs nothing in the ABI.

A prediction we withdraw. An earlier version of this section tested term 3 by banding cycles/Ω against N/G and read the resulting decline as a successful prediction of a knee at $N/G \in [6, 8]$. That test is circular and we withdraw it. Ω equals $6\lceil N/G \rceil$ on 29 of the 39 instances, so the term under test is the denominator; the absolute distance $\mathrm{cycles} - \Omega$ carries no N/G signal at all ($\rho = +0.09$, $p = 0.58$); and a one-parameter model with no mechanism in it, $\mathrm{cycles} = \Omega + 96$, reproduces the published band medians at and above the predicted knee to within 0.13. The banded table, the per-instance figure and a scan over candidate knee locations are in the companion document. Nothing is lost by the withdrawal: the direct measurement above does not involve Ω, and it points the same way term 3 does — indeed the opposite way to the withdrawn reading, since GCI serialisation binds *above* the claimed knee, not below it.

3) Distance to the lower bound: The one absolute statement we can make is how far the compiler lands from Ω, the composite lower bound[1] of §VI-A. Because Ω bounds *any* schedule on this fabric — including a non-deterministic one — the ratio cycles/Ω upper-bounds what an ideal unconstrained reducer could have saved. It is a weaker statement than η, but it is a real one, and it is measured rather than asserted.

TABLE VI
DISTANCE TO THE COMPOSITE LOWER BOUND Ω BY TOPOLOGY. Ω BOUNDS ANY SCHEDULE ON THIS FABRIC, SO THESE RATIOS UPPER-BOUND WHAT AN UNCONSTRAINED REDUCER COULD HAVE RECOVERED.

Topology	instances	median cyc/Ω	best
linear	4	1.39	1.35
hypercube	7	1.85	1.55
torus	10	2.26	1.49
mesh	4	2.58	1.62
fat-tree	8	3.40	1.83
ring	6	4.48	2.33
all	39	2.34	1.35

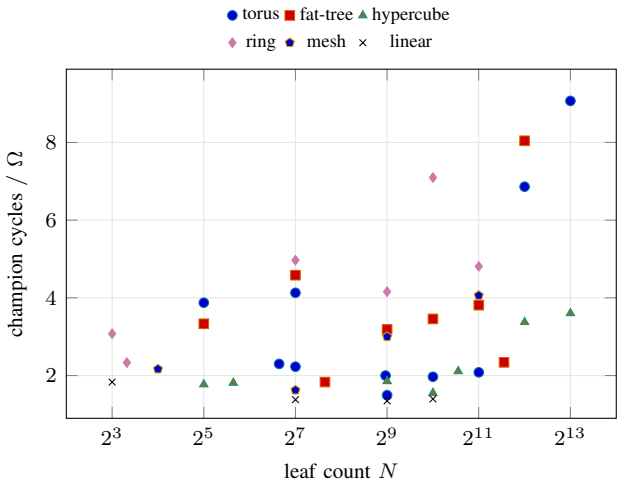


Fig. 5. Distance to the composite lower bound Ω for every HW-validated instance, against leaf count. A value of 1.0 would mean the compiler achieved the bound exactly; the bound is not known to be tight, so the gap is part slack and part real.

Figure 5 shows the per-instance picture behind Table VI. The informative reading of it is additive rather than multiplicative: the compiler's absolute distance from the bound is near-constant — median 96 cycles, interquartile range $[60, 147]$ — across a range of Ω from 13 to 384, and it is uncorrelated with N/G ($\rho = +0.09$, $p = 0.58$). The schedule carries a roughly fixed overhead over the analytic floor regardless of problem size, so its relative efficiency improves monotonically with scale and reaches $1.35\times$ at the largest instance. No instance reaches Ω, which is expected: the bound is assembled from separately-derived constraints and is not claimed to be tight.

Table VI is a descriptive per-topology summary and not a controlled contrast, and we draw no topology ordering from it. The six cells differ in median N/G by a factor of 24, the suite contains only two (N, G) points carrying more than one

[1] As implemented, Ω additionally evaluates a GCI-aware Hu/HLF term and derives $K_{\min}$ from IMEM depth as $\lceil N/86 \rceil$ rather than fixing it at 1. Neither is the argmax on any of the 40 instances, so (8) as printed reproduces every Ω reported here; the discrepancy is documentary, not numerical.

topology, and those two disagree in sign. An earlier version of this section read the column as ordering with bisection width; that reading is substantially N/G in disguise and we withdraw it.

F. Champion Equivalence Classes

Earlier versions of this paper reported a winner distribution: which assignment algorithm and which mapping algorithm produced the fastest image on each instance. We withdraw it, because it was not a measurement, and the reason it was not is the paper's own subject.

The campaign deduplicates by program image before simulating (§VI-E), so one simulated row stands for a class of bit-identical programs, and the champion of an instance is a *set* of configurations rather than a configuration. Expanding those classes back over the full 18×10 grid: the minimum measured cycle count is attained by more than one configuration on 38 of the 39 instances, by a median of ten configurations and by as many as 90; all ten mapping algorithms attain it on 27 of 39; and exactly one instance in the suite has a unique champion. Which member of a tie set the published table named was decided by a minimum over CSV rows in parallel-worker arrival order — that is, by which simulation host happened to finish first. A paper about scheduling nondeterminism had its winner table ordered by scheduling nondeterminism, and saying so is more useful than quietly re-sorting it: no tie-break repairs the table, because on 38 of 39 instances the tied set contains programs that are bit-identical and therefore cannot be separated by any measurement at all.

What replaces the ranking is stronger, and the data for it was already in the paper. Proposition 1 says the compiler's variation space cannot reach the bit result. The campaign shows that same space collapsing structurally before it ever reaches the fabric: of the 7,200 compilations performed, 59.4% produce a program image that another configuration had already produced (§VI-E). At the optimum the collapse is near-total. The compiler variation space is thus not merely incapable of changing the answer — over most of its extent it does not even yield a different program. The tournament is, from the ABI's point of view, 2,822 attempts to break Claim 1, every one of which failed; it is not a ranking of algorithms.

G. A Derived Selection Rule

The tournament was built to answer a practitioner's question — given an instance, which of the 180 compiler configurations should be run? — and champion-class membership lets us answer it in a form that is invariant to the tie structure just described. For a recommended set of configurations we report *regret*: the measured cycle count of the best member of that set, divided by the instance's measured optimum. A regret of 1.000 means the set contains a champion.

The rule is a portfolio, not a regime map. Exhaustive search over subsets of the sixteen near-universally evaluable assignment algorithms, minimising mean measured regret and refitted independently in each of 39 leave-one-out folds, returned the *identical* subset in 39 of 39 folds at every size. At size four that subset is `DSC`, `Lukes`, `Frederickson`, `ParSub`, and on the held-out instance it attains the exact hardware-measured optimum in 38 of 39 folds (0.974), with mean out-of-sample regret 1.0025 and worst case 1.099. It evaluates 4 of the 180 configurations, a median $144.6\times$ reduction in compiler time ($205.6\,\mathrm{s}$ to $2.78\,\mathrm{s}$ per instance). Table VII gives it, together with the smaller budgets.

Conditioning on the instance makes it worse, not better. This is the result we did not expect and the one that makes the portfolio finding non-obvious. Every decision tree we could fit over $(N, G, N/G, \text{topology})$ is beaten out of sample by the unconditional portfolio: the best of them reaches an exact-optimum rate of 0.564 (22 of 39) against 0.974, and its out-of-sample mean regret, 1.21–1.31, is worse even than the single-algorithm constant rule "always `ParSub`" (1.0797). A regime map over these axes is not merely unnecessary here; it is actively harmful, because it fits partitions of a 39-instance suite whose cells hold two or three instances each. The simplest conditioner of all fails the same way: taking whichever algorithm was best on this instance's topology in training attains the measured optimum on 21 of 39 held-out instances against 38 for the unconditional portfolio, at mean out-of-sample regret 1.0488 against 1.0025. Topology is the one axis on which conditioning is most intuitive, and it is the axis on which conditioning most clearly does not pay. Against the two pre-specified constant baselines the portfolio is significant on both confirmatory tests (McNemar $p = 0.0005$; Wilcoxon signed-rank on regret $p = 0.0003$; both survive Holm correction over the two comparisons).

It transfers to a topology the fit never saw. Leave-one-out holds out an instance but not its topology, and instances sharing a topology are not independent, so we also refit the portfolio with an *entire topology* withheld and score each fold only on the topology it never saw. Across the six topologies of the suite the portfolio transfers better than conditioning generalises: with the topology entirely unseen its mean out-of-sample regret is 1.0391 (95% bootstrap interval $[1.0025, 1.0885]$ over 2,000 resamples of the 39 instances), lower than the 1.0488 of a rule that conditions on topology and does see it in training. The exact measured optimum is still attained on 35 of 39 instances (89.7%), with a worst case of 1.675. Withholding an entire leaf-count band instead leaves the leave-one-out figure unchanged (1.0025). We quote the interval and not the point estimate deliberately: with only six topologies the grouped estimate is genuinely uncertain at the top end, and a practitioner adopting the rule on an unseen topology should plan against 1.0885 rather than against the mean. The corresponding leave-one-out interval is $[1.0000, 1.0076]$. Size four is a knee rather than a budget we picked: a fifth configuration is identical to four decimal places under both protocols (1.0391 with a topology held out), and only at six does held-out regret reach 1.0000.

The mapping stage can be deleted. Assignment is the whole decision. Conditional on `ParSub`, the ten mapping algorithms give byte-identical cycle counts on all 39 instances. Fixing the mapping at `Identity` for every instance and choosing only the assignment attains the exact optimum on 38 of 39, at a mean cost of 0.046% and a worst case of 1.8%. We therefore recommend removing the mapping search from the pipeline

TABLE VII
DERIVED ALGORITHM-SELECTION RULE. FOR EACH REGIME, THE S2 ASSIGNMENT ALGORITHMS A PRACTITIONER SHOULD RUN; THE S3 MAPPING MAY BE FIXED TO IDENTITY THROUGHOUT (SEC. VII-G). "OPT." COUNTS INSTANCES ON WHICH THE RECOMMENDED SET ATTAINS THE HARDWARE-MEASURED MINIMUM CYCLE COUNT EXACTLY.

Regime	n	Run these S2 algorithms	Opt.	Mean regret	Worst regret
Default (any N, G, topology)	39	DSC, Lukes, Frederickson, ParSub	38/39	1.003	1.099
restricted to non-ring	33	Lukes, Frederickson, ParSub	32/33	1.003	1.099
restricted to ring†	6	DSC, ParSub	4/6	1.019	1.063
Budget k=1: $N \geq 2048$	10	ParSub	7/10	1.094	1.607
Budget k=1: $N < 2048$, non-ring	24	Lukes	17/24	1.076	1.853
Budget k=1: $N < 2048$, ring†	5	DSC	2/5	1.092	1.300
Budget k=2 (any instance)	39	DSC, ParSub	19/39	1.030	1.220
Budget k=3 (any instance)	39	DSC, Lukes, ParSub	34/39	1.011	1.118

†Rests on six (resp. five) ring instances of which three are positive; reported for completeness, not recommended as a standalone rule. No (N-bucket × topology) cell in the suite holds more than three instances, so no cell-level rule is derivable.

rather than conditioning on it. Note what this does *not* say: `Identity` is not the best mapping — its apparent lead in the withdrawn winner table was a labelling artefact (§VII-F) — it is indistinguishable from the others and the cheapest of them to compute. In the other direction, the spread across assignment algorithms is three orders of magnitude larger. Six of the eighteen are in the champion class on *zero* of 39 instances and cost a median $2.3\times$ to $13.1\times$ in cycles, up to $51.5\times$ at worst. Greedy list scheduling is the block to avoid.

What the rule does not license. Nothing here survives Holm correction over the 36 exploratory association tests we ran between algorithm families and instance parameters; two survive Benjamini–Hochberg, and we report the portfolio rather than any of them precisely because the portfolio needs no such association to hold. N and G are confounded across this suite by construction (Spearman $\rho = 0.871$, $p = 5.6 \times 10^{-13}$), so the budget rows of Table VII that split on scale cannot be attributed to either variable separately. Of the 23 populated (size × topology) cells, 18 hold two instances or fewer, so no cell-level recommendation is derivable and we make none. The rule is untested at $G = 4096$, the one design point without a hardware result (§X). And it is a retrospective statement about measured cycle counts, not a claim that the compiler can identify the winner from its own cost model — Appendix E measures exactly how far that falls short. Every number is a measured cycle count on this pipeline for tree-structured reduction programs; none of it transfers to another compiler, another accelerator, or to MPI or NCCL collectives.

VIII. DISCUSSION

A. What the ABI Buys

The paper's contribution is not the compiler's algorithms or the fabric's microarchitecture — both are composed from standard prior art — but the interface observation: if compiler and fabric agree on a size-indexed reduction tree as a shared ABI, then cross-topology bitwise invariance follows from composition. Every subsequent design decision in the compiler and fabric amounts to *discharging obligations* (C-*) or *establishing invariants* (F-*); none of them changes the shape of the argument.

The practical payoff is that SPRS users can reshape their cluster freely — from $G = 2$ to $G = 4096$, from linear to hypercube — without touching their code, and still get the same bits out. Regression testing becomes bit-for-bit across hardware generations. Replay debugging survives migration to new silicon. ML training, rerun three months later on a larger cluster, produces the same weights. These are engineering outcomes that production collectives cannot provide today, and that single-node techniques cannot address at all.

B. When Not to Use the ABI

Three regimes make SPRS a poor fit.

Interoperability with external reducers. SPRS's bit result agrees across any reshape *within* its own fabric-compiler pair, but does not match NCCL, RCCL, ReproBLAS, ExBLAS, or any reducer using a different canonical order. A deployment that must consume or produce bit-compatible results with an external tool needs either (a) a one-time canonical-form translation, or (b) an input permutation adapter that presents leaves in the external tool's canonical order — which sacrifices the cross-reshape guarantee, trading one form of reproducibility for another.

Strict IEEE 754. Workloads whose reference implementation exercises subnormals, multiple rounding modes, or the signed-zero cancellation rule of IEEE 754-2019 §6.3 will see SPRS diverge from reference on those regimes (§III-G). The narrower FTZ+RNE semantics was a scope decision; a strict-IEEE FADD implementation inherits Claim 1 unchanged.

Bandwidth-critical inner loops. The structural asymptote determined by $\log_2 N$ versus $\log_G N$ is a real cost in inner loops where every cycle is contested. Workloads that can tolerate reshape non-reproducibility should use a non-deterministic reducer; the ABI buys invariance at a non-zero price.

C. Generalisation to Other Deterministic Operations

Claim 1 requires only (i) a deterministic binary operation (for us, $\mathrm{fadd}_{\mathrm{rtl}}$) and (ii) a size-indexed canonical tree. It does not require floating-point, associativity, or any particular algebraic property. Any associative or non-associative binary operation — FP32/FP16 ADD, integer ADD with saturating or modular arithmetic, bitwise OR/AND/XOR, integer MAX/MIN, monoidal aggregations — inherits the claim by substituting the `alu_op` code. Reductions over ordered types with a non-associative combine (string concatenation, stream merges with priority) inherit the claim trivially because the canonical tree *is* the order.

Tensors follow by element-wise extension: reducing a tensor of D elements under an ABI-compliant schedule is D independent scalar reductions. The fabric does not need per-element indexing; the compiler emits D parallel IMEM images, one per element, sharing the same routing and the same PE assignment.

Non-reduction collectives — all-gather, scatter, broadcast, alltoall — are orthogonal to the ABI: they have no intermediate combine, so their results are bit-identical by construction on any routing discipline. The ABI is specifically the statement that reductions, the one collective with an internal combine, can be reproducible too.

D. Exporting the ABI: Pinned-FBT for Production Collectives

Can the ABI discipline be adopted without the SPRS fabric? It can, in software: replace a production collective's dynamic G-dependent reduction tree with a library that enforces the $\mathrm{FBT}(N)$ post-order schedule whatever G and the topology are. We call that *pinned-FBT*. Two things have to be said before the construction is stated, because we got the first wrong and the second is not ours.

The construction we published is incorrect. It partitioned the canonical leaf indices into contiguous ranges, $\mathrm{rank}(i) = \lfloor i \cdot G/N \rfloor$, and asserted that each rank then reduces exactly the subtree of $\mathrm{FBT}(N)$ rooted at the lowest common ancestor of its leaves. That assertion is false. Swept over all 44,850 pairs with $2 \le N \le 300$ and $2 \le G \le N$ it holds on 1,217 of them (2.71%); the smallest counterexample is $N = 3$, $G = 2$, where rank 0 owns canonical leaves $\{0, 1\}$ whose lowest common ancestor is the root of the whole tree; and it fails on 8 of the 40 instances this paper evaluates. Nor is it merely a description error. Evaluated literally in FP64 on our own leaf vector (9), the composed construction returns a 64-bit root differing from $R_{\mathrm{canonical}}(N, L)$ on 15,630 of those pairs, on 45.2% of the 1,546 pairs the suite's own leaf counts admit, and on two of our own instances — $N = 10, G = 4$ by 2 ulp and $N = 200, G = 12$ by 1 ulp. The discrepancy is 1–3 ulp throughout, which is small in magnitude and is nonetheless total failure of a bitwise claim. A second, independent error survives any repair of the first: an all-reduce over sub-roots "consistent with $\mathrm{FBT}(N)$'s top $\log_2 G$ levels" is $\mathrm{FBT}(G)$ only when N and G are both powers of two, so the root differs even at pairs where the partition does happen to be subtree-aligned ($N = 10, G = 9$ and $N = 32, G = 31$ among them). The published construction is exact precisely when N and G are both powers of two.

The corrected construction. Cut $\mathrm{FBT}(N)$ at depth $d = \lceil \log_2 G \rceil$ instead of partitioning index ranges. The cut frontier — the nodes at depth d, together with any leaf shallower than d — roots disjoint complete subtrees that cover every leaf; assign those subtrees to the G ranks; each rank evaluates its subtrees by the canonical recurrence of Def. 2 under the enumeration of Def. 3; the cross-rank stage evaluates the internal nodes above the frontier, exactly $2^d - 1$ combines at depth d — the $\log_2 G$-depth all-reduce the published version wanted and did not have. The proof obligation is one sentence, and we state it rather than assert the conclusion a second time: every node of $\mathrm{FBT}(N)$ is still computed exactly once, by the same recurrence and with the same operand binding (Def. 4), so the multiset of FADD operations and the dependency order among them are those of $\mathrm{FBT}(N)$ itself, and the root is $R_{\mathrm{canonical}}(N, L)$ for every G. Enumeration discharges it over the same space that refutes the published version: every one of those 44,850 pairs is exact in FP64, with a cross-rank stage of the stated shape at each. The frontier-to-rank assignment does not affect the root — the tree fixes the order — so it is a free load-balancing choice, and the choice matters. Under a greedy largest-first deal the worst leaf-count ratio over those pairs is 1.993 and none exceeds 2 (1.969 over the subset with $G \le 64$); under the natural contiguous deal it reaches 3.53 and exceeds 2 on 13,446 of them. An export must therefore *specify* the greedy deal, and we report the bound as measured over this space rather than as proved. The proof itself belongs in the companion.

Prior art, and the residue. TBIK [14] (arXiv:2511.17826, 21 November 2025) predates this manuscript: it is prior art, not concurrent work, and we claim no priority against it. Its Appendix C, Theorem 1 proves by induction, over the same contiguous-block partition, exactly the theorem our step 1 asserted without proof, under exactly the power-of-two hypotheses that make it true, and §IX records what it then built and measured — a tree all-reduce costing 10% or less (its Appendix H.1, four H20 GPUs over NVLink at $\mathrm{TP} = 4$; its larger end-to-end figure bundles batch-invariant kernels and a custom MatMul for which this work has no analogue). The summary is uncomfortable and worth stating plainly: the correct part of pinned-FBT is what TBIK had already done, and the part TBIK did not do — arbitrary, non-power-of-two N — is the part that was wrong. What remains ours is the cut-aligned repair for arbitrary N under canonical heap-index leaf enumeration, together with the properties the export inherits from the ABI and TBIK does not test: invariance across network *topologies* rather than across tensor-parallel sizes inside one node, enforcement below the software stack, and presence-gated fail-stop (Property 1). That residue is why the repaired construction is worth measuring rather than merely citing.

Measured: the repair is invariant, and the checker can fail. We implemented both constructions on top of PyTorch's collectives and ran them on a single node with two NVIDIA RTX PRO 6000 Blackwell GPUs over PCIe (no NVLink) and 72 CPU cores — the node of §VII-B — with NCCL on the two GPUs and the CPU backend for larger world sizes. Over 32 cases spanning N from 3 to 4,096 (ten of the leaf counts not powers of two — the case TBIK does not cover, and the majority of the sweep), world sizes 2–64 (seven not powers of two), messages from 8 KiB to 128 MiB, and two frontier deals, every one of the 2,414 (case, world size, rank) cells returned a single 64-bit root, and that root equalled the single-process canonical $R_{\mathrm{canonical}}(N, L)$. The result is reportable only because the same checker demonstrably fails: run against the index-range construction as a pre-registered negative control, three of four cases broke as they must — at $N = 1{,}000$ and $N = 1{,}500$ it returned three distinct roots across world sizes $\{2, 3, 8\}$ and *none* of its 13 cells equalled $R_{\mathrm{canonical}}$. Where it agrees — ten of 13 cells at $N = 1{,}024$, all of them at $N = 7$ — it agrees by power-of-two coincidence, which is what the sweep above says analytically and this says with a running collective.

Measured: what it costs, and the answer is unfavourable. The statistic was fixed before the run: geometric-mean over-

head against stock `ncclAllReduce` at $G = 2$ in FP64 over the swept message sizes, worst case printed beside it, with a support branch at roughly 1.0–$1.3\times$ — the order of TBIK's published figure — and an explicit unfavourable branch. The measurement took the unfavourable branch. Over 20 configurations the geometric mean is $2.57\times$ stock, best $1.49\times$, worst $3.91\times$. The more flattering $2.40\times$ that also appears in the artifact's cost table is not this statistic — it pools 28 rows over both FP64 and FP32, which geomeans $2.02\times$ on its own — and survives there only as a recorded deviation from the pre-registration.

Two qualifications pull in opposite directions and neither changes the verdict. First, the sweep ran while an unrelated job saturated both GPUs, so the point estimate is *provisional*: substituting the best of the repeats at each configuration for the median gives $2.08\times$, so contention inflates the reported figure by about $1.24\times$. Both estimators are far outside the pre-registered support band, so the verdict does not depend on the correction — the point estimate does, and a re-run on a quiet machine is owed. Second, the overhead is not one quantity. At $G = 2$ the pinned tree moves $2M$ bytes, one directed send of the full vector plus a broadcast of it, against a bandwidth-optimal ring's M per rank: a factor of about two is algorithmic and no engineering removes it. The remainder is an export artefact — two dependent library calls issued from Python against one fused NCCL kernel — and it dominates at small messages ($3.29\times$ at 8 KiB, where the whole stock call takes $57\ \mu$s) while fading at large ones (1.49–$1.67\times$ between 8 MiB and 32 MiB). A two-chunk bidirectional variant separates the two terms: at 128 MiB it recovers part of the bandwidth term ($1.85\times$ against $2.21\times$) and below 1 MiB it is far worse ($8.1\times$), which is what doubling the number of dependent calls should do. We report no cost ratio from the CPU backend at all: at the larger world sizes its stock arm was itself contended and is not self-consistent — at $N = 1{,}000$, $G = 24$ it takes $3.7\times$ longer on 128 KiB than on 2 MiB, which no correct baseline does — so those cells bound nothing, and the clean re-run is queued and not reached.

What that means, and what it does not. The result contradicts no claim this paper makes and prices one it already predicts: §VIII-B names bandwidth-critical inner loops as the regime where the ABI is the wrong choice, and $2.57\times$ on a PCIe pair is that prediction with a number on it. What we claim for pinned-FBT is a correctness property, measured above; what we do not claim is that it is cheap. It should not be set beside TBIK's 10%: that figure is four H20 GPUs over NVLink on an uncontended node, a different fabric measured a different way. On this evidence the export is usable where reshape reproducibility is the requirement and the collective is not the bottleneck, and not inside a bandwidth-critical loop. Closing the non-algorithmic part of the gap is an engineering problem; the algorithmic factor of two at $G = 2$ is what the tree costs.

The broader point. Fabric-level and software-level enforcement can discharge the same structural obligation, but the record above is what it looks like when a construction is believed to discharge it and does not. Reduction order is the interface; that it is enforced rather than intended is the part a reader can check.

IX. Related Work

Relative to every axis below, what we claim is the conjunction stated in §I — reduction order as a size-indexed compiler/fabric contract, enforced in hardware with a fail-stop failure mode, invariant across network topologies, with the IEEE-754 FP64 operator unchanged — and not any single conjunct, each of which has a precedent here.

A. Reproducible Arithmetic

Compensated summation [9], [16], [17] reduces the error of a single reduction but does not make it reproducible: the same values compensated in a different order, or with a different number of lane accumulators, still differ in the last bit, so both sources survive it.

Order-independent accumulators do give reproducibility, and at cluster scale. ReproBLAS [10] carries a binned accumulator whose sum does not depend on order and ships MPI reduction operators, so the result is the same whatever reduction-tree shape MPI chooses; ExBLAS [11] combines a long accumulator with floating-point expansions, and Collange et al. [29] analyse the parallel reduction directly; Arteaga, Fuhrer and Hoefler [12] build bit-reproducible distributed reductions for a production weather code, invariant under process count to $16{,}384$ processes at under 10% overhead for large arrays; Balaji and Kimpe [36] study the same question for MPI reduction operations; Ahrens, Demmel and Nguyen [37] give the current algorithms for the binned family. All of them make the result independent of order by making the operator effectively associative. That is a strictly stronger guarantee than ours in one respect — it holds across implementations, not just across reshapes of one implementation — and weaker in another: it changes the arithmetic, so the answer is not the IEEE-754 FP64 sum any particular order would produce, and it needs a wide accumulator datatype. Our ABI leaves the arithmetic alone and constrains the order instead; the two are complementary, and a binned accumulator obeying the same operand-binding contract would inherit Claim 1 unchanged.

In hardware that line becomes exact accumulation: Kulisch's complete accumulator [38], exact accumulation and dot-product units in silicon [39], [40], and the posit quire [41]. It is the obvious alternative to what we do, and we decline it for three reasons. *Silicon*: an accumulator covering the FP64 product exponent range is thousands of bits of architected state per accumulation context, against the 64 bits an FP64 adder produces, replicated per lane and live across context switches. *Fidelity*: its intermediates are not IEEE-754 FP64 values and there is one rounding, at the end, so no partial sum a conventional reducer would compute is observable. *Interoperability*: the answer is the correctly rounded sum, which agrees with other exact accumulators and disagrees with every FP64 reduction tree, ours included. We keep an ordinary adder, and pay for it with a guarantee that is self-consistency under reshape rather than agreement with an external canonical form (§I).

B. Fixed-Order Reproducibility in Deployed Stacks

The closest prior work keeps the FP64 operator and pins the order. Intel oneMKL's strict conditional-numerical-reproducibility mode produces bitwise-identical `?gemm`, `?trsm` and `?symm` regardless of the number of threads and no matter how the inputs are partitioned [13]: a shipping library with operator-preserving invariance under a change of resource count, within one node. TBIK [14] imposes one fixed full binary tree shared by intra- and inter-GPU reduction, proves the resulting invariance for power-of-two configurations, and demonstrates bitwise-identical inference across tensor-parallel sizes in production serving stacks, the tree collective itself costing about 10% or less. RepDL [42] pursues bit-level reproducible training and inference through correct rounding and order invariance. Our ABI is the same idea — one fixed tree, indexed by problem size — and differs on the axes Table I records: arbitrary rather than power-of-two leaf counts, invariance across network topologies rather than within one node's tensor-parallel dimension, enforcement in the fabric rather than in a kernel library, and a fail-stop failure mode. We read TBIK as independent corroboration of the export path of §VIII-D: a fixed reduction tree is deployable in a production stack at a cost its users accepted.

Production collectives are deterministic only within a deployment. NCCL and RCCL fix the (algorithm, protocol) pair at communicator initialisation from architecture, node count, rank count and message size [5], [15], so pinning them fixes the order for one $(G, \text{topology})$ and no more. Shanmugavelu et al. [43] survey the resulting variability across HPC and deep-learning stacks and measure it on current accelerators.

C. Collective-Scheduling Compilers

TACCL [7], SCCL [6], and BLINK [33] compile performance-optimised schedules for collective operations. They share with SPRS the compiler-centric view of the collective and differ in objective (throughput versus reproducibility) and in output (a schedule versus an ABI contract); none states a cross-topology bitwise invariance property, nor could it, since their schedules are $(G, \text{topology})$-dependent by design. Our compiler is architecturally closest to this line, but contributes no new search algorithm: the point is that a size-indexed tree yields the invariance property *regardless* of which winning schedule the search emits. Communication-optimal tree constructions [24], [44] are subject to the same remark — they optimise at fixed $(G, \text{topology})$ and do not survive reshape.

D. In-Network Reduction

SHARP [30] reduces inside InfiniBand switches, and pins operand order to do so: each aggregation node's context includes an order of calculation, and the reduction fires only once the requests are held in a predefined deterministic order [31]. That is our instinct — fix the order in the fabric — with a different index. The specification is explicit that reproducibility holds "for a given application-to-system mapping", i.e. per deployment, where ours is indexed by N and survives the deployment changing. It also prices the discipline in hardware, offering a non-reproducible mode that saves aggregation buffering by a factor of $n/2$ in the number of inputs n — independent evidence that reproducibility in a switch is paid for structurally, which is how we price it in §VII-E. SwitchML [45] aggregates in programmable switches whose ALUs are integer-only, carrying gradients in 32-bit fixed point with adaptive scaling on the workers; its order-insensitivity therefore comes from integer associativity, like the accumulators above, and not from pinning an order.

The novelty here is framing, not results: SHARP already achieves per-deployment determinism, and we claim no advantage in the speed or cost of the reduction itself — only an index that survives a change of deployment, which a per-mapping order cannot without standardising switch hierarchies across sites.

E. Deterministic Execution and Deterministic Accelerators

In the architecture literature "determinism" usually means schedule determinism, not numerical reproducibility, and the gap between them is exactly our Source-1/Source-2 split. DMP [46] makes multithreaded execution deterministic by constraining the interleaving; the arithmetic is untouched. Groq's TSP [32] is the closest fabric-level determinism contract in the literature and a direct precedent for our framing — routing is software-scheduled, there is no dynamic arbitration, and the compiler owns when every packet moves, which is a shipping demonstration that a compiler/fabric determinism contract is viable. It is equally the clearest illustration of what schedule determinism does not buy: the TSP paper makes no bitwise claim, its determinism evidence is a latency histogram over repeated inferences, and its all-reduce is a three-stage hierarchical collective keyed to the packaging hierarchy, so the reduction order is a function of how many chips sit in a node — Source 1 in full force. Work co-authored at the same vendor lists cross-chip reproducibility as future work [43].

The same caution applies to accelerators cited for determinism generally. The TPU's "deterministic execution model" is a statement about 99th-percentile response time [47], and neither that paper nor Cambricon [48] makes a reproducibility claim; where per-chip determinism is advertised it concerns timing, and chip-to-chip aggregation remains the collective library's business. Our contribution sits at that layer: not per-chip determinism, which any accelerator can provide, but invariance of the chip-to-chip aggregation when the set of chips changes.

F. NoC Design

Dally and Towles' text [26] is the foundational reference for virtual channels, credit flow control and the deadlock-avoidance discipline our router is built to support; §IV states which parts of that discipline the present compiler discharges. Our NoC is otherwise a textbook design, with one deliberate choice: topology is a runtime parameter of adjacency and routing tables, not an RTL artefact, which is what lets a single Verilog instantiation exercise Claim 1 across six topologies.

X. LIMITATIONS

We enumerate limitations explicitly to set the scope of the work. The last item is not a limitation of the same kind as the others, and we mark it as such: it bounds the evaluation *method* rather than the artifact, and it is the part of this section we would most want a reader building a similar campaign to take away.

L0 — One unbuildable configuration, and why it is a defect rather than a budget. Hardware validation covers 39 of the 40 instances. The exception is `maxp_mod_fat` (N=8192, G=4096, fat-tree), whose five attempted testbenches all hit the 28,800 s wall-clock cap for 58.9 core-hours and no settled result — but wall-clock was not the binding constraint, and we correct an earlier account that said it was. The instance instantiates 4,192 routers while the adjacency-configuration interface carries a 12-bit router id (`noc_system.sv:67--70`) whose top code is the disconnected sentinel, so routers above 4,094 alias onto low ids and every one of the instance's 12,288 adjacency writes touches an out-of-range id. Replayed through the RTL's own truncation and range guards, the emitted configuration installs 8,223 links of which none match the intended fat-tree and delivers none of 900 sampled leaf-to-leaf routes, all 900 of which deliver on the untruncated fabric. No budget could have completed that configuration, so 58.9 core-hours bought a run that could not settle. We have since repaired the truncation — the emitter now sizes the router-id field to the instance — and re-elaborated, which settles what the earlier account could only assume. Across three attempts under two `xelab` debug configurations, two ran elaboration to completion and exited zero reporting the snapshot built — the larger taking 2 h 53 m at 64 GiB of peak resident memory and writing 6.5 GiB of snapshot database — and zero of the three wrote a simulation kernel, so there is nothing for the simulator to load. We do not assert a root cause: a tool that exits zero having written no output is misreporting and we have not established why. The exclusion is therefore a truncation defect in the emitter as shipped and, once that is repaired, a tool limit at this design point; it is not a wall-clock budget and not a property of the fabric. The N=8192 column of Table V rests on the two remaining $(G, \text{topology})$ points.

L0a — The denominator, and the runs that are not in it. We report the attempted count as well as the passing one. On the 39 instances, 2,844 distinct ABI-compliant images were attempted, 2,822 settled and passed, and 22 did not settle. All but one of the 22 are fabric watchdog fail-stops — their retained per-PE counters stop at a last retirement plus exactly `WATCHDOG_MAX` = 20,000 cycles, far inside the testbench budget — and the remaining one is a simulator wall-clock kill. Non-settling runs are not evidence for Claim 1 and we do not count them as such; they are the campaign's only observation of Property 1 on unmutated compiler output (§VI-E). Coverage of the configuration space is likewise narrower than the space we advertise: the 180 assignment/mapping pairs enumerated on each instance are 0.82% of the 21,960 (α, μ, r) triples, refinement is fixed at `None` in every measured configuration, and the remaining axis is covered by the orthogonality argument of §V-F rather than by measurement. One leaf vector is evaluated per N, and eight of the sixteen leaf counts carry a single $(G, \text{topology})$ point, so at those N nothing is witnessed across reshape.

L0b — Configuration-load bypass: stated, measured, and withdrawn. Route-table initialisation in every run we report is performed by hierarchical assignment at time zero rather than by the clocked `write_route()` protocol, because the latter costs $\mathcal{O}(G^2)$ simulated cycles and dominates wall-clock at large G; every row of every released hardware-results file carries a `_BYPASS` error code. An earlier version of this paper attached a fidelity band to that substitution — a fraction of runs bit-exact, the remainder some way lower in cycles — and, unable to re-derive it, withdrew the band without putting anything in its place. We have since measured it. Re-running every passing configuration at $G \leq 64$ under both paths on the same RTL gives the same `PASS`/`FAIL` verdict, the same 64-bit root and the same cycle count on 1,326 of 1,326 comparable points — 100% bit-exact, no exceptions — and 100% again under an alternative rule for choosing among repeated records (1,306 points). The only effect of clocked loading is a deterministic configuration prologue of exactly one clock per route-table entry (10.0 ns at our testbench period, with minimum, maximum and median all equal), which completes before `program_start` and therefore enters no reported cycle count. A negative control confirms the clocked path genuinely exercises routing: corrupting the whole route table flips `PASS` to `FAIL` on 24 of 24 runs. The uncertainty that route-table loading contributes to any cycle figure in this paper is therefore measured, and it is zero. What remains is a scope limit rather than a fidelity one: the clocked path was exercised up to $G = 64$, so the configuration protocol itself is uncharacterised at the largest G in the suite.

L0c — The FADD is combinational, and we report no timing closure. `fp64_add` is unpipelined: alignment, a 54-bit add, leading-zero detection, normalisation and rounding all resolve within one cycle. A stateless adder is what makes the arithmetic a pure function of its operand pair, which Claim 1 needs, but it is not a realistic synthesis target. We have run no synthesis and report no $F_{\max}$, slack or utilisation, so we make no frequency claim anywhere. Pipelining would change *when* a `COMPUTE` releases its scoreboard bit, not which operand pair it reads: cycle counts would move, the roots in Table V would not. Every cycle figure here belongs to the one-cycle-ALU model.

L0d — One reduction per program. Scoreboard bits are monotone within a program and are cleared only at `program_start`, so the fabric executes one reduction per reset, which is what our evaluation measures. A workload running thousands of training steps would need a reset between steps or epoch-tagged scoreboard bits; the latter does not touch the ABI, but we have not implemented it and claim no multi-pass throughput numbers.

L1 — Research-prototype envelope; no silicon. All results come from cycle-accurate RTL simulation. FPGA bring-up wrappers exist for $G \in \{2, 4\}$ but need IP and constraints we do not redistribute, and we hold no board logs, so no claim in this paper rests on silicon execution and we make none about timing closure, area or power at production-die scale.

L1b — No measured same-fabric overhead at non-power-of-two scale. We report no η: the non-deterministic baselines of §VI-B were not implemented, so the ABI's cost is priced structurally (§VII-E) and anchored only on distance to Ω, which upper-bounds what any unconstrained reducer could recover. On most of the suite that is not a gap: §VI-B shows $\eta = 1$ by construction on the 31 instances whose N and G are both powers of two, where the ABI schedule and recursive doubling are the same DAG, so a measurement there would confirm a theorem rather than test one. The residual is the 8 instances whose N or G is not a power of two. There the isomorphism does not hold, the ABI may cost something we have not priced, and no baseline was built to price it. An earlier version of this paper called the missing baseline the largest gap in the evaluation and the first thing it would close; on the evidence of §VI-B that was an overstatement, and we withdraw it.

L2 — No head-to-head with production collectives. We do not benchmark SPRS against NCCL, RCCL or oneCCL on GPU silicon; doing so would conflate the ABI's cost with the cost of a research prototype against production silicon. The premise that motivates the paper — that reshaping a production collective changes its bits — is measured rather than assumed (§VII-B), but that measurement is a statement about reduction order on one two-GPU node and above $G{=}2$ it runs on the CPU backend; it is not a performance comparison and no number here derives a timing claim from it. One head-to-head does exist and is reported: the software export of §VIII-D against stock `ncclAllReduce` on that node. It prices the export, at $G{=}2$ on PCIe, and not the fabric, and it is unfavourable.

L3 — Synthetic leaf values. Equation (9) is a designed-to-exercise-the-bits distribution, not a workload trace, and it is strictly positive at every index — see L13, which is a consequence of that second property rather than of the first. A workload of real gradients or partial sums exercises value regimes we do not, especially subnormals, which we flush.

L4 — FTZ+RNE subset, and the deviation that was not intended. §III-G documents the intended subset; workloads depending on subnormals, multiple rounding modes or the IEEE signed-zero rule diverge from reference at the affected bits. A fourth deviation was not intended: the adder we first submitted did not round to nearest-even on effective subtraction. It is repaired and revalidated (§IV-J) and no number in this paper moves. We keep the item rather than deleting it because how the defect escaped an exhaustive campaign is L13's subject and is the more transferable finding.

L5 — What the instance suite cannot support. The 40-instance suite covers the $(N, G, \text{topology})$ space unevenly (Appendix D), and three consequences bind every scale-dependent statement we make. N and G are confounded across the suite by construction (Spearman $\rho = 0.871$, $p = 5.6 \times 10^{-13}$), so no result here separates the influence of problem size from that of worker count. Of the 23 populated (size $\times$ topology) cells, 18 hold two instances or fewer, so no cell-level recommendation is derivable and we make none. And nothing in the selection-rule analysis survives Holm correction over the 36 exploratory association tests we ran (two survive Benjamini–Hochberg); we report the unconditional portfolio precisely because it needs no such association to hold (§VII-G).

L6 — The C-mem margin is thin, and past it the failure is silent. Peak DMEM occupancy over the validated configurations reaches 491 of a hard 512 slots, a margin of 21. The bound is a compile-time obligation, not a fabric check: on a constructed $\mathrm{FBT}(512)$ instance with $G = 2$, peak occupancy is 522 slots, the run terminates normally with no `P_ERROR`, and it returns a wrong root — two *different* wrong roots under two packet-delivery orders (Remark III-C). Claim 1 holds only inside that ceiling, and the campaign came within 21 slots of it.

L7 — Head-of-line blocking at large G on low-bisection topologies. Store-and-forward with a single FIFO per VC (§IV-J); wormhole forwarding is planned.

L8 — Validated at one point of a parameterised design. The fabric instantiates two virtual channels per port, two compute-node links and $k/2$ fat-tree core switches; the compiler uses one of each (§IV-J). Every defect we know of in the RTL is in an unused half. Results here characterise the parameterisation we exercised, and do not transfer to a second VC, a second link, or a multi-core-switch fat-tree without revalidation.

L9 — Soft-enforced single-writer. C-sw is a compile-time post-condition over the emitted image, not a fabric-level veto on a second write to a set scoreboard slot (§IV-J). A one-line RTL extension would close the gap.

L10 — No deadlock-freedom property, and no virtual-channel discipline. The emitted routing tables are minimal-distance rather than dimension-ordered, and the channel-dependency graph built from them is cyclic on every ring instance (cycle length G) and every torus instance (length 6–32). No escape channel exists: the compiler assigns no virtual channel and every packet travels on `VC0`, and the router's egress buffer is a single FIFO shared across channels, so a second VC would not be an escape even if one were assigned (§IV-H). C-route therefore claims destination-correctness and loop-freedom only. Deadlock is prevented by neither compiler nor fabric; it is converted into a recorded fail-stop by the watchdog, which is where most of L0a's non-settling runs come from.

L11 — Mutation testing is limited to two classes, and its verdicts are not all detections. We inject address substitution and leaf corruption only; scoreboard-set, forwarding-network and credit-return mutations are untested, so our figures are a lower bound on the harness's sensitivity. Within the two classes, oracle non-vacuity rests on 61–65 of 107 verdicts rather than on all 107: in the remainder the substituted address is one nobody writes, so the dependent `COMPUTE` stalls on the presence gate and the flag is a fail-stop, not a value comparison (§VII-C).

L12 — No formal proof of Claim 1. Our argument is by code inspection and dynamic assertion, not mechanised proof; the Alloy/Kami path of §III-I is future work.

L13 — Two defect classes this evaluation cannot see, and why we report that rather than patch it quietly. The items above bound what the campaign measured. This one bounds what it *could* have measured, and it is a finding about the method, not an apology for the artifact.

First, the campaign cannot exercise the FADD's effective-subtraction path at all. Leaf values come from (9), which is strictly positive at every index, so both operands of every FADD in all 2,822 runs carry the same sign and effective subtraction is never asserted. The defect that lived on that path (L4) therefore could not have appeared in any number this paper reports — not at this campaign size and not at any other, because the blindness does not lie along the axis the campaign enumerates: all 7,200 compilations share one leaf generator, so exhausting the (α, μ) grid explores none of it.

Second, and closer to the bone, the software oracle against which every one of the 2,822 roots is compared is not an independent model of IEEE-754 addition. It is a transcription of the same adder (`sprs_core.py::_fp64_add_bits`), and it reproduced the same defect bit for bit. Its agreement with the RTL is therefore true and evidentially empty on exactly the inputs where the RTL was wrong — and we can now say how empty: on the as-submitted pair the two agree on all 62,320 vectors of the acceptance test while both disagree with correctly-rounded addition on 15,717 of them (§IV-J). A differential test is only as strong as the independence of its two sides, and ours were not independent.

We state these together because the pattern is more useful than either instance. Both are cases of an evaluation that cannot detect a class of defect *by construction* — in the first the stimulus excludes the path, in the second the reference implements the defect — and neither is visible from inside the campaign: no pass rate, coverage figure or additional instance would have exposed either, and both were found by reading the arithmetic against an external standard. The transferable lesson, which applies to our own exhaustiveness argument as much as to anyone's, is that enumerating a parameter space exhaustively says nothing about the axes that space does not span, and the two axes that mattered here — operand sign, and the independence of the oracle — were not parameters of the campaign at all. The discipline we would adopt next time is to state, before the campaign and for each property under test, which stimulus dimension exercises it and which reference is independent of the implementation. The adder is repaired and revalidated in §IV-J; the method fix is this item.

XI. Conclusion

We have argued that cluster-reshape non-determinism in floating-point reductions is an *interface* problem rather than a library problem, and that the place to fix it is the contract between a collective-scheduling compiler and its fabric. Reduction order, treated as a size-indexed ABI over a canonical full binary tree, is orthogonal to G, topology, assignment, mapping, refinement and schedule, and yields cross-topology bitwise invariance: the 64-bit root is a pure function of N and the leaf vector. What we claim is a conjunction, not a first — that invariant, enforced at the compiler/fabric interface with a fail-stop failure mode, across network topologies rather than resource counts alone, over an unmodified IEEE-754 FP64 operator, for arbitrary non-power-of-two N. Six compiler obligations and nine fabric invariants suffice to argue it: one compiler pass, SU-ordered DMEM allocation, *constructs* the single-writer discipline the argument needs rather than assuming it, and three fabric mechanisms — scoreboard monotonicity, misroute drop, and the write-arbitration retry latch — do the rest. Connectivity repair, which an earlier account of this work called load-bearing, is available and unused: operand binding is by node identity and does not care whether a worker's task set is connected (§V-B).

We instantiated the ABI as SPRS. On 40 instances spanning $N \in [8, 8192]$, $G \in [2, 4096]$ and six direct-network topologies, the campaign enumerated the refinement-free face of the compiler's configuration space exhaustively and RTL-validated every distinct image it produced: of 2,844 attempted on 39 hardware-validated instances, 2,822 settled with a root bit-identical to the canonical one and 22 did not settle, all but one a watchdog fail-stop. That face is 0.82% of the $\approx$22,000 triples per instance, with refinement fixed at `None`; the rest is covered by the orthogonality argument, not by measurement. An injected-fault campaign establishes that the zero-divergence result is non-vacuous on 61–65 of 107 verdicts, the remainder being the halt outcome Property 1 predicts. We price the ABI structurally rather than against a non-deterministic baseline, which remains the principal open item in the evaluation.

Two negative results are worth carrying forward alongside the positive one. The compiler search this campaign was built around is not a ranking problem: the minimising configuration is a tie set on 38 of 39 instances, and what replaces the ranking — a fixed portfolio of 4 assignment algorithms that reaches the measured optimum on 38 of 39 held-out instances, beating every regime map we could fit over $(N, G, \text{topology})$ — is both cheaper and better supported than the distribution it replaces. It also transfers: refitted with an entire topology withheld and scored only on the topology it never saw, its mean out-of-sample regret is 1.0391 (95% bootstrap interval $[1.0025, 1.0885]$), lower than that of a rule which conditions on topology and does see it in training (1.0488). Conditioning on the axis where conditioning looks most obvious is the thing that loses. And two defect classes in our own artifact are invisible to this evaluation *by construction* (§X, L13): we found them by reading the arithmetic, not by running more configurations. An evaluation that enumerates a space exhaustively can still be blind along an axis that the space does not span, and that blindness is not observable from inside the campaign.

The framing suggests a more portable form of the argument: *pinned-FBT*, a software-only export of the ABI discipline onto existing collective libraries (§VIII-D). The construction we published there is exact only when N and G are both powers of two; the arbitrary-N case needs the cut-aligned variant given in that section, and both are now measured on a production collective. The repair is bitwise reshape-invariant — all 2,414 (case, world size, rank) cells swept returned the canonical root $R_{\text{canonical}}(N, L)$ — and the same checker, run against the published construction as a pre-registered negative control, broke on three of four cases. Its cost is unfavourable and the statistic was fixed before the run: a geometric mean of $2.57\times$ stock `ncclAllReduce` at $G{=}2$ in FP64 (best $1.49\times$, worst $3.91\times$), far outside the pre-registered support band and provisional besides, since the sweep ran

under GPU contention. A factor of about two of it is the algorithmic price of the pinned tree at $G{=}2$ and no engineering removes it. Reshape reproducibility is therefore available in software where the collective is not the bottleneck, and is the wrong purchase inside a bandwidth-critical loop — which is what §VIII-B predicted before it was priced. Fabric-level and software-level enforcement discharge the same structural obligation at very different prices; what makes either one checkable is that the reduction order is enforced rather than intended.

APPENDIX A
ISA AND PACKET FORMAT

The instruction word is 64 bits:

$$\underbrace{\texttt{opcode}}_{4\text{ b}} \,\|\, \underbrace{\texttt{aux}}_{4\text{ b}} \,\|\, \underbrace{\texttt{dest}}_{16\text{ b}} \,\|\, \underbrace{\texttt{addr\_a}}_{16\text{ b}} \,\|\, \underbrace{\texttt{addr\_b}}_{16\text{ b}} \,\|\, \underbrace{\texttt{reserved}}_{8\text{ b}}.$$

The `opcode` field is Table II. Two source fields are required: `COMPUTE` reads `DMEM[addr_a]` and `DMEM[addr_b]`, which is the operand pair the ABI binds. For `SEND`, `dest` is the *destination-side* DMEM address (`target_addr` in the packet), `addr_a` is the local source, and `addr_b` carries the destination PE identifier.

The `aux` field is four bits: `aux`[3:1] selects the ALU operation and `aux`[0] is a legacy link hint. Egress port selection is *not* an instruction field — no small fixed field could address a router's egress ports, which range up to $P = 66$ across the suite (Table III). The port is resolved in hardware by the emitting router's `route_table`[`dest_gpu`] lookup, which is what makes topology a configuration input rather than a property of the emitted code.

This 64-bit encoding is what Table IV costs (at the 512×64 design target; the elaborated depth is per-instance, §IV-I) and what the companion document specifies field-for-field.

The 96-bit packet format is Fig. 3. The `dest_node_id` field is cluster-wide at 16 bits, but the header is not the binding limit on cluster size: the adjacency-configuration interface carries a 12-bit router id, so router ids above 4,094 alias silently (§IV-I, L0).

APPENDIX B
ALGORITHM CATALOGUE

Table VIII lists every assignment algorithm in the portfolio with its family and its *champion-class membership*: the share of HW-validated instances on which the algorithm produces a configuration whose measured cycle count equals the instance minimum. Membership is invariant to the tie structure of §VII-F; a win count is not, which is why this column replaces the winner shares reported in earlier versions. The four algorithms marked $\star$ are the derived portfolio of §VII-G. Rows are ordered by family and deliberately not by membership: the algorithms below the top four are not distinguishable from one another on a suite of this size and should be read as a block.

Three caveats. A zero share does not mean an algorithm is useless — it means the algorithm never matched the instance minimum here, and several zero-share algorithms are close seconds on many instances. Membership is over the 39 HW-validated instances except where an algorithm is not evaluable on all of them, in which case the denominator is the eligible subset; `ExactBB` in particular is size-gated out beyond $N \approx 64$, so its figure reflects unavailability rather than defeat. Finally, the mapping (S3) stage is absent from the table by design: all ten mapping algorithms attain the champion cycle count on 27 of 39 instances, so no per-mapping share is reportable (§VII-G).

TABLE VIII
ASSIGNMENT (S2) ALGORITHMS BY FAMILY, WITH CHAMPION-CLASS MEMBERSHIP ACROSS THE 39 HW-VALIDATED INSTANCES. $\star$ MARKS THE FOUR MEMBERS OF THE DERIVED PORTFOLIO (§VII-G).

Algorithm	Family	In champion class
`Frederickson`*	tree DP	12.8%
`Lukes`*	tree DP	48.7%
`ClHEFT`	hierarchical	0.0%
`ParSub`*	hierarchical	41.0%
`SubPack`	hierarchical	5.1%
`DKway`	graph partition	2.6%
`RecBisect`	graph partition	17.9%
`CPFD`	clustering	0.0%
`Chrétienne`	clustering	0.0%
`DSC`*	clustering	7.7%
`MLfm`	multilevel	2.6%
`Centroid`	geometric	2.6%
`LvlHLF`	structural	0.0%
`Spine`	structural	2.6%
`CPOP`	list scheduling	0.0%
`DFSseed`	list scheduling	2.6%
`HEFT`	list scheduling	0.0%
`ExactBB`	exact/B&B	2.6%

Provenance for the algorithms implemented from the literature: list scheduling follows HEFT and CPOP [19] and Hu's level-based bound [49]; tree dynamic programming follows Lukes [20]; clustering follows DSC [22]; multilevel and graph partitioning follow Karypis and Kumar [23] with Fiduccia–Mattheyses refinement [21]. The remaining methods are our own constructions or straightforward adaptations, and are marked as such in the released source rather than attributed to prior work.

APPENDIX C
WORKED INVARIANCE EXAMPLE AT $N = 4$

We trace a single execution at $N = 4$ on two different topologies to show the invariance mechanism concretely.

Let $L = \langle 1.0, 2.0, 3.0, 4.0\rangle$. The canonical tree $\mathrm{FBT}(4)$ has internal nodes $\{0, 1, 2\}$ and leaves $\{3, 4, 5, 6\}$ with $\mathrm{idx}(3) = 0, \mathrm{idx}(4) = 1, \mathrm{idx}(5) = 2, \mathrm{idx}(6) = 3$. The canonical post-order is $\pi_4^{\text{internal}} = \langle 1, 2, 0\rangle$, so:

$$R_{\text{canonical}}(4, L) = \mathrm{fadd}_{\text{rtl}}(\mathrm{fadd}_{\text{rtl}}(1.0, 2.0), \mathrm{fadd}_{\text{rtl}}(3.0, 4.0)) = 10.0.$$

Configuration A: $G = 2$, linear. $\alpha = \{3 \to 0, 4 \to 0, 1 \to 0, 5 \to 1, 6 \to 1, 2 \to 1, 0 \to 0\}$ (PE 0 owns the left subtree, PE 1 the right subtree and sends its sub-root to PE 0). Emitted instructions:

- PE 0: `GEN(dest=1, val=1.0); GEN(dest=2, val=2.0); COMP(dest=3, addr_a=1,`

TABLE IX
INSTANCE-SUITE CELL COUNTS BY N-BUCKET AND TOPOLOGY. ROWS AND COLUMNS BOTH SUM TO THE 40-INSTANCE TOTAL.

N-bucket	Lin.	Ring	Mesh	Torus	Fat-tree	Hyp.	Total
8–32	1	2	1	1	1	1	7
33–256	1	1	1	3	2	1	9
257–1K	2	2	1	3	2	2	12
1K–4K	–	1	1	2	3	2	9
8K	–	–	–	1	1	1	3
Total	4	6	4	10	9	7	**40**

`addr_b=2)`; receive at addr=4; `COMP(dest=5, addr_a=3, addr_b=4)`.

- PE 1: `GEN(dest=1, val=3.0); GEN(dest=2, val=4.0); COMP(dest=3, addr_a=1, addr_b=2); SEND(src=3, dest=0, addr=4)`.

The operand bindings at PE 0's second `COMP` are $(\mathrm{DMEM}_0[3], \mathrm{DMEM}_0[4]) = (\mathrm{fadd}_{\mathrm{rtl}}(1.0, 2.0), \mathrm{fadd}_{\mathrm{rtl}}(3.0, 4.0))$, whose FADD gives 10.0. *Root result:* 10.0.

Configuration B: $G = 4$, 2D mesh with PE indices $\{(0,0),(0,1),(1,0),(1,1)\}$. $\alpha = \{3 \to (0,0), 4 \to (0,1), 1 \to (0,0), 5 \to (1,0), 6 \to (1,1), 2 \to (1,0), 0 \to (0,0)\}$ scatters every leaf to its own PE; `SEND`s fan leaves to internal nodes; `SEND`s fan internal-node sub-roots to the root. Operand bindings at internal `COMP`s are identical by construction: the `tree.children(v)` function is topology-agnostic. *Root result:* 10.0 (the same 64 bits, by Proposition 1).

What changes between A and B: (i) the number and identity of the PEs; (ii) the number of `SEND`s; (iii) the cycle count; (iv) which routers carry which packets. What does not change: the operand pair bound at each internal `COMP`, the order in which the scoreboard-gated adds commit, and therefore the final 64 bits.

APPENDIX D
INSTANCE COVERAGE AND PER-TOPOLOGY CELL COUNTS

Table IX reports per-(N-bucket, topology) cell counts in the 40-instance suite, and Table X the measured xsim wall-clock per instance. Both are generated directly from the instance manifest and the measured CSV (`scripts/gen_coverage.py`), which asserts that both margins sum to 40 before emitting the table. Cells marked – are unpopulated; cells with ≤ 2 instances are labelled "thin" in the results. Coverage is deliberately uneven — 23 of the 26 populated cells are thin and 4 cells are empty — which is the basis for limitation L5. A stratified extension to fill thin cells is in progress.

Table X reconciles exactly to the campaign totals quoted in §VII-A: 2,822 passing runs and 821.5 core-hours, at the stated 99 s median and 5.51 h maximum. `maxp_mod_fat` does not appear — it has no passing runs, and the 58.9 core-hours it consumed are accounted in §X.

TABLE X
MEASURED XSIM WALL-CLOCK PER INSTANCE, OVER PASSING RUNS ONLY. "MED." IS THE MEDIAN PER-TESTBENCH WALL-CLOCK, "MAX" THE LONGEST SINGLE TESTBENCH.

Instance	N	G	Topo.	TBs	Med. (s)	Max (h)	Total (core-h)
`tiny_dense_1d`	8	2	Lin.	12	12	0.00	0.0
`tiny_mod_ring`	8	4	Ring	16	13	0.00	0.1
`npow2n_tiny_ring`	10	4	Ring	17	14	0.00	0.1
`npow2g_small_mesh`	16	5	Mesh	21	14	0.00	0.1
`small_bal_fat`	32	8	Fat-tree	41	15	0.00	0.2
`small_dense_hc`	32	4	Hyp.	18	14	0.00	0.1
`small_mod_2d`	32	16	Torus	43	18	0.01	0.2
`npow2_tiny_hc`	50	7	Hyp.	25	15	0.00	0.1
`npow2_small_torus`	100	10	Torus	33	17	0.01	0.2
`med_bal_2d`	128	16	Torus	46	25	0.01	0.3
`med_dense_mesh`	128	8	Mesh	33	19	0.01	0.2
`med_extreme_1d`	128	4	Lin.	23	15	0.00	0.1
`med_full_torus`	128	128	Torus	106	99	0.03	2.9
`med_mod_fat`	128	32	Fat-tree	54	39	0.01	0.6
`med_mod_ring`	128	64	Ring	109	47	0.01	1.4
`npow2_med_fat`	200	12	Fat-tree	44	21	0.01	0.3
`large_unbal_torus`	500	32	Torus	98	41	0.01	1.1
`large_bal_fat`	512	64	Fat-tree	62	72	0.02	1.2
`large_bal_mesh`	512	64	Mesh	103	56	0.02	1.6
`large_dense_2d`	512	16	Torus	57	24	0.01	0.4
`large_dense_hc`	512	32	Hyp.	70	35	0.01	0.7
`large_extreme_1d`	512	8	Lin.	55	17	0.01	0.3
`large_mod_ring`	512	256	Ring	109	196	0.07	6.1
`vlarge_bal_fat`	1024	128	Fat-tree	74	131	0.04	2.7
`vlarge_dense_2d`	1024	64	Torus	95	61	0.02	1.6
`vlarge_dense_hc`	1024	32	Hyp.	67	44	0.01	0.8
`vlarge_extreme_1d`	1024	16	Lin.	80	29	0.01	0.7
`vlarge_mod_ring`	1024	512	Ring	109	479	0.18	14.9
`vlarge_unbal_hc`	1500	64	Hyp.	104	69	0.03	2.0
`vlarge2_bal_fat`	2048	256	Fat-tree	75	420	0.15	9.1
`vlarge2_dense_2d`	2048	128	Torus	126	111	0.04	3.8
`vlarge2_mod_ring`	2048	512	Ring	109	510	0.18	15.3
`vlarge_bal_mesh`	2048	256	Mesh	127	231	0.08	8.3
`max_unbal_fat`	3000	128	Fat-tree	77	191	0.08	4.5
`max_bal_2d`	4096	1024	Torus	131	1172	0.43	43.5
`max_dense_hc`	4096	512	Hyp.	108	529	0.20	16.3
`max_mod_fat`	4096	2048	Fat-tree	92	16086	5.51	416.1
`maxp_bal_2d`	8192	2048	Torus	141	4641	2.05	191.3
`maxp_dense_hc`	8192	1024	Hyp.	112	2350	0.89	72.5
Total				**2822**			**821.5**

APPENDIX E
SELECTION-BIAS ABLATION

The campaign simulated every distinct program image (§VI-E). This appendix asks what a cost-constrained campaign that had instead promoted a top-K-by-`makespan` shortlist to hardware would have missed, since software `makespan` and measured `hw_cycles` need not rank configurations alike.

For each HW-validated instance we compute the Spearman correlation between the software makespan ranking and the measured cycle ranking over all its configurations, and the *leakage* at cutoff K: the fraction of the true top-K-by-cycles that falls outside the top-K-by-makespan.

K	instances	median ρ	median leakage
10	39	0.95	0.30
15	37	0.95	0.20
30	32	0.95	0.10

The correlation is high ($\rho \approx 0.95$ at the median), but leakage at the operating point is not negligible: at $K = 15$ a median 20% of the genuine top-15 would have been missed by a makespan-chosen shortlist, and even at the more permissive $K = 30$ cutoff 10% is still missed. The practical reading is that software makespan is a good *filter* and a poor *selector* — adequate for deciding what to simulate, not for deciding what won.

None of this touches Claim 1, which quantifies over every HW-validated configuration rather than a selected subset: no

TABLE XI
PER-INSTANCE MINIMUM MEASURED CYCLE COUNT ACROSS THE HW-VALIDATED SUITE. CYCLE COUNTS ARE THE MAXIMUM OVER ALL PEs' PERF_TOTAL COUNTERS; CYC/Ω IS THE DISTANCE TO THE COMPOSITE LOWER BOUND OF §VI-A.

N	G	Topology	cycles	cyc/Ω
8	2	linear	44	1.83
8	4	ring	40	3.08
10	4	ring	42	2.33
16	5	mesh	52	2.17
32	4	hypercube	85	1.77
32	8	fat-tree	80	3.33
32	16	torus	62	3.88
50	7	hypercube	87	1.81
100	10	torus	138	2.30
128	4	linear	265	1.38
128	8	mesh	156	1.62
128	16	torus	107	2.23
128	32	fat-tree	110	4.58
128	64	ring	159	4.97
128	128	torus	95	4.13
200	12	fat-tree	187	1.83
500	32	torus	192	2.00
512	8	linear	518	1.35
512	16	torus	287	1.49
512	32	hypercube	178	1.85
512	64	fat-tree	153	3.19
512	64	mesh	144	3.00
512	256	ring	341	4.16
1024	16	linear	537	1.40
1024	32	hypercube	298	1.55
1024	64	torus	189	1.97
1024	128	fat-tree	166	3.46
1024	512	ring	1043	7.10
1500	64	hypercube	304	2.11
2048	128	torus	200	2.08
2048	256	fat-tree	183	3.81
2048	256	mesh	195	4.06
2048	512	ring	707	4.81
3000	128	fat-tree	337	2.34
4096	512	hypercube	162	3.38
4096	1024	torus	247	6.86
4096	2048	fat-tree	185	8.04
8192	1024	hypercube	173	3.60
8192	2048	torus	399	9.07

shortlist was applied, and the leakage above is the cost we avoided by not applying one.

APPENDIX F
PER-INSTANCE RESULTS

Table XI gives, for every HW-validated instance, the minimum measured cycle count and its ratio to the composite lower bound Ω. Both quantities are invariant to the champion tie structure of §VII-F; the assignment and mapping algorithms that attain the minimum are not, and are deliberately omitted — on 38 of 39 instances the minimum is attained by a median of ten configurations, so naming one of them would report an arrival order rather than a measurement. The complete per-configuration record is released as `live_hw_results_p1_maxpe.csv` in the artefact.

APPENDIX G
REPRODUCIBILITY CHECKLIST

- **RTL.** 13 SystemVerilog files totalling 3,743 lines (10 fabric sources plus 3 FPGA bring-up wrappers, the latter needing IP and constraints we do not redistribute); tested under Vivado xsim 2025.2 on Linux.
- **Compiler.** Python 3.9+; deterministic under insertion-ordered dict semantics; RNG seeds pinned at 42 (`random.seed`, `numpy.random.seed`).
- **Testbench.** Per-instance testbench emitted by `emit_sv_testbench`; SHA-256 dedup at the testbench string level.
- **Oracle.** Python $\mathrm{fadd}_{\mathrm{rtl}}$ mirror at `sprs_core.py::_fp64_add_bits` — a transcription of `fp64_add.sv`, not an independent model (L13). Bit-identical to the RTL on 160,800 vectors, zero mismatches (`tests/fp64/test_sw_rtl_equiv.py`); the same script on the as-submitted pair is the negative control.
- **Checks.** The per-run verdict is the testbench's comparison against the compile-time `EXPECTED` root (§III-I); the C-* post-conditions are a static audit over the released images (§V-E). We ship no SVA suite.
- **Mutation tests.** `experiments/mutation/` contains the address-substitution and leaf-corruption harness.
- **Canonical leaf-value formula.** For the FP64 path (`op = ALU_FADD`), leaf i carries $L_i = 1000(i{+}1) + \big((37i{+}13) \bmod 100\big)/100 + 0.007$, chosen so every leaf has non-trivial mantissa bits and reassociation is therefore observable. SHA-256 of the `make_leaf_val` source: `20aae138844da00e165cd0a490f7960d 3dd806f11ed51950bc3be40600800bb1`.
- **Invariance-hash per instance.** Every HW-validated configuration at the same N produces a 64-bit root whose SHA-256 matches the column 4 of Table V.

REFERENCES

[1] A. Sergeev and M. D. Balso, "Horovod: Fast and easy distributed deep learning in TensorFlow," *arXiv preprint arXiv:1802.05799*, 2018. [Online]. Available: https://arxiv.org/abs/1802.05799

[2] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, "PyTorch distributed: Experiences on accelerating data parallel training," in *VLDB*, 2020. [Online]. Available: https://dl.acm.org/doi/10.14778/3415478.3415530

[3] J. Dongarra and P. Luszczek, "HPC challenge: Design, history, and implementation highlights," *Contemporary High Performance Computing: From Petascale Toward Exascale*, pp. 13–30, 2013. [Online]. Available: https://www.taylorfrancis.com/chapters/edit/10.1201/9781351104005-2/hpc-challenge-design-history-implementation-highlights-jack-dongarra-piotr-luszczek

[4] R. Thakur, R. Rabenseifner, and W. Gropp, "Optimization of collective communication operations in MPICH," *International Journal of High Performance Computing Applications*, vol. 19, no. 1, pp. 49–66, 2005. [Online]. Available: https://doi.org/10.1177/1094342005051521

[5] NVIDIA, "NCCL: Optimized primitives for collective multi-gpu communication," https://github.com/NVIDIA/nccl, 2018–2025. [Online]. Available: https://github.com/NVIDIA/nccl

[6] Z. Cai, Z. Liu, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, and O. Saarikivi, "Synthesizing optimal collective algorithms," in *PPoPP*, 2021. [Online]. Available: https://dl.acm.org/doi/10.1145/3437801.3441620

[7] A. Shah, V. Chidambaram, M. Cowan, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, O. Saarikivi, and R. Singh, "TACCL: Guiding collective algorithm synthesis using communication sketches," in *NSDI*, 2023. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/shah

[8] R. Rabenseifner, "Optimization of collective reduction operations," in *ICCS 2004*, ser. LNCS, vol. 3036. Springer, 2004, pp. 1–9. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-540-24685-5_1

[9] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2002. [Online]. Available: https://epubs.siam.org/doi/10.1137/1.9780898718027

[10] J. Demmel and H. D. Nguyen, "Fast reproducible floating-point summation," in *21st IEEE Symposium on Computer Arithmetic (ARITH)*. IEEE, 2013, pp. 163–172. [Online]. Available: https://ieeexplore.ieee.org/document/6545904

[11] R. Iakymchuk, S. Collange, D. Defour, and S. Graillat, "ExBLAS: Reproducible and accurate BLAS library," *Numerical Reproducibility at Exascale (NRE) Workshop, SC*, 2015. [Online]. Available: https://hal.science/hal-01202396

[12] A. Arteaga, O. Fuhrer, and T. Hoefler, "Designing bit-reproducible portable high-performance applications," in *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2014.

[13] Intel Corporation, "Conditional numerical reproducibility in the oneAPI Math Kernel Library," Intel oneMKL Developer Guide.

[14] Zhang, Ding, Yuan, Liu, Mao, Xing, and Liu, "Deterministic inference across tensor parallel sizes that eliminates training–inference mismatch," arXiv:2511.17826, 2025. [Online]. Available: https://arxiv.org/abs/2511.17826

[15] AMD, "RCCL: ROCm collective communication library," https://github.com/ROCmSoftwarePlatform/rccl, 2019–2025. [Online]. Available: https://github.com/ROCmSoftwarePlatform/rccl

[16] W. Kahan, "Pracniques: Further remarks on reducing truncation errors," in *Communications of the ACM*, vol. 8, no. 1, 1965, p. 40. [Online]. Available: https://dl.acm.org/doi/10.1145/363707.363723

[17] S. M. Rump, T. Ogita, and S. Oishi, "Accurate floating-point summation part I: Faithful rounding," *SIAM Journal on Scientific Computing*, vol. 31, no. 1, pp. 189–224, 2008. [Online]. Available: https://epubs.siam.org/doi/10.1137/050645671

[18] IEEE, "IEEE standard for floating-point arithmetic," IEEE Std 754-2019 (Revision of IEEE 754-2008), pp. 1–84, 2019, doi: 10.1109/IEEESTD.2019.8766229. [Online]. Available: https://ieeexplore.ieee.org/document/8766229

[19] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002. [Online]. Available: https://ieeexplore.ieee.org/document/993206

[20] J. A. Lukes, "Efficient algorithm for the partitioning of trees," *IBM Journal of Research and Development*, vol. 18, no. 3, pp. 217–224, 1974. [Online]. Available: https://ieeexplore.ieee.org/document/5391463

[21] C. M. Fiduccia and R. M. Mattheyses, "A linear-time heuristic for improving network partitions," in *19th Design Automation Conference*, 1982, pp. 175–181. [Online]. Available: https://dl.acm.org/doi/10.1145/800263.809204

[22] T. Yang and A. Gerasoulis, "DSC: Scheduling parallel tasks on an unbounded number of processors," *IEEE Transactions on Parallel and Distributed Systems*, vol. 5, no. 9, pp. 951–967, 1994. [Online]. Available: https://ieeexplore.ieee.org/document/311795

[23] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998. [Online]. Available: https://doi.org/10.1137/S1064827595287997

[24] E. Jeannot, G. Mercier, and F. Tessier, "Process placement in multicore clusters: Algorithmic issues and practical techniques," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 4, pp. 993–1002, 2014. [Online]. Available: https://doi.org/10.1109/TPDS.2013.104

[25] W. J. Dally and C. L. Seitz, "Deadlock-free message routing in multiprocessor interconnection networks," *IEEE Transactions on Computers*, vol. C-36, no. 5, pp. 547–553, 1987.

[26] W. J. Dally and B. P. Towles, *Principles and Practices of Interconnection Networks*. San Francisco, CA: Morgan Kaufmann, 2004. [Online]. Available: https://www.sciencedirect.com/book/9780122007514/principles-and-practices-of-interconnection-networks

[27] P. Patarasuk and X. Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," *Journal of Parallel and Distributed Computing*, vol. 69, no. 2, pp. 117–124, 2009. [Online]. Available: https://doi.org/10.1016/j.jpdc.2008.09.002

[28] P. Sanders, J. Speck, and J. L. Träff, "Two-tree algorithms for full bandwidth broadcast, reduction, and scan," in *Parallel Computing*, vol. 35, no. 12, 2009, pp. 581–594. [Online]. Available: https://doi.org/10.1016/j.parco.2009.09.001

[29] S. Collange, D. Defour, S. Graillat, and R. Iakymchuk, "Numerical reproducibility for the parallel reduction on multi- and many-core architectures," *Parallel Computing*, vol. 49, pp. 83–97, 2015.

[30] R. L. Graham, D. Bureddy, P. Lui, H. Rosenstock, G. Shainer, G. Bloch, D. Goldenberg, M. Dubman, S. Kotchubievsky, V. Koushnir, L. Levi, A. Margolin, T. Ronen, A. Shpiner, O. Wertheim, and E. Zahavi, "Scalable hierarchical aggregation protocol (SHArP): A hardware architecture for efficient data reduction," in *COMHPC*, 2016, pp. 1–10. [Online]. Available: https://ieeexplore.ieee.org/document/7836669

[31] Mellanox Technologies, "Aggregation protocol," U.S. Patent 10,284,383 B2, 2019.

[32] D. Abts *et al.*, "A software-defined tensor streaming multiprocessor for large-scale machine learning," in *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2022.

[33] G. Wang, S. Venkataraman, A. Phanishayee, J. Thelin, N. Devanur, and I. Stoica, "Blink: Fast and generic collectives for distributed ML," in *MLSys*, 2020. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2020/hash/cd3a9a55f7f3723133fa4a13628cdf03-Abstract.html

[34] D. Jackson, *Software Abstractions: Logic, Language, and Analysis*, revised ed. MIT Press, 2012. [Online]. Available: https://mitpress.mit.edu/9780262016865/software-abstractions/

[35] J. Choi, M. Vijayaraghavan, B. Sherman, A. Chlipala, and Arvind, "Kami: A platform for high-level parametric hardware specification and its modular verification," in *ICFP*, 2017. [Online]. Available: https://dl.acm.org/doi/10.1145/3110268

[36] P. Balaji and D. Kimpe, "On the reproducibility of MPI reduction operations," in *IEEE International Conference on High Performance Computing and Communications (HPCC)*. IEEE, 2013.

[37] P. Ahrens, J. Demmel, and H. D. Nguyen, "Algorithms for efficient reproducible floating point summation," *ACM Transactions on Mathematical Software*, 2020.

[38] U. W. Kulisch, *Computer Arithmetic and Validity: Theory, Implementation, and Applications*. de Gruyter, 2013.

[39] F. de Dinechin, B. Pasca, O. Creţ, and R. Tudoran, "An FPGA-specific approach to floating-point accumulation and sum-of-products," in *International Conference on Field-Programmable Technology (FPT)*. IEEE, 2008.

[40] J. Koenig, D. Biancolin, J. Bachrach, and K. Asanović, "A hardware accelerator for computing an exact dot product," in *IEEE Symposium on Computer Arithmetic (ARITH)*. IEEE, 2017.

[41] J. L. Gustafson and I. Yonemoto, "Beating floating point at its own game: Posit arithmetic," *Supercomputing Frontiers and Innovations*, 2017.

[42] Xie, Zhang, and Chen, "RepDL: Bit-level reproducible deep learning training and inference," arXiv:2510.09180, 2025. [Online]. Available: https://arxiv.org/abs/2510.09180

[43] S. Shanmugavelu, M. Taillefumier, C. Culver, O. Hernandez, M. Coletti, and A. Sedova, "Impacts of floating-point non-associativity on reproducibility for HPC and deep learning applications," arXiv:2408.05148, 2024. [Online]. Available: https://arxiv.org/abs/2408.05148

[44] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[45] A. Sapio, M. Canini, C.-Y. Ho, J. Nelson, P. Kalnis, C. Kim, A. Krishnamurthy, M. Moshref, D. R. K. Ports, and P. Richtárik, "Scaling distributed machine learning with in-network aggregation," in *NSDI*, 2021. [Online]. Available: https://www.usenix.org/conference/nsdi21/presentation/sapio

[46] J. Devietti, B. Lucia, L. Ceze, and M. Oskin, "DMP: Deterministic shared memory multiprocessing," in *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2009.

[47] N. P. Jouppi, C. Young, N. Patil, D. Patterson *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *ISCA*, 2017. [Online]. Available: https://dl.acm.org/doi/10.1145/3079856.3080246

[48] S. Liu, Z. Du, J. Tao, D. Han, T. Luo, Y. Xie, Y. Chen, and T. Chen, "Cambricon: An instruction set architecture for neural networks," in *ISCA*, 2016. [Online]. Available: https://ieeexplore.ieee.org/document/7551409

[49] T. C. Hu, "Parallel sequencing and assembly line problems," *Operations Research*, vol. 9, no. 6, pp. 841–848, 1961. [Online]. Available: https://doi.org/10.1287/opre.9.6.841